

PROXIAN T ACADEMY

Professional Reference Series

AI, ML, and Generative AI Interview Questions

A Comprehensive Reference for Engineers, Researchers, and Architects

Pavan R

First Edition, Revision 1.1 | 2026

AI, ML, and Generative AI Interview Questions

by Pavan R

Published by Proxiant Academy, San Francisco Bay Area.

First Edition, Revision 1.1, 2026.

Copyright © 2026 Proxiant Academy. All rights reserved.

Reference Identifier (RID): PA-AIMLIQ-2026-01

No part of this work may be reproduced or transmitted in any form or by any means without the prior written permission of the publisher, except for brief quotations in critical reviews and certain other non-commercial uses permitted by copyright law.

While every precaution has been taken in the preparation of this book, the publisher and author assume no responsibility for errors or omissions, or for damages resulting from the use of the information contained herein.

Trademarked product names mentioned in this book are the property of their respective owners. The use of such names is for editorial purposes only and does not imply endorsement.

Preface

This textbook is a curated, deep reference for technical interview preparation across the modern AI stack: neural networks, classical NLP, transformers, large language models, embeddings, retrieval-augmented generation, fine-tuning, vector databases, LLMOps, and evaluation. The answers are written to interview depth: enough detail to demonstrate real understanding, with diagrams and code snippets where helpful.

Each chapter follows a question-and-answer format. Use it as a study companion, a quick reference during interview prep, or a daily reading for the working engineer.

How to read this book: pick a chapter, work through the questions, attempt your own answer first, then read the provided answer. Pay particular attention to the tradeoffs and design choices, which are the focus of senior-level interviews.

Table of Contents

Chapter 1: Artificial Neural Networks (15 questions)

Chapter 2: Classical Natural Language Processing (13 questions)

Chapter 3: Transformers and Extended Architectures (32 questions)

Chapter 4: Fundamentals of Large Language Models (13 questions)

Chapter 5: Word and Sentence Embeddings (22 questions)

Chapter 6: Retrieval-Augmented Generation (RAG) and Multimodal RAG (34 questions)

Chapter 7: Fine-Tuning Large Language Models (26 questions)

Chapter 8: Vector Databases (10 questions)

Chapter 9: LLMOps and System Design (11 questions)

Chapter 10: Evaluation Methods (22 questions)

Chapter 11: Miscellaneous and Advanced Topics (21 questions)

Chapter 12: Supplementary Topics, Formulas, and Deep Dives (13 questions)

Chapter 1

Artificial Neural Networks

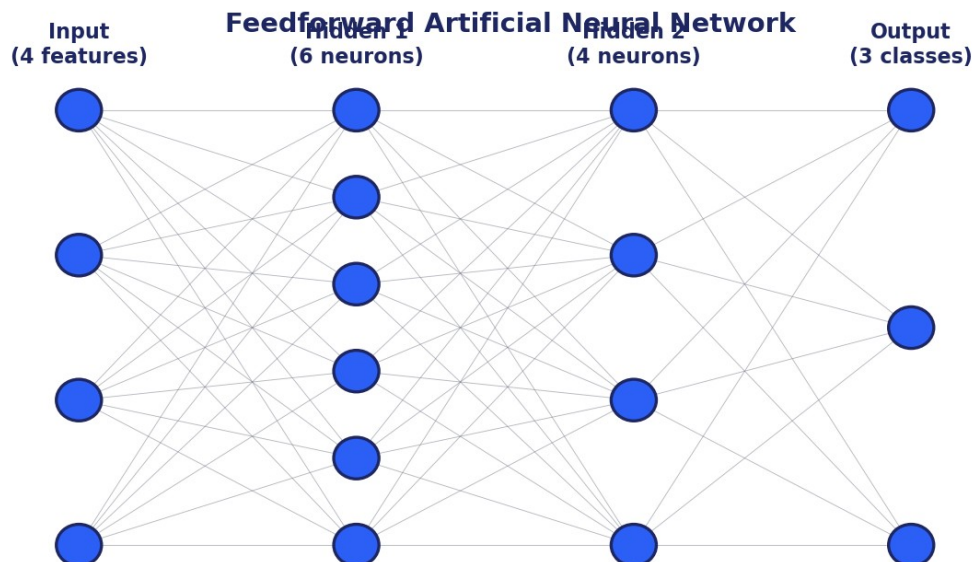
This chapter covers 15 questions on artificial neural networks.

Q1. What is an Artificial Neural Network, and how does it work?

An Artificial Neural Network (ANN) is a computational model loosely inspired by biological neurons. It is a directed graph of simple units called neurons arranged in layers: an input layer that receives features, one or more hidden layers that transform them, and an output layer that produces a prediction.

Each neuron computes a weighted sum of its inputs plus a bias term, then applies a non-linear activation function: $a = f(w \cdot x + b)$. Stacked layers let the network build up progressively more abstract representations of the input. The training process adjusts the weights and biases so that the output matches the target on a training set, using gradient descent driven by backpropagation.

The universal approximation theorem says a sufficiently wide single-hidden-layer network with a non-linear activation can approximate any continuous function on a compact set. In practice, depth is what gives modern networks their power: each layer composes simpler features into more abstract ones, and depth is far more parameter-efficient than width for the same expressive capacity.



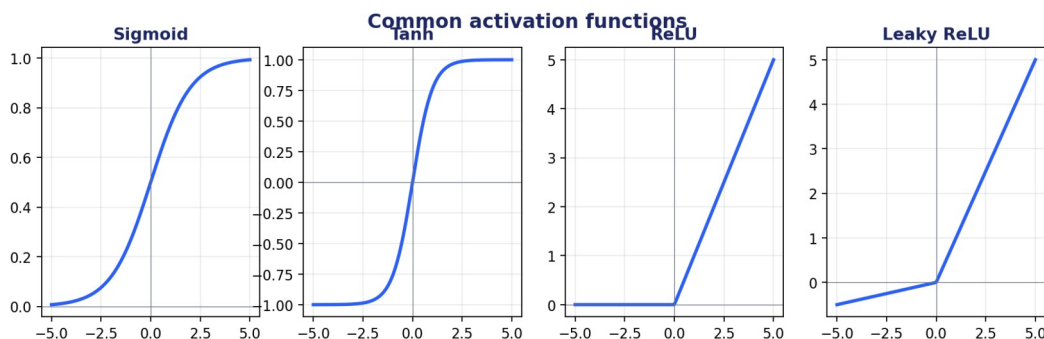
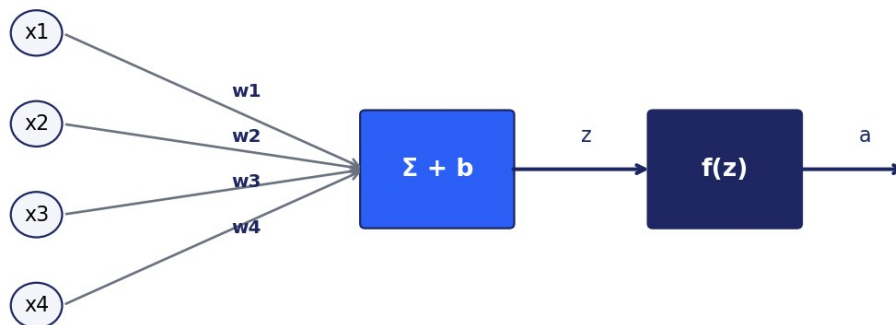
Q2. What are activation functions, what types exist, and why are they used in neural networks?

An activation function is a non-linear transformation applied to the weighted sum computed by a neuron. Without non-linear activations, a stack of layers would collapse into a single linear transformation, no matter how many layers were used. Non-linearity is what gives neural networks the capacity to model complex patterns.

Common activation functions: Sigmoid $\sigma(x) = 1/(1+\exp(-x))$, bounded to (0, 1), historically used for binary outputs but suffers from saturation and vanishing gradients. Tanh, bounded to (-1, 1), zero-centered, better than sigmoid for hidden layers but still saturates. ReLU $\max(0, x)$ is the modern default for hidden layers: cheap, no saturation in the positive region, sparse activations. Leaky ReLU is ReLU with a small slope for negative inputs to avoid dead neurons. GELU and SiLU (Swish) are smoother variants used in transformers. Softmax normalizes a vector into a probability distribution over classes for the final layer of a classifier.

Choice guidance: ReLU or GELU for hidden layers, sigmoid for binary output, softmax for multiclass output. Tanh sees use in RNNs and certain attention components.

Single neuron: weighted sum, bias, activation



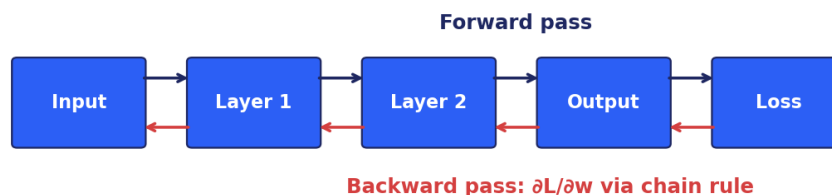
Q3. What is backpropagation, and how does it work in training neural networks?

Backpropagation is the algorithm for efficiently computing the gradient of a loss function with respect to every weight in a neural network. It is the chain rule of calculus applied systematically from the output back to the inputs.

The process has two passes. The forward pass computes the output of every layer given the input and stores intermediate activations. The backward pass starts from the loss at the output and propagates gradient information back layer by layer, computing $\partial L / \partial w$ for every weight by combining the upstream gradient with the local derivative at that layer.

These per-weight gradients are then used by an optimizer (SGD, Adam, etc.) to update the weights in the direction that reduces the loss. Modern deep learning frameworks (PyTorch, TensorFlow) compute backpropagation automatically via a computational graph and reverse-mode automatic differentiation, so you almost never write the backward step yourself. The cost of backpropagation is roughly the same as the forward pass, which is what makes training deep networks tractable.

Backpropagation: forward then backward



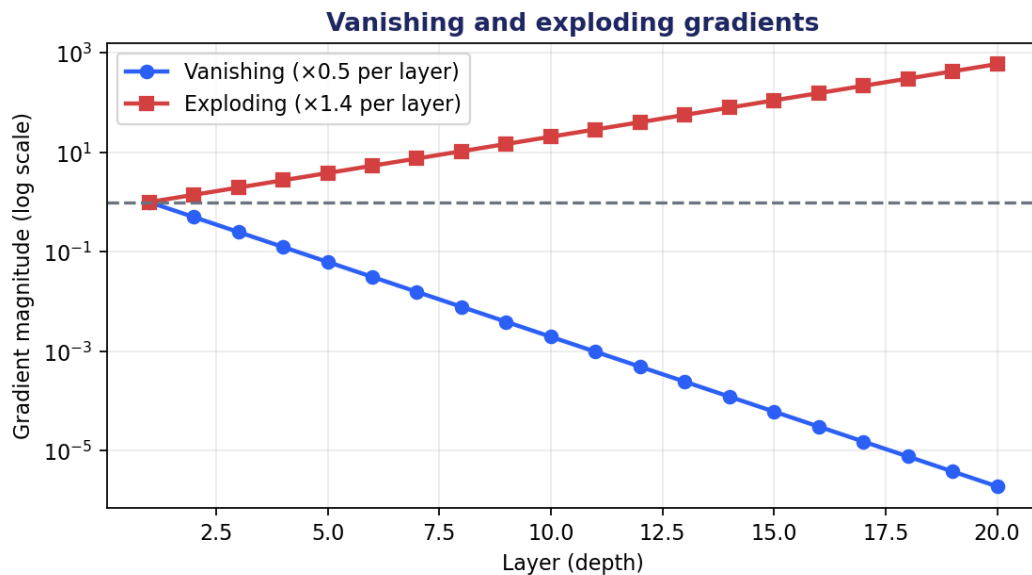
Q4. What is the vanishing and exploding gradient problem, and how does it affect training?

When backpropagation multiplies gradients through many layers, those products can shrink to near zero (vanishing) or blow up to very large values (exploding). With vanishing gradients, the lower layers learn extremely slowly because the update signal that reaches them is too small. With exploding gradients, weights oscillate or diverge, and training becomes unstable.

Vanishing is common with saturating activations like sigmoid or tanh in deep networks: their derivative is less than 1, and the product of many small numbers approaches zero. Exploding is common in RNNs unrolled over many timesteps with weight matrices whose largest singular value exceeds one.

Mitigations: use ReLU-family activations (derivative is 1 on the active side), use careful weight initialization (Xavier/Glorot for tanh-like, He for ReLU), use batch normalization or layer normalization, use residual connections (let gradients flow through identity paths), use gradient clipping (for explosion in RNNs), and

use architectures such as LSTM/GRU and transformers that are designed to keep gradient norms healthy across long sequences.



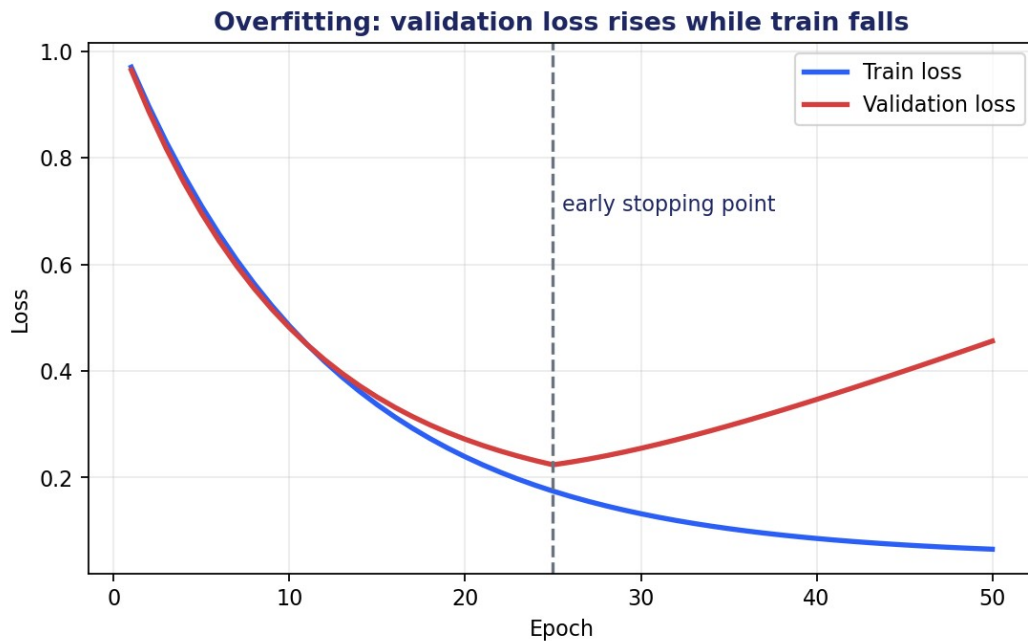
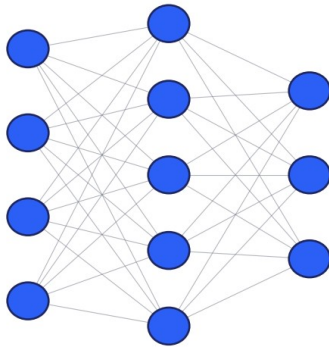
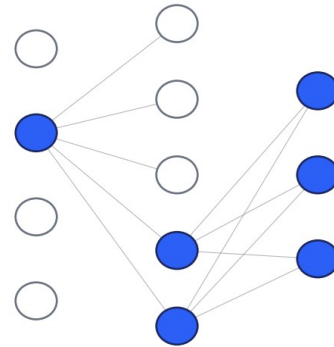
Q5. How do you prevent overfitting in neural networks?

Overfitting happens when a model fits the training set so precisely that it loses the ability to generalize to new data. The training loss keeps falling while the validation loss starts rising. The signs are easy to read; the fixes come in several complementary families.

Data-side fixes: collect more training data; apply data augmentation (random crops, flips, noise for images; back-translation and synonym replacement for text); use cross-validation to get a more honest estimate of generalization.

Model-side fixes: reduce model capacity (fewer layers, narrower layers); use weight regularization (L1 or L2 penalty on weights); apply dropout, which randomly zeros activations during training; use batch normalization or layer normalization to keep activations well-behaved; use early stopping based on validation loss.

Training-side fixes: use a held-out validation set and stop training when validation loss starts rising; ensemble multiple models; transfer learning from a pretrained model when the target dataset is small.

**Full network****With dropout (p=0.5)**

Q6. What is dropout, and how does it help train neural networks?

Dropout is a regularization technique where, during training, each neuron is independently dropped (its output set to zero) with some probability p (typically 0.2 to 0.5). At test time all neurons are active and their outputs are scaled by $(1 - p)$ to compensate for the larger sum (inverted dropout scales during training instead, leaving inference cleaner).

Why it helps: dropout prevents neurons from co-adapting too strongly. Each neuron must produce useful features regardless of which other neurons happen to be present in any given batch. The effect is roughly equivalent to training a large ensemble of subnetworks and averaging them at inference.

Practical notes: apply dropout after fully connected and certain convolutional layers; do not apply between batch norm and the next layer (the two regularizers interact badly); typical p is 0.5 for fully connected, 0.1

to 0.3 for convolutional and transformer blocks; modern transformer models use dropout sparingly and rely more on data scale and weight decay.

Q7. How do you choose the number of layers and neurons for a neural network?

There is no closed-form answer; the choice is empirical and constrained by the task, data size, and compute budget. The practical approach is to start small, establish a working baseline, then scale up while watching the validation loss.

Heuristics that work: for tabular data, two to four hidden layers with widths in the range 64 to 512 is a reasonable starting point. For images, use a published backbone (ResNet, EfficientNet, ViT) rather than designing from scratch. For text, use a published transformer with a known parameter count for your model size class.

Scaling: if the model underfits (train loss is high), add capacity. If it overfits (train loss low, val loss high), reduce capacity or add regularization. Use the validation curve as the signal. Architecture search and hyperparameter sweeps (Optuna, Ray Tune) can automate the search when the cost is justified.

Chinchilla scaling for large language models: parameter count and training tokens should scale together at roughly 20 tokens per parameter for compute-optimal training.

Q8. What is transfer learning, and when is it useful?

Transfer learning is the practice of reusing knowledge from a model trained on one task or dataset to improve performance on a related task. In modern deep learning this almost always means: take a model pretrained on a large general corpus, freeze or fine-tune part of it on your specific task.

It is useful when: (a) your target dataset is small, (b) the source task is related enough that low-level features transfer, (c) you cannot afford to train from scratch. Image classifiers fine-tune from ImageNet-pretrained backbones. NLP tasks fine-tune from BERT, RoBERTa, or instruction-tuned LLMs. Domain adaptation for medical imaging starts from a general-purpose vision backbone.

Typical workflow: load the pretrained model, replace the final task-specific head, optionally freeze early layers, train the new head and (optionally) the rest of the model with a small learning rate. PEFT methods like LoRA take this further by training only a tiny low-rank update on top of frozen weights.

Q9. What is a loss function, and how do you choose the right one?

A loss function is a scalar that measures how wrong a model's predictions are on a training example or batch. It is what the optimizer minimizes. The choice of loss is driven by the task type and the distribution you assume over your data.

Common choices: Mean Squared Error (MSE) for regression with Gaussian noise. Mean Absolute Error (MAE) for regression robust to outliers. Huber loss combines the two. Binary cross-entropy for binary classification. Categorical cross-entropy for multiclass classification. Negative log-likelihood (NLL) is the same as cross-entropy in most cases. Hinge loss for SVM-style classifiers. CTC loss for sequence alignment without explicit alignment labels. Triplet loss and InfoNCE for embedding learning. KL divergence for distributional matching.

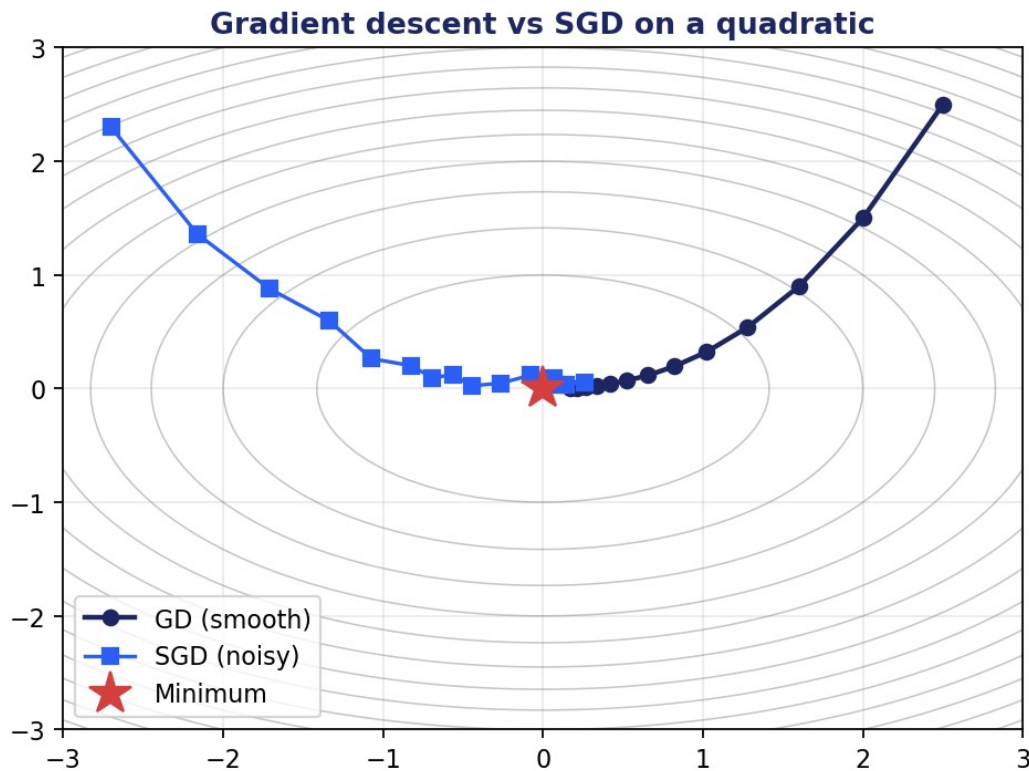
Choose by asking three questions: what does my model output (a number, a class, a distribution, an embedding)? What is the noise model in my data? What do I actually want to optimize at deployment? Aligning the loss with the deployment metric is more important than picking the fanciest loss.

Q10. Explain gradient descent and its variations: SGD and mini-batch gradient descent.

Gradient descent is an iterative optimization algorithm that updates parameters in the direction opposite the gradient of the loss. The update rule is $\theta \leftarrow \theta - \eta \cdot \nabla L(\theta)$, where η is the learning rate.

Batch gradient descent uses the gradient computed on the entire training set per step. Smooth convergence, but each step is expensive and the entire set must fit in memory. Stochastic gradient descent (SGD) uses one example per step. Cheap and noisy; the noise can help escape sharp minima but slows final convergence. Mini-batch gradient descent is the practical compromise: compute the gradient on a small batch (32 to 4096 examples) per step. GPU-friendly and noise-controlled.

Modern optimizers add adaptive learning rates and momentum: SGD with momentum, RMSProp, Adam, AdamW. Adam combines momentum with per-parameter adaptive learning rates and is the default for most deep learning workloads. AdamW decouples weight decay from the gradient update and is the better choice when regularization matters.



Q11. What is the role of the learning rate, and how do you tune it?

The learning rate η controls how large a step the optimizer takes in the direction opposite the gradient. Too small and training is needlessly slow; too large and the loss diverges or oscillates without converging.

Practical strategies: start with the standard default for your optimizer (e.g. $3e-4$ for Adam on most tasks, $1e-5$ to $5e-5$ for fine-tuning transformers). Use a learning rate range test (Smith): linearly increase η from very small to very large over a few hundred steps and plot loss; pick η around the steepest descent point. Use a schedule: warm-up (linearly increase η for the first few hundred steps), then decay (cosine, polynomial, or step). One-cycle learning rate works well for many tasks.

Symptoms of bad learning rates: loss is constant or jumps up immediately (too high); loss decreases extremely slowly (too low); loss oscillates after initial progress (could be too high, or batch size too small).

Q12. What are common neural network architectures, and when do you use each?

Multilayer Perceptron (MLP) / Feedforward: tabular data, small models, classical regression and classification.

Convolutional Neural Network (CNN): image classification, detection, segmentation. Examples: ResNet, EfficientNet, ConvNeXt.

Recurrent Neural Network (RNN), LSTM, GRU: sequence modeling when transformers are too expensive or context is short. Largely superseded by transformers for NLP.

Transformer: text, audio, vision, multimodal. BERT, GPT, T5, LLaMA, ViT, CLIP. The default for modern NLP and increasingly for vision.

Graph Neural Network (GNN): graph-structured data (molecules, social networks, knowledge graphs). GraphSAGE, GAT, GIN.

Autoencoder, Variational Autoencoder (VAE): unsupervised representation learning, anomaly detection, generation.

Generative Adversarial Network (GAN): image synthesis, super-resolution. Largely displaced by diffusion models for high-quality image generation.

Diffusion model: image, audio, and video generation. Stable Diffusion, DALL-E 3.

Mixture of Experts (MoE): scaling LLMs by activating only a subset of parameters per token. Mixtral, Switch Transformer.

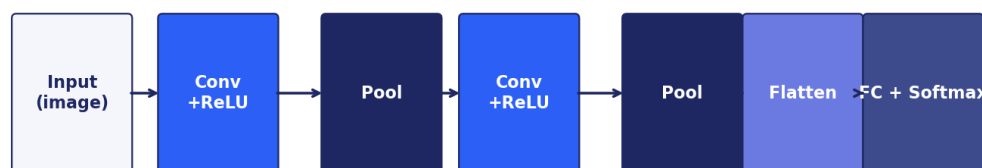
Q13. What is a CNN, and how does it differ from a feedforward ANN?

A Convolutional Neural Network (CNN) is built around two ideas: local receptive fields (each neuron sees only a small spatial neighborhood) and weight sharing (the same filter is applied at every spatial position). These ideas encode the prior that local pixel patterns matter and translation invariance should hold.

A CNN has three kinds of layers: convolutional layers (apply learned filters), pooling layers (downsample spatially), and fully connected layers (final classification head). A typical pipeline alternates conv and pool layers to build spatial hierarchy, then flattens and runs a few fully connected layers.

Differences from a plain MLP: an MLP would treat each pixel as an independent feature, ignoring spatial structure, and would need vastly more parameters. A CNN's weight sharing makes it parameter-efficient and translation-equivariant. Modern vision is dominated by CNNs and Vision Transformers, with ViTs winning at scale.

Typical CNN pipeline for image classification



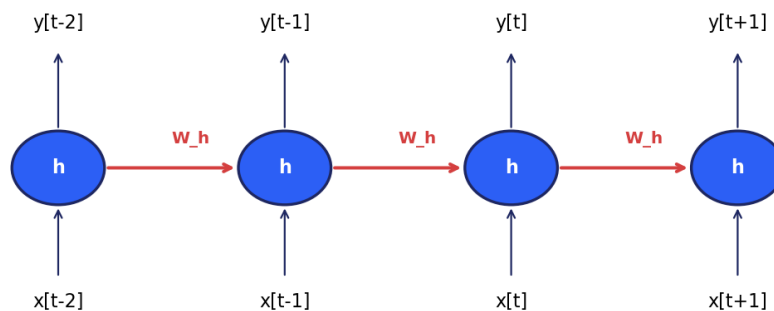
Q14. How does an RNN work, and what are its limitations?

A Recurrent Neural Network processes a sequence one element at a time, maintaining a hidden state that summarizes everything seen so far. At each step t : $h_t = f(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$, and the output y_t comes from h_t .

Variants: LSTM (Long Short-Term Memory) adds gates (input, forget, output) and a cell state to better preserve long-range dependencies. GRU (Gated Recurrent Unit) is a simpler version with two gates.

Limitations: training is inherently sequential (cannot parallelize over the time axis), which makes RNNs slow on long sequences. Gradients vanish or explode over long unrolls. The hidden state has fixed capacity, so very long contexts get compressed lossily. These limitations are exactly why transformers won the NLP race: transformers parallelize over tokens via attention, avoid sequential bottleneck, and handle long-range dependencies more directly.

Recurrent Neural Network unrolled in time



Q15. What are batch normalization and layer normalization, and when do you use each?

Both normalize activations to stabilize training, but along different axes. Batch normalization (BatchNorm) normalizes each feature across the batch dimension. Layer normalization (LayerNorm) normalizes each example across the feature dimension.

BatchNorm works well for CNNs and large batches but is sensitive to batch size and awkward for RNNs and variable-length sequences. LayerNorm has no batch dependence and is the default in transformers. RMSNorm is a recent simplification that drops the mean-centering and is used in LLaMA and similar models.

Benefits: faster convergence, allows higher learning rates, mild regularization effect, reduced sensitivity to initialization.

Chapter 2

Classical Natural Language Processing

This chapter covers 13 questions on classical natural language processing.

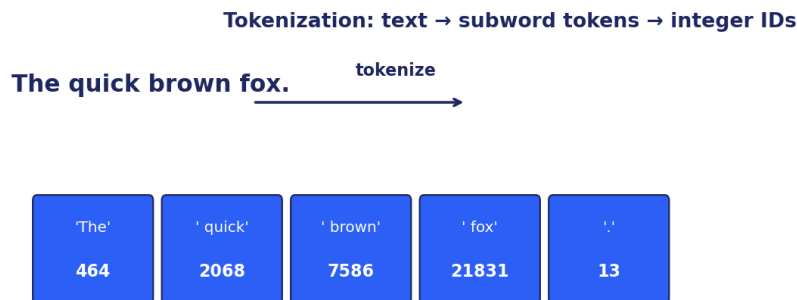
Q1. What is tokenization? Give the difference between lemmatization and stemming.

Tokenization is the process of splitting raw text into discrete units called tokens. The units may be words, subwords, characters, or sentences. Modern LLMs use subword tokenization (BPE, WordPiece, SentencePiece) to balance vocabulary size with the ability to represent rare and out-of-vocabulary words.

Stemming chops off suffixes to reduce a word to its root form using rule-based heuristics. 'Running' → 'run', 'studies' → 'studi'. Fast but produces non-words.

Lemmatization reduces a word to its dictionary form (lemma) using a vocabulary and morphological analysis. 'Better' → 'good', 'ran' → 'run'. Slower but produces real words and respects part of speech.

Modern NLP with subword tokenizers and large pretrained models rarely needs stemming or lemmatization explicitly; the tokenizer handles surface variation implicitly through shared subword units.



Q2. Explain Bag of Words (BoW) and its limitations.

Bag of Words represents a document as a vector of word counts, ignoring word order. The vocabulary is the union of words across the corpus; each document becomes a sparse vector where the i -th entry is the count of vocabulary word i .

Limitations: BoW loses all word order ('dog bites man' vs 'man bites dog' have identical representations). It cannot capture meaning: synonyms get separate dimensions, polysemous words collapse. The vocabulary

grows with the corpus, producing very high-dimensional sparse vectors. It cannot handle out-of-vocabulary words at test time.

Mitigations: n-grams capture limited local order; TF-IDF reweights rare words; hashing tricks bound the vocabulary size. The real fix is dense embeddings and neural models, which BoW preceded.

Q3. How does TF-IDF work, and how is it different from simple word frequency?

TF-IDF stands for Term Frequency $\times$ Inverse Document Frequency. It is a weighting scheme that gives higher scores to words that are frequent in a document but rare across the corpus.

$TF(t, d)$ = count of term t in document d (often normalized by document length). $IDF(t) = \log(N / df(t))$ where N is total documents and $df(t)$ is the number of documents containing t . $TF-IDF(t, d) = TF \times IDF$.

Difference from raw frequency: raw frequency lets common words ('the', 'is') dominate. TF-IDF down-weights them via IDF, so a word that appears 10 times in one document but in every document scores low. A word that appears 5 times in one document and almost nowhere else scores high. TF-IDF is still used as a strong baseline retriever (BM25 is its more refined cousin).

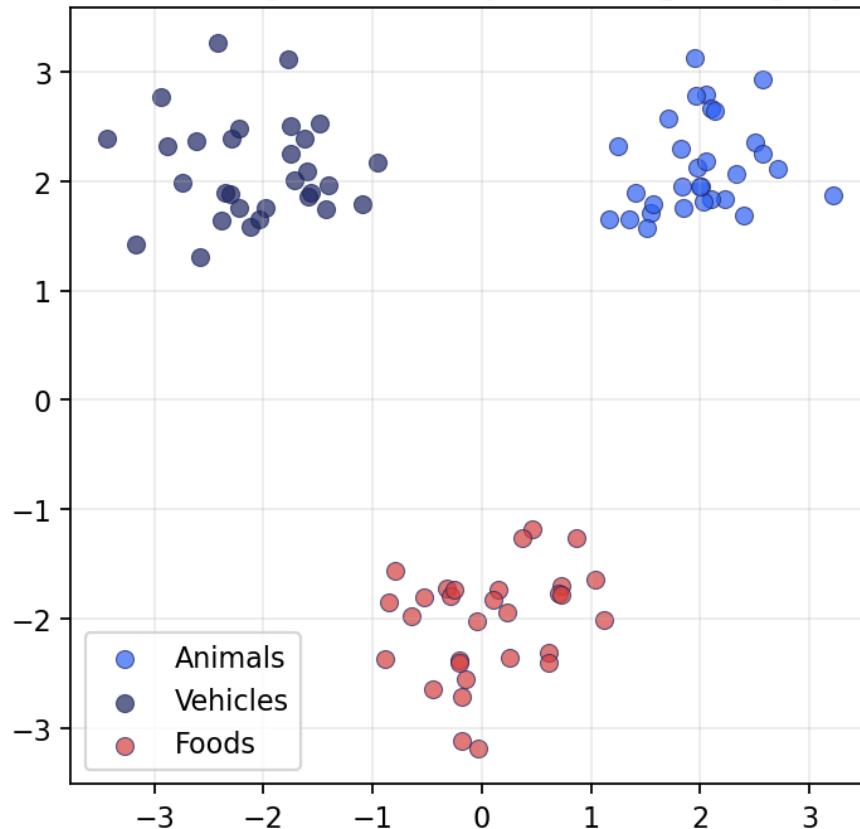
Q4. What is word embedding, and why is it useful in NLP?

A word embedding is a dense, low-dimensional vector representation of a word (typically 50 to 1024 dimensions) where the geometric relationships approximate semantic ones. Cosine similarity captures meaning similarity; vector arithmetic can capture analogies (king - man + woman $\approx$ queen).

Useful because: it replaces sparse one-hot vectors with dense vectors that share information across related words; it provides a strong initialization for downstream models; it enables similarity search, clustering, and retrieval; it makes models robust to out-of-vocabulary variation when using subword embeddings.

Methods: Word2Vec (CBOW and Skip-Gram), GloVe (global co-occurrence factorization), FastText (subword-aware), and contextual embeddings from BERT or modern LLMs. Contextual embeddings are the modern default since the same word can have different embeddings in different contexts.

Word embeddings cluster by meaning (2D projection)



Q5. What are some common real-world applications of NLP?

Search and information retrieval. Question answering and assistants. Translation. Text summarization. Sentiment analysis and opinion mining. Named entity recognition for extracting people, places, organizations from text. Chatbots and conversational agents. Document classification (spam, topic, intent). Speech recognition and text-to-speech. Text generation (writing assistance, code completion, marketing copy). Compliance and content moderation. Legal contract analysis. Clinical note summarization in healthcare. Automated grading and feedback in education. Code search and program synthesis.

Q6. What is Named Entity Recognition (NER), and where is it applied?

NER is the task of detecting spans in text that refer to entities of interest and assigning them a category: Person, Organization, Location, Date, Money, Product, and so on. It is typically framed as token-level sequence labeling using IOB or BIOES tagging schemes.

Applications: information extraction from news, financial filings, and clinical notes; populating knowledge graphs; redacting personally identifiable information; indexing documents for search; question-answering pipelines; compliance (detecting regulated entities); GraphRAG triplet extraction; resume parsing.

Modern systems use transformer-based encoders (BERT, RoBERTa) fine-tuned on labeled NER corpora (CoNLL-2003, OntoNotes, MIT Restaurant), or zero-shot via instruction-tuned LLMs for novel domains.

Q7. How does Latent Dirichlet Allocation (LDA) work for topic modeling?

LDA is a generative probabilistic model that assumes each document is a mixture of topics, and each topic is a distribution over words. Given a corpus, LDA infers (a) the topic distribution per document and (b) the word distribution per topic.

Generative story: for each document, sample a distribution over K topics from a Dirichlet prior. For each word position, sample a topic from that distribution, then sample a word from that topic's word distribution. Inference reverses this: given the words, estimate the topic distributions, typically via variational inference or collapsed Gibbs sampling.

Strengths: unsupervised, interpretable (you can read the top words per topic), fast on moderate corpora.

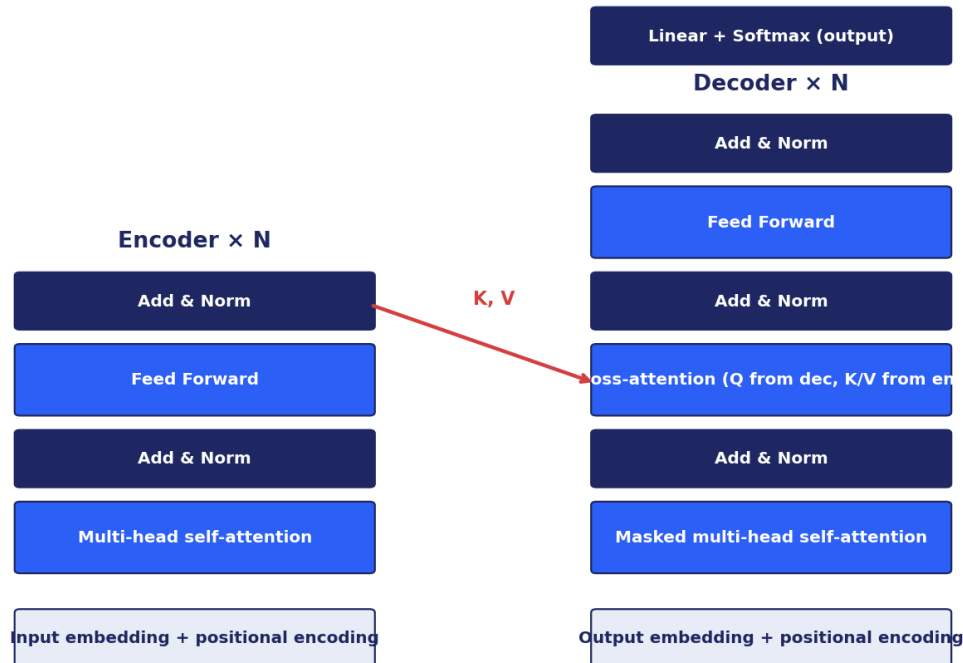
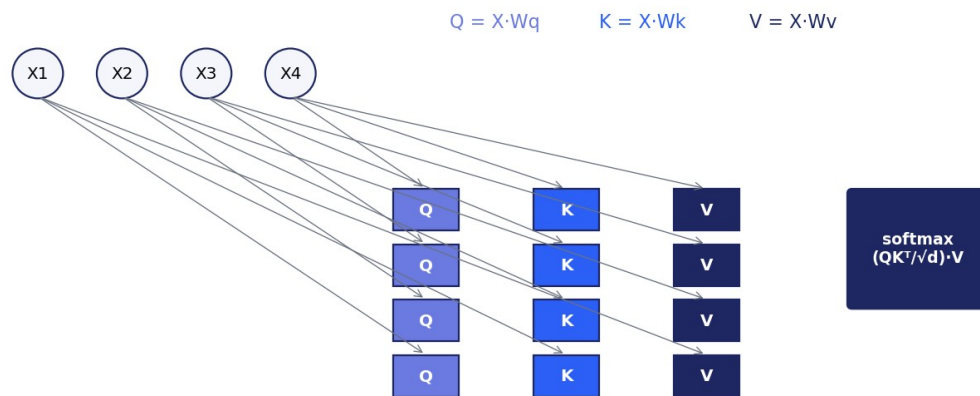
Limitations: requires choosing K , assumes bag-of-words (no order), topics can be incoherent on noisy data.

Modern alternatives: BERTopic, which clusters document embeddings and assigns interpretable labels using class-based TF-IDF.

Q8. What are transformers in NLP, and how have they impacted the field?

Transformers are a neural architecture introduced in 'Attention Is All You Need' (Vaswani et al., 2017). They replace recurrence with self-attention, allowing every token to directly attend to every other token in the input. This breaks the sequential bottleneck of RNNs and parallelizes across the sequence axis.

Impact: transformers swept NLP within three years. BERT (encoder-only) set new state-of-the-art on classification, NER, question answering. GPT (decoder-only) showed that scaling decoder transformers produces general-purpose text generators. T5 unified tasks as text-to-text. By 2020 transformers were ubiquitous in NLP and by 2022 they had expanded to vision (ViT), audio (Whisper), and multimodal models (CLIP, DALL-E). Every modern LLM is a transformer or a transformer derivative.

Transformer architecture (Vaswani et al. 2017)**Self-attention: Q, K, V from the same input****Q9. What is transfer learning in NLP, and how is it applied?**

Transfer learning in NLP means reusing knowledge from a model pretrained on a large general corpus to improve a downstream task with limited data. The dominant pattern: pretrain a transformer on a self-supervised objective on billions of tokens, then fine-tune on a specific task.

Application patterns: (1) feature extraction: use the pretrained model as a frozen encoder and train a small classifier on top of its embeddings. (2) Full fine-tuning: continue training the pretrained model on the target task. (3) PEFT: train small additional parameters (LoRA, adapters, prefix tuning) while keeping the base

frozen. (4) In-context learning: pass examples in the prompt with no parameter updates. (5) RAG: retrieve relevant context at inference time.

Choice depends on data size, compute budget, and how far the target domain is from the pretraining distribution. Small data plus small domain shift → in-context learning. Larger data with domain shift → PEFT or full fine-tuning.

Q10. How do you handle out-of-vocabulary (OOV) words in NLP models?

Classical approach: replace OOV words with a special <UNK> token. Loses information and degrades quality.

Subword tokenization (BPE, WordPiece, SentencePiece, Unigram): split rare words into known subword units. 'unbelievable' becomes ['un', 'believ', 'able']. Modern LLMs use this approach and effectively never face OOV at the input level: any string can be encoded as a sequence of subword tokens.

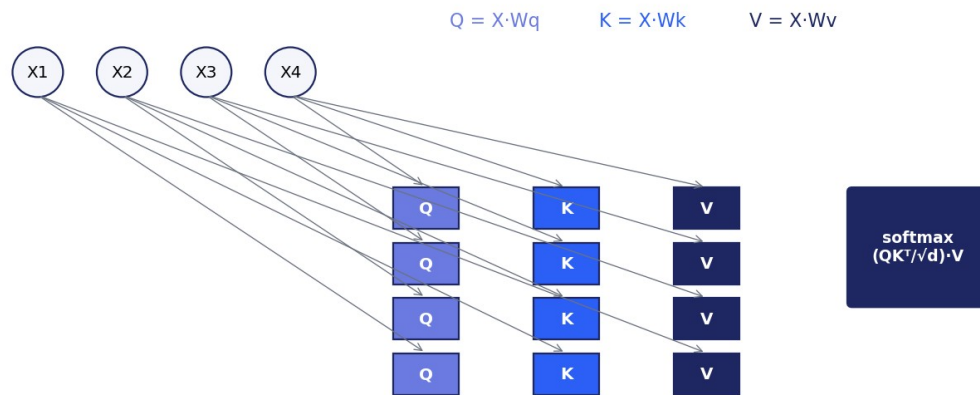
Character-level models avoid OOV entirely but are slower and lose word-level structure. FastText averages character n-gram embeddings to give every word (including unseen ones) a meaningful vector. For domain-specific terms, extend the vocabulary with domain tokens, then continue pretraining briefly on domain text so the new tokens learn meaningful embeddings.

Q11. Explain attention mechanisms and their role in sequence-to-sequence tasks.

Attention lets the decoder of a sequence-to-sequence model look back at the encoder outputs and decide which positions are relevant for generating the next output token. Instead of compressing the entire input into one fixed vector (as the original RNN encoder-decoder did), the decoder forms a weighted combination of all encoder hidden states, where the weights depend on the current decoder state.

Mechanics: compute attention scores between the current query and all keys; normalize with softmax to produce weights; combine the values with those weights. Modern transformer attention uses scaled dot-product: $\text{softmax}(Q \cdot K^T / \sqrt{d}) \cdot V$.

Role: attention solves the bottleneck of fixed-size context vectors and enables much better long-sequence modeling. It is also interpretable to a useful degree: attention weights tell you which input positions the model relied on, which is valuable for debugging and explanation.

Self-attention: Q, K, V from the same input**Q12. What is a language model, and how is it evaluated?**

A language model assigns probabilities to sequences of words. Equivalently, it predicts the next word given the previous words: $P(w_t | w_1, \dots, w_{t-1})$. It may be unidirectional (decoder-only, GPT-style) or bidirectional (encoder-only, BERT-style with masked language modeling).

Intrinsic evaluation: perplexity = $\exp(\text{average negative log-likelihood per token})$ on a held-out corpus. Lower is better. Perplexity measures how well the model predicts the next token but is only weakly correlated with downstream task performance once perplexity is below a usable threshold.

Extrinsic evaluation: performance on downstream benchmarks. Classification accuracy, F1 for NER, BLEU/ROUGE for translation/summarization, exact match for QA, MMLU and HellaSwag for general reasoning, HumanEval for code, MT-Bench and Chatbot Arena for open-ended chat. For generation, also human evaluation and LLM-as-a-judge with bias control.

Q13. What is BM25 and why is it still relevant?

BM25 (Best Matching 25) is a probabilistic ranking function that extends TF-IDF with term-frequency saturation and document-length normalization. It computes a relevance score between a query and a document based on term overlap.

Why it still matters: BM25 is fast, requires no training, no GPU, and produces strong baselines on keyword-heavy queries. Modern hybrid retrieval combines BM25 with dense retrieval via reciprocal rank fusion, and almost always beats either alone. Production search systems (Elasticsearch, OpenSearch, Lucene) ship BM25 as the default scoring.

Chapter 3

Transformers and Extended Architectures

This chapter covers 32 questions on transformers and extended architectures.

Q1. Describe learning rate scheduling and its role in optimizing generative model training.

A learning rate schedule changes the learning rate over the course of training rather than holding it constant. The two motivations: (1) very early in training, the model is far from any good basin, so large steps help quickly; but the random initialization can produce huge gradients, so a brief warm-up smooths this. (2) Late in training, smaller steps help the optimizer settle into a sharp minimum instead of bouncing around it.

Common schedules: linear warm-up followed by linear or cosine decay (transformer default). One-cycle (Smith): linear warm-up, then linear decay below the initial rate. Step decay: drop by a factor every N epochs. Inverse-square-root: classical for transformers, $\eta \propto 1/\sqrt{t}$ after warm-up. Plateau-based: decay when validation stops improving.

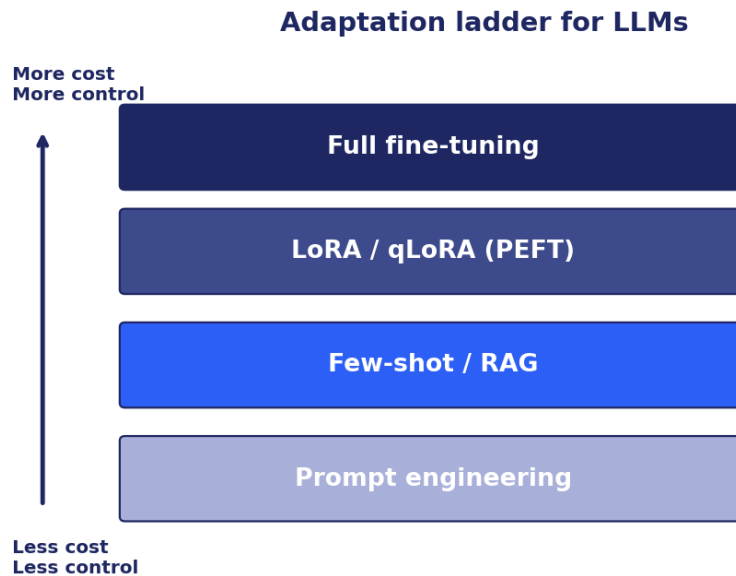
For generative models specifically, schedules matter more because the loss surface is non-stationary as the model's outputs improve. RLHF often uses much smaller learning rates than SFT ($1e-6$ vs $1e-5$) because policy drift is the main risk.

Q2. Discuss transfer learning in NLP. How do pretrained language models help downstream tasks?

The dominant paradigm in NLP since 2018: pretrain a transformer on self-supervised objectives over a massive corpus, then transfer that knowledge to downstream tasks. The pretrained model has learned general syntactic and semantic patterns and can be specialized cheaply.

Forms of transfer: (a) feature extraction with frozen encoder, (b) full fine-tuning, (c) parameter-efficient fine-tuning (LoRA, adapters, prefix tuning), (d) prompt-based zero-/few-shot via in-context learning, (e) instruction tuning to make a base model follow instructions, (f) RLHF for alignment.

Benefit: the cost of pretraining is paid once (hundreds of GPU-years for a frontier model). Downstream users pay a tiny fraction to adapt. This is why a small startup can fine-tune LLaMA-3-70B in a few hours on a handful of GPUs and produce a model competitive on a specific task.



Q3. What are the key differences between GPT and BERT?

Architecture: BERT is encoder-only with bidirectional self-attention. GPT is decoder-only with causal (left-to-right) masked self-attention. BERT sees the full input at once; GPT sees only the prefix.

Pretraining objective: BERT uses masked language modeling (predict masked tokens from bidirectional context) plus next-sentence prediction. GPT uses causal language modeling (predict next token from left context only).

Use case: BERT is best for understanding tasks (classification, NER, extractive QA, similarity) where you need rich bidirectional representations of an input. GPT is best for generation (chat, completion, text-to-text, code generation) where you produce tokens one at a time conditioned on the prefix.

Modern state: instruction-tuned decoder-only LLMs (GPT-4, Claude, LLaMA, Gemini) have become the dominant pattern because they handle both understanding and generation via prompting. BERT-style encoders remain dominant for embedding generation and classification at scale where latency and cost matter.

Q4. What problems with RNNs do transformer models solve?

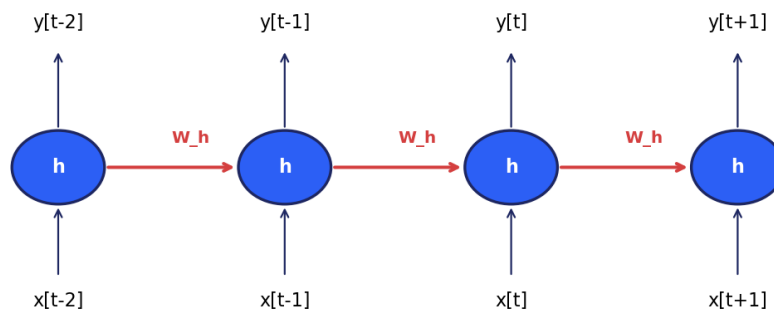
Sequential processing: an RNN must process tokens one at a time because each step depends on the previous hidden state. Transformers process all tokens in parallel during training via self-attention, dramatically speeding up training on modern GPUs and TPUs.

Long-range dependencies: an RNN compresses everything before time t into a single hidden vector. Information from distant tokens gets diluted or lost. Self-attention lets every token directly attend to every other token, regardless of distance.

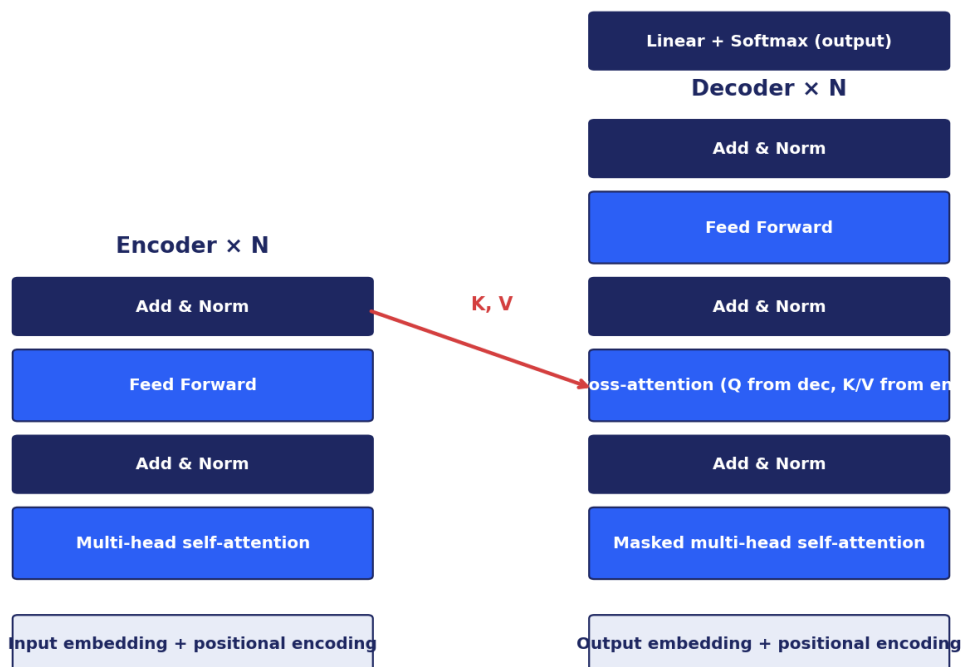
Vanishing/exploding gradients: BPTT through many timesteps suffers gradient instability. Transformers have a shallower effective depth from any one token to any other (one attention hop), and use residual connections plus layer normalization to keep gradients well-behaved.

Modeling capacity per parameter: empirically, transformers scale better than RNNs. Doubling parameters in a transformer reliably improves performance; the same is not true for vanilla RNNs.

Recurrent Neural Network unrolled in time



Transformer architecture (Vaswani et al. 2017)



Q5. How is the transformer different from RNN and LSTM?

RNN/LSTM process sequences sequentially with a recurrent hidden state. Transformers process sequences in parallel using self-attention. RNN/LSTM mix information one step at a time; transformers mix in one shot via the Q·K matrix.

Compute cost: RNN/LSTM are $O(n)$ sequential steps with $O(d^2)$ work per step. Transformer self-attention is $O(n^2 \cdot d)$ per layer but fully parallel. For typical n and d on a GPU, the transformer is much faster per training step.

Memory: transformer attention is $O(n^2)$ memory in the naive form, which is the main scalability bottleneck. FlashAttention and other techniques reduce constants; linear-attention and state-space models (Mamba) trade off expressiveness for asymptotic memory.

Position handling: RNN/LSTM encode position implicitly through the sequential processing. Transformers have no built-in notion of position and must add explicit positional encodings (absolute, learned, or relative such as RoPE).

Q6. How does BERT work, and what makes it different from previous NLP models?

BERT is a stack of transformer encoder layers pretrained on two self-supervised objectives: Masked Language Modeling (mask 15% of input tokens, predict them from both left and right context) and Next Sentence Prediction (binary classification: are these two sentences contiguous?). It uses WordPiece tokenization.

What was new: previous NLP models either used left-to-right context (ELMo could concatenate forward and backward, but not jointly) or task-specific architectures. BERT pretrained a single bidirectional model and then added a thin task-specific head for any downstream task. This unified pattern, plus the strength of bidirectional context, set new state-of-the-art across many benchmarks.

Downstream pattern: prepend a [CLS] token to the input; for classification, run a linear head on the final [CLS] embedding; for token-level tasks like NER, run a head on each token's final embedding; for span tasks like SQuAD, predict start and end token positions.

Q7. Why is incorporating relative positional information crucial in transformer models?

Self-attention is permutation-equivariant: shuffling the input tokens produces a shuffled output. To restore order, you must inject position information. Absolute positional encoding (sinusoidal in the original transformer, learned in BERT) tells the model 'this token is at position 5'.

Relative position encoding instead tells the model 'this query is k positions after this key'. This is closer to what the attention computation actually needs: the relevance of one token to another usually depends on their separation, not their absolute positions.

When relative position helps: tasks where similar patterns appear at different absolute positions (most language tasks), and when generalizing to sequences longer than those seen during training. Rotary Position Embeddings (RoPE), used in LLaMA, GPT-NeoX, and many modern LLMs, encode relative position by rotating the Q and K vectors. ALiBi adds a position-dependent bias to attention scores. Both extrapolate better than absolute encodings.

Q8. What challenges arise from the fixed and limited attention span in vanilla Transformers?

The vanilla transformer has a fixed maximum context length set at pretraining time (typically 512 for BERT, 2048 for GPT-2, up to 200K+ for modern LLMs). Inputs longer than this must be truncated or chunked, losing information.

Effects: tasks requiring long-range dependencies (long documents, codebases, video) lose context that should inform the prediction. The model cannot reference details outside its window. Memory and compute scale $O(n^2)$ with context length, so simply training with a longer window is expensive.

Mitigations: sliding-window attention; sparse attention patterns (Longformer, BigBird); retrieval augmentation (RAG) so the model sees only relevant chunks; hierarchical models that summarize chunks then attend over summaries; state-space models (Mamba) and linear attention for sub-quadratic scaling.

Q9. Why is naively increasing context length not straightforward?

Three constraints: memory, compute, and quality.

Memory: self-attention stores an $n \times n$ score matrix per head per layer. Doubling n quadruples memory. For 32K context with 32 heads and 32 layers, the attention memory alone is many GB. FlashAttention reduces this by tiling but cannot beat the n^2 fundamental.

Compute: every attention operation costs $O(n^2 \cdot d)$. Doubling n quadruples FLOPs. Training and inference cost scale accordingly.

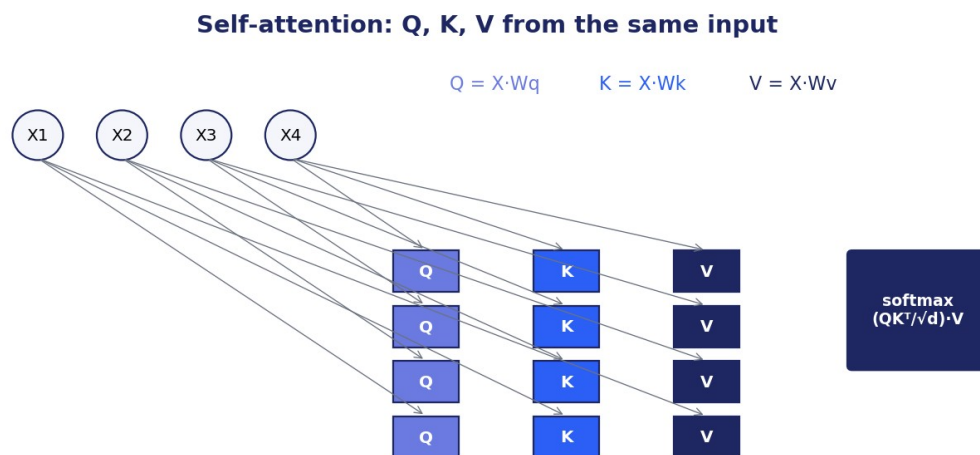
Quality: models pretrained on short contexts do not automatically generalize to long contexts. Positional encodings may extrapolate poorly. The model needs training on long-context examples, which requires long-context training data and is itself expensive. Length generalization techniques (RoPE scaling, ALiBi, YaRN) help but require care.

Q10. How does self-attention work?

Self-attention computes a new representation of each token as a weighted average of all token representations, where the weights depend on similarity between tokens. Mechanics in five steps:

1. Project each input token x_i into three vectors: query $Q_i = W_Q \cdot x_i$, key $K_i = W_K \cdot x_i$, value $V_i = W_V \cdot x_i$.
2. Compute pairwise scores: $S_{ij} = Q_i \cdot K_j / \sqrt{d}$. The $\sqrt{d}$ scaling keeps softmax in a usable range.
3. Apply softmax row-wise to get attention weights: $A_{ij} = \text{softmax}(S)_{ij}$. Each row sums to 1.
4. Combine value vectors: $\text{output}_i = \sum_j A_{ij} \cdot V_j$.
5. Multi-head attention runs steps 1-4 in parallel h times with different W_Q, W_K, W_V matrices, concatenates the outputs, and projects back to the model dimension.

For causal (decoder) attention, mask scores S_{ij} for $j > i$ to $-\infty$ before softmax so each position only attends to itself and earlier positions.



Q11. What pretraining mechanisms are used for LLMs?

Causal language modeling (next-token prediction). Predict token t given tokens 1 to $t-1$. Used in GPT, LLaMA, Mistral, and most decoder-only LLMs. Simple, scales well, and naturally enables generation at inference.

Masked language modeling (MLM). Mask a fraction of tokens (15% in BERT) and predict them from bidirectional context. Used in BERT, RoBERTa, DeBERTa.

Span corruption. Mask contiguous spans of tokens and predict them. Used in T5 and UL2. Generalizes MLM to longer spans.

Prefix language modeling. Bidirectional attention on a prefix, causal attention on a suffix. Used in some encoder-decoder models.

Contrastive objectives. SimCSE, sentence-transformers: pull positive pairs together, push negatives apart. Used to learn sentence embeddings.

Replaced token detection. ELECTRA: a small generator proposes replacements; the main model classifies which tokens were replaced. More sample-efficient than MLM.

Q12. Why is multi-head attention needed?

Single-head attention forces the model to mix all kinds of relationships into one set of weights. Multi-head attention lets the model attend to different aspects in parallel: one head might track syntactic dependencies, another semantic co-reference, another local n-gram patterns.

Mechanics: split the model dimension d into h heads of dimension d/h each. Run attention independently in each head with its own Q, K, V projections. Concatenate the h outputs and apply a final linear projection.

Cost is the same as single-head with full dimension (the per-head dimension is smaller), so multi-head gives you specialization for free. Typical h is 8, 16, or 32. Modern LLMs sometimes use grouped-query attention (GQA) or multi-query attention (MQA) where Q heads outnumber K/V heads, trading a small quality drop for much faster inference.

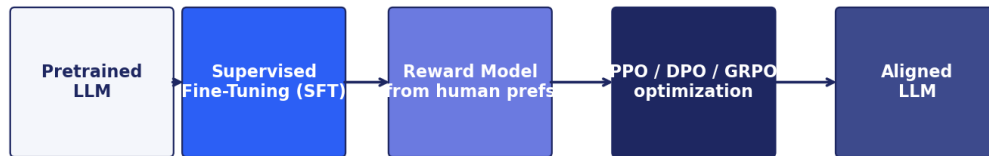
Q13. What is RLHF, and how is it used?

Reinforcement Learning from Human Feedback (RLHF) is the standard alignment pipeline for instruction-following LLMs. It has three stages.

Stage 1: Supervised Fine-Tuning (SFT). Fine-tune the base LLM on a curated set of (prompt, ideal response) pairs written by humans. The model learns to follow instructions.

Stage 2: Reward Model (RM). For each prompt, generate multiple responses and have humans rank them. Train a separate model to predict the human preference: $RM(prompt, response) \rightarrow \text{scalar}$. The reward model encodes 'what humans prefer'.

Stage 3: Policy optimization. Use the reward model as the reward signal to fine-tune the SFT model with reinforcement learning, typically PPO. Add a KL penalty against the SFT model to prevent the policy from drifting into reward-hacking regions. Modern variants: DPO (skips the explicit reward model and trains directly on preference pairs), GRPO (group-relative advantage, drops the value head), RLAI (AI-generated preferences).

RLHF: from pretrained base to aligned LLM**Q14. What is catastrophic forgetting in LLMs?**

Catastrophic forgetting is when a neural network, fine-tuned on a new task or domain, loses its performance on tasks it previously knew. Weights that encoded old knowledge get overwritten by gradients from the new task.

In LLMs: aggressive fine-tuning on a narrow domain can degrade general reasoning, instruction-following, or safety behavior. The model becomes 'good at the new task, worse at everything else.'

Mitigations: use parameter-efficient fine-tuning (LoRA, adapters) so the base weights are frozen. Mix the new data with a sample of original-distribution data (replay or rehearsal). Apply Elastic Weight Consolidation or similar regularizers that penalize large changes to weights deemed important for old tasks. Use lower learning rates. Evaluate on a regression suite covering base capabilities after any fine-tuning run.

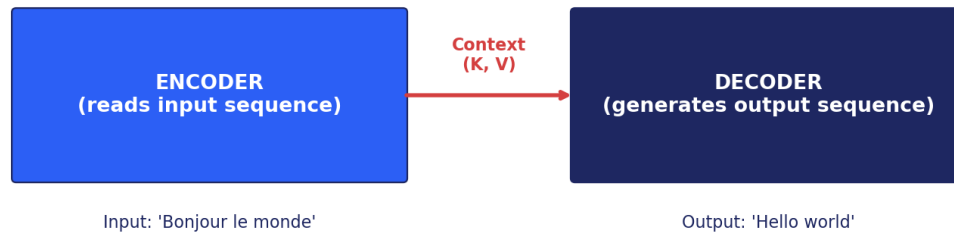
Q15. In a transformer seq2seq model, what do the encoder and decoder do, and how does information flow?

Encoder: a stack of transformer encoder layers that processes the entire input sequence with bidirectional self-attention. Its output is a sequence of contextual representations (one per input token).

Decoder: a stack of decoder layers that generates the output sequence one token at a time. Each decoder layer has three sub-layers: (1) masked self-attention over the already-generated output tokens, (2) cross-attention where Q comes from the decoder and K, V come from the encoder outputs, (3) feed-forward.

Information flow during training: the entire target sequence is shifted by one position and fed into the decoder. Cross-attention lets the decoder peek at the encoder representations. The loss is teacher-forced cross-entropy on the predicted tokens.

Information flow during inference: feed the start token. Run the decoder. Sample or argmax the next token. Append. Repeat. Each step costs $O(n^2)$ for fresh decoding or $O(n)$ with KV-cache reuse. Inference is autoregressive and sequential.

Encoder-decoder transformer (translation example)**Q16. Why is positional encoding crucial?**

Self-attention treats its input as a set, not a sequence. Without positional information, the model cannot distinguish 'dog bites man' from 'man bites dog'.

Sinusoidal positional encoding (original transformer): add fixed sin/cos functions of position to the input embeddings. Different frequencies per dimension. Generalizes (in theory) to unseen positions.

Learned positional encoding (BERT, GPT-2): treat each position as a learnable embedding. Simple, works well within trained range, does not extrapolate.

Rotary Position Embedding (RoPE, LLaMA): rotate Q and K vectors by an angle proportional to position. Encodes relative position naturally in the dot product. Extrapolates better than absolute. ALiBi adds a position-dependent bias to attention scores; very strong extrapolation.

Q17. When fine-tuning a pretrained transformer for a domain-specific NLP task, what strategies ensure effective knowledge transfer?

Data: collect a high-quality target dataset. Quality beats quantity. Deduplicate, filter for relevance, balance classes or topics if applicable. Domain-mixed training data (some target, some general) reduces catastrophic forgetting.

Method: prefer PEFT (LoRA, qLoRA) for cost and to avoid catastrophic forgetting. Full fine-tuning when you have abundant target data and need maximum quality. Continued pretraining (a brief pass of LM objective on domain text) before fine-tuning helps when the domain vocabulary diverges from general.

Hyperparameters: use a small learning rate (1e-5 to 5e-5 for full FT; 1e-4 to 3e-4 for LoRA). Use warm-up. Use weight decay. Monitor validation loss every few hundred steps and apply early stopping.

Evaluation: hold out a representative test set. Include a regression suite for base capabilities so you can detect catastrophic forgetting early.

Q18. What is cross-attention in encoder-decoder transformers?

Cross-attention is the mechanism that lets the decoder consult the encoder output when generating each output token. In a decoder layer, after masked self-attention over generated tokens, cross-attention takes Q from the decoder's current representation and K, V from the encoder outputs.

Each decoder position emits an attention weight over all encoder positions and produces a context vector. This is how an English-to-French translator decides which French word to emit based on which English words are relevant at this decoding step.

Cross-attention is also central to multimodal transformers. In Flamingo, BLIP-2, and similar, cross-attention lets the language decoder attend to image features. In DALL-E 2 / Stable Diffusion, the U-Net cross-attends to the text encoder's outputs to condition image generation on the prompt.

Q19. Compare sparse (cross-entropy) vs dense (MSE) loss functions in training language models.

Cross-entropy is the standard loss for next-token prediction. It treats each token as a classification target over the vocabulary and penalizes the model based on the probability assigned to the correct token. Cross-entropy is well-matched to the categorical nature of language: a token is or is not the correct token.

Mean Squared Error treats the prediction as a vector in continuous space. It is appropriate for regression but is wrong for token prediction because tokens are categorical. Using MSE on token IDs is mathematically nonsensical (token 5 is not 'closer' to token 6 than token 100).

Where MSE-like losses appear in language modeling: distillation (match the student's logits to the teacher's via KL or MSE on logits), representation learning (match embeddings), and consistency objectives. For token prediction itself, cross-entropy is the right choice.

Q20. How can reinforcement learning be integrated into LLM training, and what challenges arise in loss design?

RL is integrated through RLHF (with a reward model) or RLVR (with a programmatic verifier). The LLM is treated as a policy: the prompt is the initial state, each generated token is an action, the final sequence receives a reward.

Algorithms: PPO with a value head and KL penalty against a reference model. DPO skips the explicit reward model and trains directly on preference pairs via a closed-form objective. GRPO drops the value head and uses group-relative advantage.

Challenges in loss design: (1) reward hacking when the reward model has exploitable patterns. (2) Sparse, delayed rewards make credit assignment hard. (3) KL penalty weight must balance task performance against staying close to the SFT policy. (4) Off-policy correction: rolling out from a slightly stale policy introduces bias unless corrected with importance sampling. (5) Variance reduction: advantage estimation has high variance, hurting sample efficiency.

RLHF: from pretrained base to aligned LLM



Q21. How is information from visual and textual modalities integrated in multimodal LLMs?

Three common patterns. (1) Shared embedding space (CLIP): train a vision encoder and a text encoder jointly with contrastive loss on image-caption pairs. The resulting embeddings live in the same space and support cross-modal retrieval and zero-shot classification.

(2) Cross-attention bridges (Flamingo, BLIP-2): a small bridge module (Perceiver Resampler or Q-Former) projects visual features into the language model's space and lets the LM cross-attend to them. The base LM stays frozen.

(3) Direct token projection (LLaVA, GPT-4V style): a vision encoder produces patch tokens; a linear or MLP projection maps each visual token into the LM's token embedding space; the LM processes a mixed sequence of visual and text tokens uniformly via self-attention.

For tasks like image captioning, the decoder generates caption tokens conditioned on the visual representation. For VQA, the question is fed as text tokens and the answer is generated conditioned on both the question and the image.

Q22. What is the role of cross-modal attention in models like VisualBERT and CLIP?

Cross-modal attention lets one modality query another. In VisualBERT, image region features and text tokens are concatenated into one sequence and processed by a transformer with full self-attention. Text tokens can attend to image regions and vice versa, learning joint representations.

CLIP's setup is different: separate text and image encoders, no cross-attention between them. Alignment comes from the contrastive objective during training: image and text embeddings of a matching pair are pulled together; mismatched pairs are pushed apart. The geometry of the joint space encodes the cross-modal relationship.

Cross-attention is essential where direct token-level interaction matters (caption generation, VQA, grounded reasoning). Contrastive joint spaces excel at retrieval, search, and zero-shot classification where you need cheap, dense comparison.

Q23. How is training data annotated for image-text matching, and what considerations matter?

Annotation styles: aligned captions (each image has one or more descriptive captions, like COCO Captions); product image-title pairs from e-commerce; scraped alt-text from the web (LAION, used in CLIP and Stable Diffusion); click-through pairs (a user searched X, clicked image Y).

Considerations: alignment quality (a noisy caption hurts contrastive training); scale (hundreds of millions of pairs typical for foundation models); diversity (domain coverage matters more than raw count); bias (web-scraped data reflects web biases including stereotypes); copyright and licensing (LAION-5B has documented issues); deduplication (near-duplicates inflate eval metrics if they leak).

Hard negative mining helps when easy negatives dominate: pick negatives that look similar to the positive but are wrong, so the model learns finer distinctions.

Q24. When training a generative model for image synthesis, what loss functions are used?

GANs: adversarial loss. The generator tries to fool a discriminator; the discriminator tries to distinguish real from generated. Pure adversarial loss is notoriously unstable; modern GANs add feature matching, gradient penalty, and spectral normalization.

VAEs: a reconstruction loss (pixel-wise MSE or BCE) plus a KL divergence term regularizing the latent distribution toward a prior.

Diffusion models: predict the noise added to a clean image at random timesteps, minimizing MSE between predicted and actual noise. Simple and stable. Modern extensions: latent diffusion (train in a compressed latent space), classifier-free guidance, v-prediction, EDM parameterization.

Perceptual losses: compare features extracted by a pretrained network (VGG, LPIPS) rather than raw pixels. Better correlated with human judgment.

Q25. What is perceptual loss, and how does it differ from pixel-wise loss?

Perceptual loss measures the distance between generated and target images in a deep feature space (typically intermediate layers of a pretrained VGG or LPIPS network) rather than pixel-by-pixel. Two images can be perceptually similar (same content, slight color shift) but have high pixel-wise distance; perceptual loss is much closer to human judgment.

Pixel-wise losses (L_1 , L_2): treat each pixel independently. Penalize even tiny shifts and color changes that humans do not notice. Lead to blurry outputs in generation because the optimal solution under L_2 is the mean of plausible outputs.

Perceptual loss: feeds both images through a frozen feature extractor and compares activations. Captures texture, structure, and semantic content. Used in super-resolution (ESRGAN), style transfer, image-to-image translation. Often combined with adversarial loss for sharper outputs.

Q26. What is masked language-image modeling?

Masked language-image modeling is a multimodal self-supervised objective: mask tokens from both modalities and reconstruct them from joint context. Used by models like VisualBERT, ViLBERT, MaskGIT, and BEiT-3.

Setup: take an image and its caption. Tokenize the image (into patches or VQ-VAE tokens) and the text. Concatenate. Mask some image tokens and some text tokens. Train the model to predict the masked tokens given the remaining ones.

Why it helps: the model learns cross-modal grounding. Predicting a masked image patch may require reading the caption; predicting a masked word may require looking at the image. The shared representation encodes the relationship between modalities.

Q27. How do cross-attention weights influence generation in multimodal models?

In a multimodal generator with cross-attention (Flamingo, BLIP-2, Stable Diffusion's U-Net), cross-attention weights determine how much each modality contributes to each generation step.

Concretely: at each generation step, the decoder forms a query and attends to keys from the conditioning modality (image features for text generation, or text encoder outputs for image generation). High attention weight on a particular image patch means that patch is strongly influencing the next token.

Practical implications: classifier-free guidance scales these weights to amplify the conditioning signal. Inspecting cross-attention maps explains where the model is looking. In diffusion models, attention manipulation enables compositional control (e.g. attention slicing for prompt editing).

Q28. What are unique challenges in training multimodal generative models?

Data alignment: multimodal training requires aligned pairs across modalities, which are expensive to curate. Noisy alignment hurts training. Misalignment in one direction (e.g. captions that don't describe images) is hard to detect.

Modality imbalance: text data is far more abundant than aligned image-text pairs. Models may overfit to language patterns and underuse visual signals.

Optimization: different modalities have different gradient dynamics. Naive joint training can let one modality dominate. Techniques: modality-specific learning rates, gradient balancing, separate warm-up schedules.

Evaluation: there is no single perplexity-like metric. Image generation needs FID, CLIP-score, human evaluation. Captioning needs BLEU, CIDEr, plus human eval. VQA needs exact match and human judgment. Cross-modal retrieval needs Recall@k.

Compute: training a state-of-the-art multimodal model takes thousands of GPU-years, vastly more than unimodal models of similar parameter count.

Q29. How do multimodal generative models address data sparsity?

Leverage abundant unimodal data: pretrain text and image encoders separately on unimodal corpora (BERT for text, ViT for images), then align with a smaller supervised paired dataset.

Web scraping at scale: LAION-5B, COYO-700M and similar collect billions of image-text pairs from the web. Noisy but large.

Synthetic data: use one model to generate training data for another. Image captioning models can generate captions for un-captioned images; text-to-image models can generate images from textual prompts. Risk of compounding errors.

Cross-modal contrastive objectives (CLIP-style): align modalities with minimal supervision (just paired examples, no per-pair labels). Scales to billions of pairs.

Self-distillation and pseudo-labeling: train a model, use it to label new data, retrain.

Q30. Explain Vision-Language Pretraining (VLP) and its significance.

VLP is the practice of pretraining a single model jointly on visual and textual data so that downstream vision-language tasks can be solved by light fine-tuning or zero-shot prompting. It is the multimodal analog of BERT pretraining.

Pretraining objectives mix image-text matching, masked language modeling conditioned on images, masked image modeling conditioned on text, and contrastive image-text alignment.

Significance: a well-pretrained VLP model transfers to image captioning, visual question answering, image-text retrieval, visual reasoning, and image generation with minimal task-specific work. CLIP enabled zero-shot image classification at ImageNet quality. Flamingo and BLIP-2 enabled few-shot VQA. GPT-4V and Gemini have taken this further into multimodal reasoning over images, charts, and documents.

Q31. How do CLIP and DALL-E integrate vision and language?

CLIP (Contrastive Language-Image Pretraining): trains a vision encoder and a text encoder jointly with contrastive loss on 400M image-caption pairs. Outputs live in a shared embedding space. Use cases: zero-shot image classification (encode candidate labels as text, embed image, take argmax similarity), image-text retrieval, content moderation, dataset filtering.

DALL-E: text-to-image generation. The original DALL-E used a discrete VAE plus an autoregressive transformer over image tokens conditioned on text. DALL-E 2 and 3 replaced this with diffusion models: a CLIP text encoder conditions a diffusion decoder that generates the image. Stable Diffusion is the open-source exemplar of the same pattern.

Integration pattern: text provides the conditioning signal; the image generator (autoregressive or diffusion) attends to text embeddings at every generation step via cross-attention, producing images that follow the prompt.

Q32. How do attention mechanisms enhance vision-language models?

Cross-modal grounding: attention lets the model focus on specific image regions when generating text about them, and on specific words when interpreting an image. This grounding is what enables answers like 'the red car on the left' to actually correspond to the red car on the left.

Compositional reasoning: attention over image regions plus attention over text tokens lets the model reason about combinations ('the man with the umbrella next to the woman in red'). Without attention, the model collapses everything into a single vector and loses compositionality.

Interpretability: attention maps over an image show which regions drove a given output token, which is useful for debugging and explanation.

Generation control: techniques like attention manipulation, classifier-free guidance scaling, and prompt-based attention editing rely on attention as the control surface for steering generation.

Chapter 4

Fundamentals of Large Language Models

This chapter covers 13 questions on fundamentals of large language models.

Q1. Describe your experience working with text generation using generative models.

A strong interview answer here is concrete and grounded in real artifacts. Cover three dimensions: (a) which models you have used (GPT-4, Claude, LLaMA, Mistral, Gemini); (b) how you used them (API calls with few-shot prompting; fine-tuning with LoRA; RLHF on a custom domain; building a RAG over enterprise documents); (c) what you learned (prompt patterns that worked, latency and cost tradeoffs, places where the model failed and how you mitigated).

Avoid generic descriptions. Say 'I built a customer-support assistant on top of Claude 3.5 Sonnet with a RAG over 80K Confluence pages, hit P95 of 2.1s, and cut support ticket handle time by 18%'. Specifics signal experience.

Q2. What are the fundamental differences between discriminative and generative models?

Discriminative models learn $P(y | x)$: the conditional probability of a label given the input. They draw decision boundaries between classes. Examples: logistic regression, SVMs, random forests, classification transformers like BERT.

Generative models learn $P(x, y)$ or $P(x)$: the joint or marginal distribution of the data. They model how the data was produced and can sample new examples. Examples: Naive Bayes, HMMs, VAEs, GANs, diffusion models, autoregressive language models like GPT.

Practical differences: discriminative models often achieve higher accuracy with less data because they only model what is needed for the task. Generative models support more tasks (sampling, density estimation, anomaly detection, in-context learning) and often capture richer structure. Modern LLMs are generative but solve discriminative tasks via prompting, blurring the historical distinction.

Q3. What types of generative models have you worked with, and in what contexts?

Examples of strong context-rich answers (use your own actual work):

Autoregressive LLMs (GPT-4, Claude, LLaMA): customer-facing chatbots, code completion assistants, document summarization, structured extraction.

Diffusion models (Stable Diffusion, FLUX): marketing image generation, product photography augmentation, controllable image editing.

VAEs: anomaly detection, latent-space exploration for tabular data.

GANs: synthetic data generation for training, super-resolution.

Specialized: Whisper for transcription, Codex/StarCoder for code, MusicLM for audio.

Frame each example with the business outcome, not just the model name. Interviewers want to know you can connect model capabilities to product value.

Q4. What is multimodal AI, and why is it important?

Multimodal AI processes more than one modality (text, image, audio, video, structured data) within a single system. The model may take multiple modalities as input, produce multiple modalities as output, or both.

Why it matters: real-world problems are rarely single-modality. A medical diagnosis uses images, lab results, and clinical notes together. A search query may include a photo and a question. A self-driving car fuses camera, radar, and lidar. Multimodal models match how humans actually solve these problems.

Modern examples: GPT-4V and Gemini accept images and text. Whisper translates speech across languages. CLIP enables image search by text. Sora generates video from text prompts. Multimodal is now the default for foundation models.

Q5. Explain cross-modal learning with examples.

Cross-modal learning trains a model to use information from one modality to improve another. The classic example: training on (image, caption) pairs lets the model use language to disambiguate visual content and use visual content to ground language.

CLIP: image and text encoders trained with contrastive loss on 400M pairs. Enables zero-shot image classification (no labeled image data needed) by encoding class names as text and choosing the closest match. Powers Stable Diffusion's text conditioning.

Image captioning: trained on (image, caption) pairs. Generates captions for new images, using language structure learned from captions and visual understanding from the encoder.

Visual question answering (VQA): given (image, question), produce an answer. Requires combining visual perception and language reasoning.

Audio-visual speech recognition: combining lip movement and audio improves transcription accuracy in noisy environments.

Q6. What are common challenges in developing multimodal models, and how do you address them?

Data alignment: paired data across modalities is expensive. Mitigation: leverage abundant unimodal data with separate pretraining, then align with a smaller paired set. Use noisy web pairs at scale (LAION).

Modality imbalance: one modality may dominate, drowning out the other. Mitigation: modality-specific learning rates; gradient balancing; modality dropout (randomly withhold one modality during training).

Architecture complexity: shared vs. separate encoders, cross-attention vs. shared embedding space, frozen vs. trainable. Mitigation: standardize on a published pattern (Q-Former bridge, direct projection) before customizing.

Evaluation: each modality has its own metrics. Mitigation: build per-modality evaluations plus a unified end-task metric. Add human evaluation for open-ended outputs.

Compute: multimodal training is expensive. Mitigation: freeze pretrained encoders, train only the bridge or projection module.

Q7. How do CLIP and DALL-E innovate in multimodal learning?

CLIP innovation: showed that a simple contrastive objective on web-scale image-text pairs produces a model with strong zero-shot transfer. No labels needed for classification at inference time. The CLIP embedding space became a foundational asset reused in countless downstream systems (Stable Diffusion conditioning, image search, content moderation, dataset filtering).

DALL-E innovation: text-to-image generation at quality and prompt-following fidelity unseen before. DALL-E 1 used a discrete VAE plus autoregressive transformer; DALL-E 2 and 3 use diffusion with CLIP-based text conditioning. The result: creative tools that ordinary users can drive with natural language.

Bigger picture: both demonstrated that scale plus a good cross-modal objective produces emergent capabilities. They opened the door to today's multimodal foundation models.

Q8. Why does data preprocessing and representation matter in multimodal learning?

Different modalities need different preprocessing: text needs tokenization, normalization, length capping; images need resizing, normalization, augmentation; audio needs sampling rate conversion and mel-spectrogram computation; video needs frame sampling and temporal aggregation.

Representation: each modality is mapped to a vector space. Text via a transformer encoder. Images via a CNN or ViT. Audio via a spectrogram encoder. These vectors then need to live in a comparable space for the multimodal fusion to make sense.

Alignment: representations are aligned either via a contrastive objective (CLIP style) or via cross-attention (BLIP, Flamingo style). The choice depends on downstream use: contrastive for retrieval, cross-attention for generation.

Quality control: garbage in, garbage out applies double in multimodal. Filter for alignment quality, deduplicate near-duplicates, handle missing modalities gracefully.

Q9. How does multimodal sentiment analysis improve on text-only models?

Text alone misses sarcasm, tone, and visual context. A tweet 'great' with a crying emoji means the opposite. A product review's accompanying video might show a broken item while the text is polite.

Multimodal sentiment analysis combines text with audio (tone of voice, prosody), video (facial expressions), or images (visual content). Aggregating across modalities reduces ambiguity.

Real cases: customer service analytics on call recordings (text transcript + audio tone); social media monitoring (text + image/video); video review analysis (text + visual + audio).

Accuracy gains are commonly 5-15% over text-only models on benchmarks like CMU-MOSEI and IEMOCAP. The gains are larger when text alone is ambiguous, which is precisely the high-value case.

Q10. What metrics evaluate multimodal models, and how do they differ from unimodal metrics?

Modality-specific metrics: BLEU/ROUGE for text generation, FID for image generation, WER for speech, Recall@k for retrieval. Each measures one output type.

Cross-modal metrics: CLIP score (text-image alignment), CIDEr (image captioning), Image-Text Matching accuracy. These measure how well the modalities work together.

End-task metrics: VQA accuracy, retrieval Recall@k, downstream task performance. These are what users actually care about.

Human evaluation: especially for generative multimodal outputs. Crowd ratings on fluency, relevance, factuality, aesthetics.

Bias and fairness: multimodal models can have biases across both modalities. Evaluate on demographic balanced sets, check for stereotypes in generation.

Difference from unimodal: you need both per-modality and joint metrics. A model may have perfect image classification and perfect language modeling but fail at the joint task. Joint metrics catch this.

Q11. How do you handle imbalanced data across modalities?

Data-side: collect more data for underrepresented modalities. Oversample the minority modality. Use modality-specific augmentation (back-translation for text, augmentation for images).

Training-side: modality-specific learning rates. Loss balancing (e.g. multi-task uncertainty weighting).

Modality dropout: randomly mask one modality so the model learns to use either.

Architecture-side: separate processing branches per modality with appropriately scaled representations. Avoid bottlenecks that force one modality through a narrow channel.

Synthetic data: use a strong generator from the majority modality to produce training data for the minority. For example, use a TTS system to generate audio from text transcripts when audio is scarce.

Q12. Where is multimodal AI making a significant impact?

Healthcare: combining medical images (X-ray, MRI) with clinical notes for diagnosis; radiology report generation; pathology image analysis with patient history.

Autonomous systems: self-driving cars fuse camera, lidar, radar, and HD maps; drones combine visual and inertial data.

Content moderation: combining text, image, and video to flag policy violations.

E-commerce: visual search, image-based recommendation, AR product try-on.

Education: multimodal tutoring systems that parse handwritten work, voice questions, and visual diagrams.

Accessibility: image captioning for visually impaired users, real-time speech transcription, sign language recognition.

Creative tools: text-to-image (DALL-E, Stable Diffusion), text-to-video (Sora, Runway), text-to-music (Suno).

Q13. What future trends do you foresee in multimodal AI?

More modalities natively integrated: text + image + audio + video + 3D + structured data in one model.

Gemini and GPT-4o are early steps.

Longer context across modalities: hours of video, full documents with images, multi-day conversational history.

Better grounding: models that point to specific image regions or video timestamps for their claims, reducing hallucination.

Agentic multimodal systems: models that take actions across modalities (read a screenshot, click a button, summarize a video, generate a chart).

Smaller efficient multimodal models: on-device multimodal assistants, real-time AR/VR multimodal interfaces.

Ethical concerns: deepfakes (text + image + voice), bias amplification across modalities, copyright (training data sources), consent (when models can perceive and act in real environments).

Chapter 5

Word and Sentence Embeddings

This chapter covers 22 questions on word and sentence embeddings.

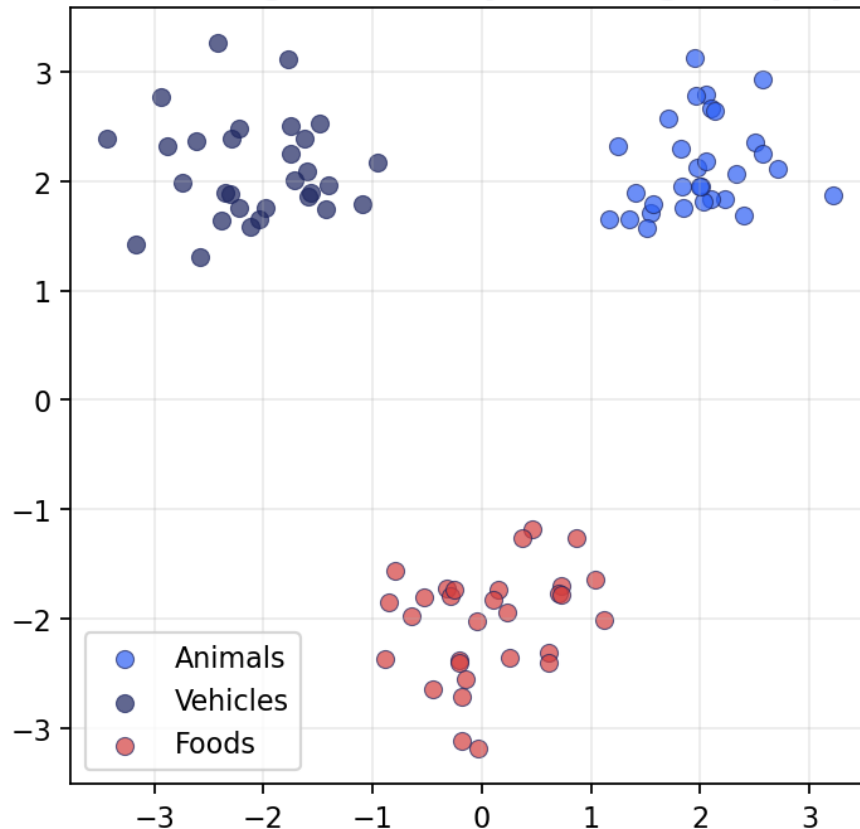
Q1. What is the fundamental concept of embeddings in ML?

An embedding is a dense vector representation of a discrete or complex object (word, sentence, image, user, product) such that geometric relationships in the vector space approximate semantic relationships in the original domain.

Compared to raw input or one-hot encodings, embeddings are much lower-dimensional (typically 64 to 1024 dimensions vs vocabulary size), dense (not mostly zero), and carry meaning (similar objects have nearby vectors).

Why this matters: embeddings let downstream models share information across related items. Similar inputs produce similar embeddings, so a classifier trained on a few examples generalizes to unseen ones. Retrieval, clustering, recommendation, and search all reduce to operations in embedding space.

Word embeddings cluster by meaning (2D projection)



Q2. Compare word embeddings and sentence embeddings.

Word embeddings represent individual words (or subwords). Static word embeddings (Word2Vec, GloVe, FastText) give each word one vector regardless of context. Contextual word embeddings (from BERT, GPT) give different vectors for the same word in different contexts.

Sentence embeddings represent an entire sentence as a single vector. The naive approach (average word embeddings) works poorly because it loses word order and compositionality. Modern sentence embedders (Sentence-BERT, MPNet, BGE, E5) fine-tune a pretrained transformer with a contrastive objective on pairs of semantically related sentences.

When to use each: word embeddings for token-level analysis, classical NLP features. Sentence embeddings for retrieval, similarity, clustering at the document or query level, RAG.

Q3. Explain contextual embeddings.

Contextual embeddings produce a different vector for the same word depending on its surrounding context. 'Bank' in 'river bank' and 'bank account' gets two different embeddings.

How BERT generates them: the input text is tokenized, embedded, and run through the transformer stack. The output at each token position is its contextual embedding (the final hidden state). Each token's embedding has been influenced by every other token via self-attention.

Advantages over static embeddings: handles polysemy correctly; captures syntactic role; produces representations that transfer to many downstream tasks. The cost is compute (running the encoder per query) and storage (you cannot precompute one vector per vocabulary word).

Q4. Discuss challenges and strategies for cross-modal embeddings.

Challenge 1: alignment. Embeddings from different modalities must live in the same or comparable space. Strategy: train with a contrastive objective on aligned pairs (CLIP), or train one modality to project into the other's space (Q-Former).

Challenge 2: scale and quality of paired data. Strategy: web-scale noisy pairs (LAION) plus high-quality curated pairs for fine-tuning.

Challenge 3: modality-specific dimensionality and structure. Strategy: encode each modality with a specialized architecture, then project to a shared dimension via a linear or MLP head.

Challenge 4: evaluation. Cross-modal retrieval (Recall@k) is the standard. Zero-shot classification (encode candidate labels as text, embed image, match) is a strong downstream test.

Q5. How do you capture embeddings for rare words?

Subword tokenization: BPE, WordPiece, SentencePiece break rare words into known subword pieces. 'cryptocurrency' becomes ['crypto', 'currency'], each with a well-trained embedding. The rare word's representation is composed from its parts.

Character-level fallback: encode characters as fallback for any token unseen at the subword level. FastText averages character n-gram embeddings.

Subsampling and rare-word boosting: oversample rare words during training so each gets enough gradient signal. Word2Vec and similar use sub-sampling to balance.

Domain-specific tokenizer extension: when working in a specialized domain (medical, legal), extend the tokenizer vocabulary with domain terms and briefly continue pretraining so the new tokens learn meaningful embeddings.

Q6. Discuss regularization techniques for embedding training.

L2 regularization on embedding norms: penalize large embeddings to prevent a few from dominating. Common in word2vec.

Dropout on embeddings: zero out random dimensions or random tokens during training to prevent overfitting to specific dimensions.

Weight tying: share embedding and output projection weights in language models. Reduces parameters and improves generalization.

Contrastive loss with hard negative mining: harder negatives provide a stronger regularization signal than random negatives.

Label smoothing: distribute a small probability mass across non-target classes; useful in classification heads on top of embeddings.

Early stopping based on a held-out evaluation metric (STS-B for sentence embeddings, MTEB for retrieval).

Q7. How are pretrained embeddings leveraged for transfer learning?

Load pretrained embeddings as initialization for a downstream model. The embedding layer is initialized from Word2Vec, GloVe, or a pretrained transformer's input embeddings. The rest of the model trains from scratch or is also pretrained.

Freeze vs fine-tune: freezing keeps the pretrained signal intact and reduces overfitting on small datasets. Fine-tuning adapts to the target domain but can destroy the pretrained knowledge if the learning rate is too high.

Two-stage training: train the new layers first with the embeddings frozen, then unfreeze and continue training with a small learning rate.

Embedding-only retrieval: use a frozen pretrained sentence embedder for retrieval in a RAG system. No task-specific training needed; the pretrained model is already strong on general similarity.

Benefit: dramatic data efficiency. A target task that would need millions of labels from scratch may need only thousands with pretrained embeddings.

Q8. What is quantization in embeddings, and what does it provide?

Quantization compresses high-precision (float32) embeddings into lower-precision representations: int8, int4, binary, or product-quantized codes. The result: 4-32x reduction in memory footprint with controlled accuracy loss.

Scalar quantization (int8, int4): each dimension is quantized to a fixed-precision integer with a per-vector or per-dimension scale and offset.

Product quantization (PQ): split each vector into sub-vectors; quantize each sub-vector to a codebook of typical sub-vectors. Used by FAISS for billion-scale vector search.

Binary embeddings: each dimension reduced to one bit. Hamming distance replaces cosine. 32x compression. Quality loss is real but acceptable for first-stage retrieval.

Tradeoff: lower precision means faster inference and smaller storage but degraded recall. Production systems often use int8 for retrieval with re-ranking on int32 for the final stage.

Q9. How do you implement embeddings for high-cardinality categorical features in tabular data?

Use a learned embedding layer per categorical feature. Each unique value maps to a low-dimensional vector (typically 4 to 64 dimensions). The embedding table has size (vocab_size, embedding_dim) and is trained jointly with the rest of the model.

Sizing: a common heuristic is $\min(50, \text{ceil}(\text{vocab_size}^{0.25} * 4))$. High-cardinality features (millions of users) may need larger dimensions but watch memory.

Handling rare categories: bucket all categories with frequency below a threshold into a single <RARE> token. Helps generalization and reduces noise.

Hash trick: for very high cardinality, hash the category into a fixed-size table. Trades hash collisions for bounded memory.

Production pattern: train the embedder jointly with downstream model. For online serving, export the embedding table as a lookup; new categories appearing at serve time map to the <UNK> embedding.

Q10. How do you efficiently search billions of embeddings?

Use approximate nearest neighbor (ANN) indexes. Brute-force search is $O(n \cdot d)$ per query and infeasible at billion scale.

HNSW (Hierarchical Navigable Small World): graph-based, fast queries, high recall. Default for many production systems. Memory-heavy.

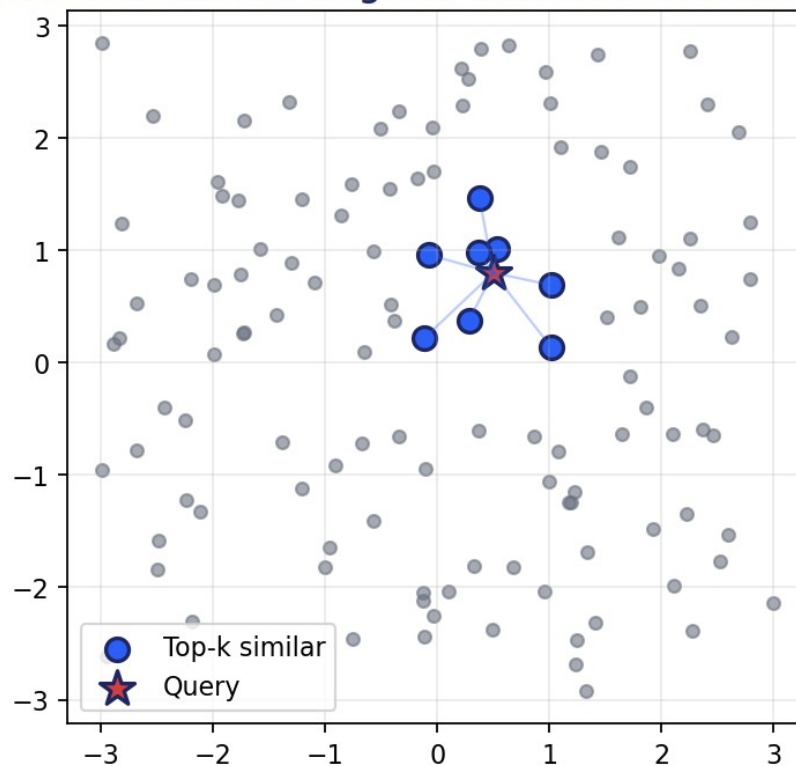
IVF (Inverted File): cluster the vectors into k centroids; at query time search only the most relevant clusters. Memory-friendly. Often combined with PQ.

Product Quantization (PQ): compress each vector for memory efficiency. Combine with IVF for billion-scale indexes.

FAISS, ScaNN, Annoy, NMSLIB are popular libraries. Pinecone, Weaviate, Qdrant, Milvus are managed vector databases.

Tuning: pick the index type and parameters that hit your recall and latency targets. Use a held-out test set to measure recall@k and 95th percentile latency.

Approximate nearest neighbor search in embedding space



Q11. How do you handle OOV words at embedding time in an LLM?

Modern LLMs use subword tokenizers (BPE, WordPiece, SentencePiece) that have no OOV problem at the input level: any string decomposes into known subword pieces. Even invented words become a sequence of meaningful subword embeddings.

Where OOV-like issues still arise: domain-specific terminology that decomposes poorly into general subwords (e.g. drug names, chemical compounds, code identifiers). Mitigation: extend the tokenizer with domain tokens and continue pretraining briefly.

Rare token quality: tokens that appear only a handful of times during pretraining have poor embeddings. Mitigation: subword fallback already helps; for critical rare tokens, supervised fine-tuning examples that exercise them improve quality.

Cross-lingual OOV: tokens from a language poorly represented in pretraining have weak embeddings. Mitigation: continue pretraining on target-language corpora.

Q12. How do you evaluate embedding quality?

Intrinsic evaluation: similarity benchmarks (STS-B, SICK) measure correlation between embedding cosine similarity and human similarity ratings. Word analogy tasks (king - man + woman = ?). Word similarity benchmarks (WordSim-353, SimLex-999).

Extrinsic evaluation: downstream task performance. Use the embeddings as features for classification, retrieval, clustering. Report task-specific metrics: accuracy for classification, Recall@k for retrieval, V-measure for clustering.

MTEB (Massive Text Embedding Benchmark): standardized benchmark across 50+ tasks in 8 categories. The de facto standard for comparing sentence embedders. Public leaderboard.

Domain-specific evaluation: build a held-out set of relevant queries and known answers in your domain. Measure Recall@k and Mean Reciprocal Rank.

Q13. Explain triplet loss in embedding learning.

Triplet loss trains embeddings such that an anchor is closer to a positive example than to a negative example by at least a margin. The loss for one triplet (a, p, n) is $\max(0, d(a, p) - d(a, n) + \text{margin})$, where d is distance.

Triplet sampling matters a lot. Random negatives are usually too easy (already far from the anchor) and provide little signal. Semi-hard negatives (negatives that are far enough from the anchor to satisfy the margin but still relatively close) provide the strongest gradient.

Use cases: face recognition (FaceNet), person re-identification, fine-grained similarity. InfoNCE has largely replaced triplet loss in modern embedding training because it handles many negatives per anchor in one batch and is more stable.

Q14. What is the margin in triplet or contrastive loss?

The margin is the minimum gap by which the anchor-positive distance must be less than the anchor-negative distance for the loss to be zero. A larger margin forces the model to learn more discriminative embeddings.

Practical effects: too small a margin (0.01) makes the task too easy and produces weak embeddings. Too large a margin (10) is hard to satisfy and slows learning. Typical values: 0.2 to 1.0 depending on the distance metric and normalization.

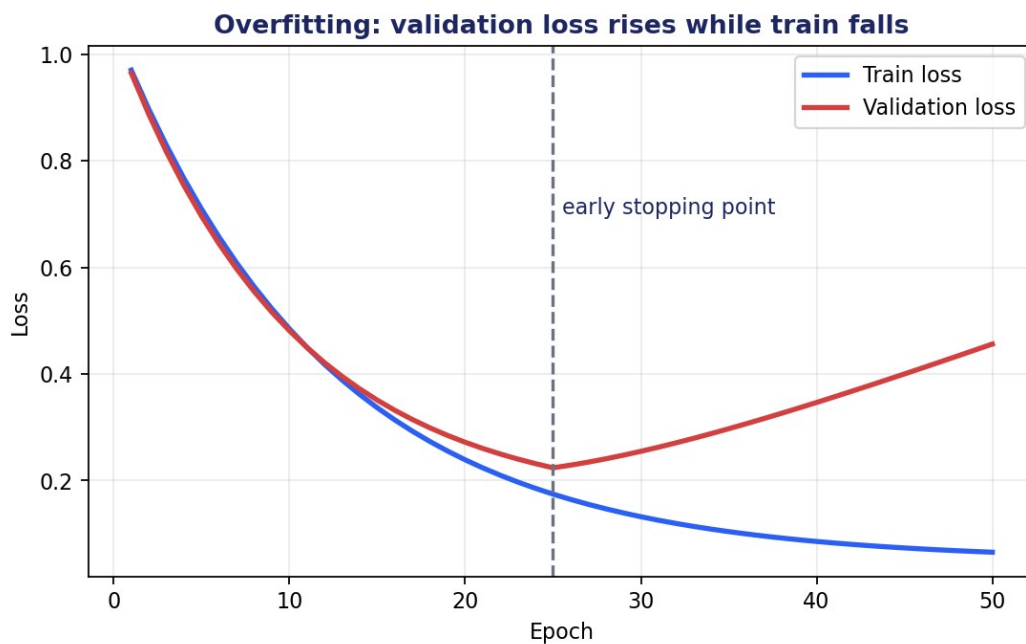
InfoNCE replaces the explicit margin with a temperature τ in $\text{softmax}(\text{score}/\tau)$. Lower τ behaves like a larger margin: only the closest negatives matter.

Q15. Discuss overfitting in LLMs during training and how to prevent it.

LLMs trained on massive web-scale corpora rarely overfit during pretraining. Overfitting becomes a real concern during fine-tuning on smaller specialized datasets.

Symptoms: training loss falls while held-out loss rises. Specific behaviors: memorization of training examples (the model recites them verbatim), narrow domain capability with degraded general reasoning (catastrophic forgetting).

Mitigation: parameter-efficient fine-tuning (LoRA, qLoRA) limits trainable parameters and naturally regularizes. Mix domain data with general data to preserve broad capability. Apply weight decay (typical 0.01 to 0.1). Use early stopping based on validation. Use lower learning rates ($1e-5$ to $5e-5$ for full FT, $1e-4$ for LoRA). Avoid overtraining: 1-3 epochs is usually enough for fine-tuning.



Q16. How do you adapt learning rates for LLM training stability?

Warm-up: linearly increase from near-zero to the target learning rate over the first 1-5% of training steps. Prevents huge initial updates that destabilize the model.

Decay: after warm-up, decay the learning rate. Cosine decay is the default for pretraining. Linear decay or constant-with-decay for fine-tuning.

Learning rate magnitude: pretraining typically uses $1e-4$ to $6e-4$ peak. Full fine-tuning uses $1e-5$ to $5e-5$. LoRA can use $1e-4$ to $3e-4$. RLHF/RL uses $1e-6$ to $5e-6$.

Layer-wise learning rate decay: smaller learning rates for earlier layers (which encode general patterns) and larger for later layers (which encode task-specific patterns). Helps prevent catastrophic forgetting during fine-tuning.

Monitor: training loss, gradient norm, parameter norm. Diverging loss or exploding gradient norm signal the learning rate is too high.

Q17. How do you handle long context lengths efficiently?

Architectural: use sparse or sliding-window attention (Longformer, Mistral's sliding window). Use linear attention or state-space models (Mamba). Use grouped-query or multi-query attention to reduce KV-cache size.

System: FlashAttention reduces memory and speeds up exact attention. KV-cache compression. KV-cache offloading to CPU or disk for very long sessions. PagedAttention (vLLM) manages KV-cache like virtual memory.

Modeling: extend position encodings via RoPE scaling, YaRN, ALiBi for length extrapolation. Continue pretraining on long-context data.

Application: RAG to inject only relevant chunks instead of stuffing the full context. Hierarchical summarization for very long documents. Sliding-window inference for streaming inputs.

Q18. What metrics judge LLM generation quality?

Reference-based: BLEU (translation), ROUGE (summarization), METEOR, CIDEr (captioning), exact match (QA). All measure n-gram overlap with reference, which correlates only weakly with quality for open-ended generation.

Embedding-based: BERTScore, BLEURT use contextual embeddings to compare candidate and reference. Better than n-gram overlap.

LLM-as-a-judge: a separate strong model rates the candidate against a rubric or pairwise against another candidate. Beware position, verbosity, and self-preference biases.

Human evaluation: still the gold standard for open-ended generation. Cohort ratings on fluency, relevance, factuality, helpfulness, harmlessness.

Task-specific: pass@1 / pass@k for code (HumanEval, MBPP). Win-rate on MT-Bench or Chatbot Arena for chat. Factuality on TruthfulQA. Reasoning on MMLU, BIG-Bench, GSM8K.

Q19. How do you evaluate and mitigate hallucinations in LLMs?

Detect: build a held-out fact-verification set. Compare model claims against a trusted knowledge base. Use an NLI model to check whether generated claims are entailed by retrieved evidence. TruthfulQA tests for known factual errors. FactScore decomposes claims and verifies each.

Mitigate at inference: RAG with high-quality retrieval keeps generation grounded in evidence. Force the model to cite sources and verify citations. Use lower temperature for factual tasks. Add chain-of-thought to expose reasoning.

Mitigate at training: fine-tune on data with explicit grounding (Self-RAG-style with critique tokens). Apply RLHF or DPO with preferences for grounded over ungrounded responses. Train on examples where the correct response is 'I don't know.' Reward calibrated uncertainty.

Monitoring in production: sample traffic for offline verification; alert when hallucination rate drifts; collect user feedback signals.

Q20. What is a Mixture of Experts (MoE) model?

MoE is an architecture where multiple specialized expert subnetworks (typically feed-forward blocks) coexist, and a learned router activates only a few experts per token. Total parameter count is large, but FLOPs per token stay close to a much smaller dense model.

Mechanics: a gating network produces a score per expert per token. The top-k experts (often $k=1$ or $k=2$) are activated. Each activated expert processes the token; outputs are combined weighted by gating scores.

Benefits: compute-efficient scaling. A 8x7B MoE can match a 70B dense model's quality at a fraction of the inference cost.

Challenges: load balancing (experts get specialized to specific kinds of input, but you want them used roughly equally), training stability (router behavior is non-smooth), memory (all expert weights must be in memory even if rarely used). Examples: Mixtral 8x7B, Switch Transformer, DeepSeek-V3.

Q21. Why is perplexity a problematic metric for LLM evaluation?

Perplexity measures average per-token uncertainty on a held-out corpus: lower is 'better at predicting the next token.' But many things matter for LLM utility that perplexity does not capture.

What perplexity misses: factuality (a fluent lie has low perplexity), reasoning (predicting the next token of a correct chain-of-thought is different from being able to reason), instruction following, harmlessness, calibration (knowing when to refuse), code correctness, multi-turn coherence.

Perplexity also depends on the evaluation corpus. A model with low perplexity on Reddit may have higher perplexity on legal text yet still be the better choice for legal tasks.

Better: combine perplexity (for sanity-check on representativeness) with task-specific benchmarks (MMLU, HumanEval, MT-Bench) and human evaluation.

Q22. How does Stable Diffusion leverage LLMs to generate high-quality images from text?

Stable Diffusion is a latent diffusion model. It consists of: (a) a VAE that encodes images to a small latent space (8x compression) and decodes back; (b) a U-Net that performs the diffusion process in the latent space; (c) a text encoder (CLIP or T5) that converts the text prompt into a sequence of embeddings.

The U-Net cross-attends to the text embeddings at every layer and timestep. This is the mechanism by which language conditions image generation. The text encoder must understand the prompt well enough that the U-Net can condition on it; this is why SDXL uses two text encoders and Stable Diffusion 3 uses a T5-XXL.

Process: encode the prompt with the text encoder; sample Gaussian noise in latent space; iteratively denoise the latent for 20-50 steps via the U-Net, conditioned on text embeddings; decode the final latent through the VAE to produce a pixel image.

Classifier-free guidance amplifies the text conditioning: at each step, compute both the conditional and unconditional denoising prediction, then push the prediction in the direction of the conditional prediction by a guidance scale.

Chapter 6

Retrieval-Augmented Generation (RAG) and Multimodal RAG

This chapter covers 34 questions on retrieval-augmented generation (rag) and multimodal rag.

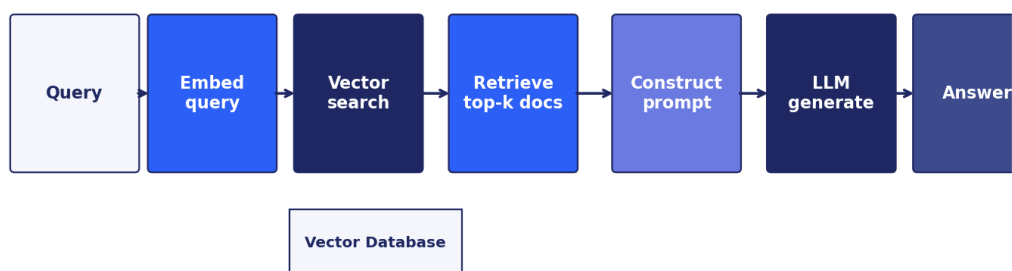
Q1. What is Retrieval-Augmented Generation (RAG)?

RAG is an architecture that combines a retrieval system with a generative language model. At inference time, the user query triggers a search over an external knowledge base; the retrieved passages are inserted into the LLM's prompt; the LLM generates a response grounded in those passages.

Why it matters: LLMs are limited to what they saw during training. Their knowledge is frozen, can be wrong, and cannot reference private or recent data. RAG patches this by giving the model up-to-date or proprietary information at inference time.

Pipeline: (1) ingest documents and chunk them; (2) embed chunks and store in a vector database; (3) at query time, embed the query, retrieve top-k similar chunks; (4) construct a prompt with the retrieved chunks and the user question; (5) generate a response with the LLM.

Retrieval-Augmented Generation pipeline



Q2. How does text generation differ between RAG and direct language models?

Direct LLM: the model generates from its parametric memory alone. It has whatever knowledge was baked in during pretraining. Cannot reference private data, recent events, or sources it has not seen. Prone to hallucination on facts it half-knows.

RAG: the model generates conditioned on retrieved evidence. The retrieved passages ground the response. The model can cite sources. Knowledge can be updated by updating the index, not retraining the model. Hallucinations are reduced when the retrieved evidence is good.

Quality tradeoff: RAG quality depends on both retrieval quality and generation quality. Bad retrieval leads to wrong context and poor answers. RAG also adds latency (retrieval step) and complexity (chunking strategy, index maintenance, reranking).

Q3. What are common applications of RAG?

Customer support assistants over internal documentation. Legal research over case law and contracts. Medical question answering over clinical guidelines and literature. Enterprise search across wikis, code, and emails. Compliance question answering. Educational tutoring grounded in course material. Code assistants that look up repository-specific functions. Personal assistants over notes and calendar. Open-domain QA with grounding for factuality.

Q4. How does RAG improve response accuracy?

Direct LLMs hallucinate when asked about specifics they only vaguely remember from pretraining. RAG narrows the scope: the model answers from a small, relevant set of passages, not from its full parametric memory.

Specific improvements: (a) factuality (the model can quote sources); (b) recency (the index can be updated as the world changes without retraining); (c) domain coverage (the model never saw your private documents during pretraining but can use them at inference); (d) auditability (every response can be traced back to its retrieved sources).

Effectiveness depends on retrieval quality. A precise, comprehensive retrieval stage is usually more important than the LLM choice for RAG quality.

Q5. Why are retrieval models significant in RAG?

The retrieval model is the gate between the user's question and the LLM's context. If the right passages are retrieved, the LLM has a chance to produce a good answer. If the wrong passages are retrieved, the LLM either ignores them (unhelpful) or grounds in them (wrong answer with confidence).

Retrieval quality drives almost every downstream metric: faithfulness, factuality, answer relevance, user satisfaction. Tuning the retrieval stage (embedder, chunking, hybrid sparse+dense, reranking) usually has higher ROI than tuning the generator.

Practical implication: invest in retrieval evaluation. Build a held-out query set with known relevant passages; measure Recall@k, NDCG, MRR. Iterate on the pipeline until retrieval is solid before optimizing the generator.

Q6. What data sources are typically used in RAG?

Unstructured text: internal wikis (Confluence, Notion), documentation, knowledge base articles, PDFs, emails, chat logs, customer support tickets.

Semi-structured: HTML pages, markdown, JSON documents, API responses.

Structured: SQL databases (via Text-to-SQL), data warehouses (BigQuery, Snowflake), spreadsheets, structured product catalogs.

Multimedia: images with captions, video transcripts, audio recordings (via ASR), diagrams with extracted text.

Code: source repositories indexed by file, function, or chunk; embedded with code-aware models.

Live: real-time APIs (weather, stock, news), web search results, sensor streams. Cached with appropriate TTL.

Q7. How does RAG contribute to conversational AI?

Conversational AI without grounding is limited to general knowledge. RAG lets a chatbot reference specific company policies, product details, account state, or regulatory rules. Each user turn triggers a retrieval over the relevant corpus.

Multi-turn handling: queries are often follow-ups that require rewriting before retrieval. 'When was it founded?' depends on the previous turn. A query rewriter (small LLM call) produces a standalone query from the conversation history before retrieval.

Memory: long-term conversational memory can itself be a RAG index. Past conversation summaries get embedded and retrieved when relevant to the current turn.

Production patterns: RAG combined with tool use, persistent memory, and identity-aware retrieval is the dominant pattern for production conversational AI.

Q8. What is the role of the retrieval component in RAG?

Retrieval selects the small subset of the knowledge base that goes into the LLM's prompt. Three roles in detail.

Filtering: out of millions of documents, return the few dozen most relevant to the query. Reduces noise that would confuse the generator.

Grounding: provides the evidence the LLM uses to construct its answer. The quality of grounding determines factuality.

Auditability: each retrieved passage is a source the user can verify. RAG systems that surface citations let humans check the model's claims.

Q9. How does RAG handle bias and misinformation?

RAG can both reduce and amplify bias depending on the corpus and pipeline.

Reduces bias by: grounding answers in vetted sources rather than the model's (potentially biased) parametric memory; making the source of each claim auditable; allowing curation of the knowledge base.

Amplifies bias when: the corpus itself is biased (the model now reliably parrots biased sources); retrieval is biased (e.g. always retrieves majority-perspective documents and ignores minority views); the generator was trained with bias and selectively uses retrieved evidence.

Mitigations: curate the corpus, audit for representation, use diversity in retrieval (avoid only top-k from one source), measure bias on a held-out test set, allow users to flag biased outputs.

Q10. What are the benefits of RAG over other NLP techniques?

Vs. pure prompting: RAG can access information not in the prompt or in the model's training data. No prompt length limit on knowledge base size.

Vs. fine-tuning: RAG updates by re-indexing, not retraining. Knowledge updates are fast and cheap. The model can cite sources. Fine-tuning bakes knowledge into weights opaquely.

Vs. building a custom model: RAG layers on top of an existing strong LLM. No training cost. Faster time-to-value.

Vs. search alone: search returns documents; the user has to read and synthesize. RAG returns synthesized answers grounded in those documents. Better for conversational and direct-answer use cases.

Q11. Describe a scenario where RAG is particularly useful.

Enterprise customer support: a SaaS company has 50K internal documentation pages, 10K Confluence articles, and 500K resolved support tickets. New tickets arrive at thousands per day. A direct LLM cannot answer specific questions about features, edge cases, or recent bug fixes.

RAG solution: index all docs, articles, and tickets with a domain-tuned embedder. At ticket creation, retrieve top relevant passages; assemble a prompt; have an LLM draft a response that cites sources. Human support reviews and edits before sending. Outcome: support tickets resolved 30-50% faster; agent satisfaction improves because they get a strong starting draft.

Why RAG wins here: knowledge changes daily (new features, new bugs); auditability matters (legal and brand-safety review); fine-tuning would lag behind reality.

Q12. How does RAG integrate with ML pipelines?

Ingestion pipeline: ETL job pulls documents from source systems (S3, databases, APIs), chunks them, embeds them, writes to the vector index. Runs continuously or on schedule. Same patterns as a feature engineering pipeline.

Serving pipeline: API service receives query, embeds it, calls the vector database for retrieval, optionally reranks, constructs prompt, calls the LLM, returns response with citations. Latency-sensitive.

Evaluation: held-out test sets for retrieval quality (Recall@k, NDCG) and generation quality (Ragas, LLM-as-judge). Continuous evaluation in production via sampled traffic.

Monitoring: track retrieval recall, generation latency, citation usage, user feedback signals. Alert on drift (corpus changes, query distribution shift).

Standard tools: LangChain, LlamaIndex for orchestration; Pinecone, Weaviate, Qdrant, Milvus for vector storage; Ragas for evaluation; OpenTelemetry for observability.

Q13. What challenges does RAG solve in NLP?

Knowledge staleness: pretrained LLMs are stuck at their training cutoff. RAG lets them answer questions about new information.

Hallucination: ungrounded generation is the source of many factual errors. Grounding in retrieved evidence reduces this.

Domain specialization: training a model on every enterprise's private data is impractical. RAG separates knowledge from model and lets each enterprise plug in its data.

Auditability and compliance: regulated industries need to trace outputs back to sources. RAG citations support audit trails.

Cost: fine-tuning per knowledge update is expensive. Re-indexing is cheap.

Q14. How does RAG keep retrieved information up to date?

Re-indexing pipelines: when source documents change, re-embed and update the vector store. Patterns: full re-index on a schedule (nightly, weekly); incremental re-index (only changed documents); event-driven (re-index when source system signals a change).

Time-aware retrieval: store creation/update timestamps on each chunk; weight recency in scoring or filter to recent documents only.

TTL on cached data: for live API-backed sources (news, prices), use short TTLs or pass-through to the live API rather than caching in the vector store.

Versioning: track which version of each document is in the index. Roll back if an ingestion job introduces bad data.

Detection: monitor for stale answers via user feedback signals or periodic ground-truth checks against the source system.

Q15. How are RAG models trained?

RAG models can be trained end-to-end or assembled from pretrained components.

Component-wise (most common in practice): use a pretrained sentence embedder for retrieval and a pretrained LLM for generation. No additional training needed. Fine-tune the embedder on in-domain (query, relevant-doc) pairs to improve retrieval recall. Fine-tune the LLM on grounded-generation examples to improve faithfulness.

End-to-end (original RAG paper): jointly train the retriever and the generator with marginal likelihood over retrieved documents. The retriever learns to retrieve documents that help generation. Expensive but produces strongly coupled components.

REALM, DPR, RAG, FiD, RETRO, Self-RAG, Atlas are research benchmarks worth studying. Production systems usually start with component-wise and tune from there.

Q16. What is the impact of RAG on LLM efficiency?

Latency: RAG adds retrieval latency (typically 50-300 ms) and may increase generation latency because the prompt is longer.

Cost: longer prompts mean more input tokens, which dominate LLM cost for many workloads. Mitigations: aggressive reranking to send fewer chunks, prompt caching for repeated context.

Throughput: RAG is throughput-friendly because retrieval is cheap and parallelizable. The bottleneck is the LLM.

Net efficiency: for tasks RAG enables (questions over private docs, recent events), RAG is dramatically cheaper than fine-tuning. For tasks the LLM could handle alone, RAG is overhead. Apply RAG only where it adds clear value.

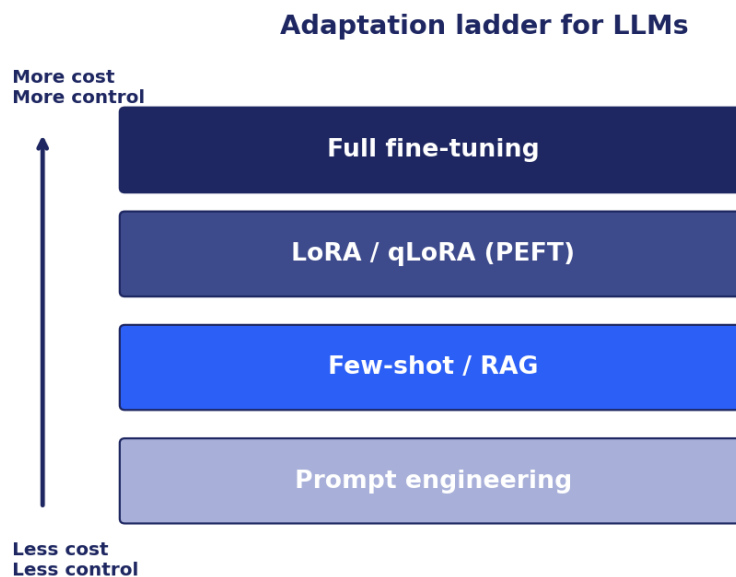
Q17. How does RAG differ from Parameter-Efficient Fine-Tuning (PEFT)?

RAG provides knowledge externally via retrieval. PEFT modifies a small number of model parameters to teach the model new behavior or knowledge.

When to use RAG: knowledge changes often, answers must cite sources, domain has lots of factual content. Auditability matters.

When to use PEFT: behavior change (tone, persona, format), narrow task specialization, latency-sensitive (no retrieval step). Knowledge is stable.

Combine: many production systems use both. PEFT teaches the model the right format, persona, and reasoning patterns; RAG provides factual grounding at inference. The combination beats either alone for many enterprise tasks.



Q18. How can RAG enhance human-AI collaboration?

Citations: RAG outputs cite sources, letting humans verify claims and learn from the underlying material. This shifts AI from 'magic answer' to 'researcher who shows their work.'

Trust calibration: users see what evidence the model used. If the evidence is weak, the user can override or seek more sources.

Iteration: humans can refine queries based on retrieved evidence. 'These passages aren't quite what I meant. Let me rephrase.'

Editing workflows: in customer support and content creation, RAG drafts grounded responses; humans review and edit. The human-AI pair is faster than either alone.

Knowledge discovery: users find related documents they didn't know existed. RAG becomes a discovery tool, not just an answer tool.

Q19. Explain the technical architecture of a RAG system.

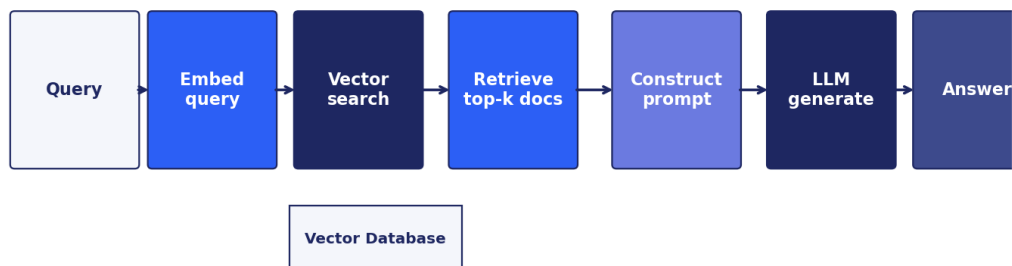
Ingestion: connectors pull documents from sources (S3, GitHub, Notion). Document loaders parse formats (PDF, HTML, DOCX). Chunkers split into 200-1000 token chunks with overlap or semantic boundaries. Embedder generates dense vectors. Vector store persists with metadata.

Retrieval: query embedder produces query vector. Vector store performs ANN search for top-k similar chunks. Optional: hybrid with BM25 via reciprocal rank fusion. Optional: cross-encoder reranker rescores the top candidates.

Generation: prompt template combines retrieved chunks with the user query. LLM generates the response, optionally with citations. Post-processing: format citations, run guardrails, log for evaluation.

Cross-cutting: caching (semantic cache for repeated queries), observability (trace each retrieval and generation), evaluation (sample traffic for offline scoring), security (PII handling, prompt injection defense).

Retrieval-Augmented Generation pipeline



Q20. How does RAG maintain context in a conversation?

Query rewriting: a small LLM call rewrites the latest user turn into a standalone query using the conversation history. 'When was it founded?' becomes 'When was Acme Corp founded?' before retrieval.

Conversation memory as a RAG source: past turns or summaries get embedded and indexed. Long conversations no longer fit in the prompt, but their key points can be retrieved when relevant.

Sliding context window: maintain a rolling window of recent turns plus a summarized history of earlier turns. Combines short-term verbatim memory with long-term semantic memory.

Session-level retrieval state: cache documents already retrieved this session so the model doesn't keep re-introducing them.

Q21. What are the limitations of RAG?

Quality ceiling: RAG cannot exceed the quality of its retrieval. If relevant documents exist but retrieval misses them, the answer suffers.

Latency: extra round-trip to the vector store. Hard to keep total latency below 1 second for complex pipelines.

Cost: longer prompts mean more tokens. For high-volume applications, RAG can be more expensive than a fine-tuned model.

Multi-hop questions: simple top-k retrieval struggles with questions that need to combine evidence from multiple unrelated documents.

Schema mismatch: structured questions over tabular data are not RAG's strength (use Text-to-SQL instead).

Index maintenance: ingestion pipelines need monitoring; stale or incorrect indexes silently degrade quality.

Q22. How does RAG handle complex queries requiring multi-hop reasoning?

Vanilla RAG often fails on multi-hop. Mitigations: query decomposition, iterative retrieval, GraphRAG.

Query decomposition: an LLM splits the complex query into sub-queries. Each sub-query retrieves; partial answers feed the next sub-query. 'What is the GDP of the country where Apple is headquartered?' becomes (1) where is Apple headquartered, (2) what is the GDP of [USA].

Iterative agentic RAG: the model retrieves, generates partial answer, decides what to retrieve next based on what it now knows. Continues until the question is answered or a step limit is reached.

GraphRAG: build a knowledge graph from the corpus. Multi-hop queries traverse the graph rather than relying on a single vector similarity. Microsoft GraphRAG and LightRAG are reference implementations.

HyDE (Hypothetical Document Embeddings): generate a plausible answer first; embed and use as the retrieval query. Closes the gap between question style and document style.

Q23. Discuss the role of knowledge graphs in RAG.

Knowledge graphs encode entities and their relationships explicitly. A RAG system can use a graph to follow chains of relationships, find structural patterns, and answer questions that pure text retrieval misses.

GraphRAG pattern (Microsoft): extract triplets from documents with an LLM, build a graph, detect communities, summarize each community. At query time, retrieve relevant community summaries (for global questions) or entity neighborhoods (for specific questions).

Hybrid: use vector retrieval for fact-style questions and graph traversal for relational or multi-hop questions. A router classifies the question and dispatches accordingly.

Tradeoffs: graph construction is expensive (one LLM call per chunk). Graph queries can be slow without good indexing. Maintenance is harder than a flat vector index. Worth it when multi-hop reasoning matters.

Q24. What are the ethical considerations when implementing RAG?

Source bias: a corpus dominated by one perspective produces biased outputs even if the model is unbiased. Audit the corpus for representation.

PII and privacy: indexed documents may contain personal data. Apply PII detection, redaction, and access controls. Ensure retrieval respects authorization (one user cannot retrieve another user's documents).

Misinformation: RAG can authoritatively present misinformation if the corpus contains it. Curate trusted sources; flag low-confidence retrievals.

Copyright: indexing third-party content may have licensing implications. Track data provenance and licenses.

Transparency: surface sources to users. Disclose that responses are AI-generated.

Hallucination risk: even with citations, models can misrepresent sources. Test for grounding faithfulness, not just citation presence.

Q25. What is multimodal RAG, and how does it differ from text-only RAG?

Multimodal RAG retrieves and reasons over data from multiple modalities: text, images, tables, audio, video. The retrieval may return mixed-modality results; the generator may be a multimodal LLM that takes images and text as input.

Architectures: (a) shared embedding space (CLIP-style) for images and text, with a multimodal LLM (GPT-4V, Gemini) for generation; (b) per-modality indexes with a router that picks the right one; (c) text-only retrieval over image captions and OCR with a text-only LLM (cheaper but loses visual nuance).

Differences from text-only RAG: more chunking choices (how to chunk a document with images); more retrieval choices (which modalities to retrieve for a given query); generation depends on a multimodal LLM; evaluation needs multimodal metrics.

Q26. How can multimodal data improve RAG?

Richer context: a financial report's text plus its charts; a medical record's text plus images; a product page's description plus photos. Retrieving across modalities gives the LLM the full picture.

Cross-modal queries: 'find me products that look like this image' or 'show me the trend in this chart.' Pure text retrieval cannot answer these.

Better grounding: visual evidence is harder to misinterpret than paraphrased text. A retrieved chart that shows the trend is more faithful than a text description of it.

User flexibility: users can ask questions with mixed inputs (text query + image), matching natural human communication.

Q27. What are challenges of implementing multimodal RAG?

Alignment across modalities: text and image embeddings must live in a comparable space. Use CLIP-style embedders or modality-specific embedders with explicit alignment.

Heterogeneous chunking: a PDF page contains text, tables, and images. How to split? Often each non-text element becomes its own chunk with surrounding text as context.

Generation cost: multimodal LLMs are typically more expensive and slower than text-only LLMs.

Evaluation: no single metric covers retrieval, generation, and modality interaction. Combine per-modality metrics with end-task metrics.

Data quality: paired multimodal data is scarce. Web-scraped pairs are noisy. Curate carefully.

Privacy: images may contain identifiable information (faces, addresses on documents, location metadata). Apply detection and redaction.

Q28. Describe a real-world multimodal RAG application.

Healthcare diagnostic assistance: ingest patient records (text), lab results (tables), medical imaging (X-rays, MRIs), and clinical literature. At consult time, retrieve relevant prior cases, imaging, and literature for the current presentation. A multimodal LLM (GPT-4V or Med-PaLM) drafts a differential diagnosis grounded in retrieved evidence.

Benefits over unimodal: doctors see the patient's record, similar prior images, and relevant literature together. Decisions are better calibrated. Drafts save documentation time. Citations preserve the audit trail required for regulated use.

Other domains: e-commerce (text + images for product search), education (text + video + diagrams), legal (text + scanned exhibits), real estate (text + photos + floorplans).

Q29. What evaluation metrics suit multimodal RAG?

Retrieval metrics, per modality: Recall@k, NDCG, MRR computed separately for text and image (and other modality) retrieval. Also cross-modal: image-to-text Recall@k, text-to-image Recall@k.

Generation metrics: BLEU/ROUGE for text generation. CIDEr for caption-style outputs. CLIPScore for text-image alignment.

End-to-end: Ragas faithfulness (extended to handle visual claims), answer relevance, context precision/recall.

Multimodal-specific: visual question answering accuracy, citation accuracy (does the cited image actually contain the referenced content?), modality coverage (did the system use the right modality for the query?).

Human evaluation: multimodal generative outputs almost always need humans in the loop for final quality judgment.

Q30. How would you design a multimodal RAG for a specific industry?

Take healthcare as an example. Components: (1) ingestion pipeline for EHRs (text), DICOM imaging (medical images), lab CSVs (structured); (2) modality-specific embedders (clinical-text encoder, medical-image encoder, table encoder); (3) per-modality vector indexes with PHI-aware access control; (4) query router that classifies the query and picks the right index(es); (5) retrieval with reranking; (6) multimodal LLM with HIPAA-compliant deployment; (7) citation and explanation layer; (8) human-in-the-loop verification before any clinical use.

Cross-cutting: PHI redaction at ingestion and serving, audit logs for every retrieval, role-based access control, evaluation set built with clinician annotation, monitoring for drift and hallucination.

Q31. Techniques for aligning and integrating modalities in a RAG pipeline?

Shared embedding space (CLIP-style): one embedder per modality, trained jointly with contrastive loss. Image and text embeddings are directly comparable.

Cross-modal translation: convert one modality to another at ingestion. Generate captions for images; extract text from documents via OCR; transcribe audio. Then index everything as text. Cheap, but loses non-textual nuance.

Late fusion: retrieve from each modality separately, then combine results via reciprocal rank fusion or weighted reranking.

Early fusion: train a single multimodal encoder that takes inputs from multiple modalities and produces a joint representation.

Hybrid: most production systems combine approaches. Caption images for fast text-only retrieval, but also keep the original images for visual reranking and for the multimodal generator.

Q32. How do you evaluate generated content in multimodal RAG?

Reference-based metrics: BLEU/ROUGE for text outputs vs reference text. CIDEr for image captioning style. BLEURT and BERTScore as embedding-based alternatives.

Grounding metrics: faithfulness (every claim in the output supported by retrieved evidence). Citation accuracy (cited documents actually contain the claim). Modality grounding (visual claims supported by the cited image).

Task-specific: VQA exact match for question-answer pairs. Slot-fill accuracy for structured extraction. Click-through or engagement for recommendation tasks.

Human evaluation: panel ratings on relevance, accuracy, helpfulness. Especially important for open-ended generation.

User-facing feedback signals: thumbs up/down, regenerate rate, time spent reading. Aggregate over time and segment by query type.

Q33. What challenges arise when scaling multimodal RAG to large datasets?

Storage: image and video embeddings are not larger than text embeddings, but the raw assets are. Plan for multi-TB storage at scale.

Latency: retrieving and processing visual content takes longer than text. Use lazy loading, caching, and reranking to keep P95 manageable.

Index maintenance: when source images change, embeddings must be recomputed. Plan incremental updates.

Cost: multimodal LLMs are 5-20x more expensive per query than text LLMs. Use them only when the query genuinely needs visual reasoning.

Quality at scale: noisy data drowns out signal as the index grows. Curate ingestion sources. Apply quality filters.

Evaluation at scale: cannot human-review every query. Build automated evaluations and sample for human review.

Q34. Give an example of an ethical concern in multimodal RAG, and how to mitigate it.

Example: a medical multimodal RAG retrieves images and records. A breach of access control exposes a patient's images and history to an unauthorized user. Or: the system reliably retrieves images of one demographic when given clinical queries about it, reinforcing stereotypes in case reviews.

Mitigations for the breach: role-based access control at the retrieval layer, not just the application layer. Audit logs for every retrieval. PHI redaction for any image leaving the system. Encryption at rest and in transit.

Mitigations for bias: train the embedder on a demographically balanced corpus. Audit retrievals for demographic skew. Add diversity-aware reranking. Surface the limitation to users and clinicians.

Process: ethics review before deployment, with clinical and patient representatives. Continuous monitoring after deployment.

Chapter 7

Fine-Tuning Large Language Models

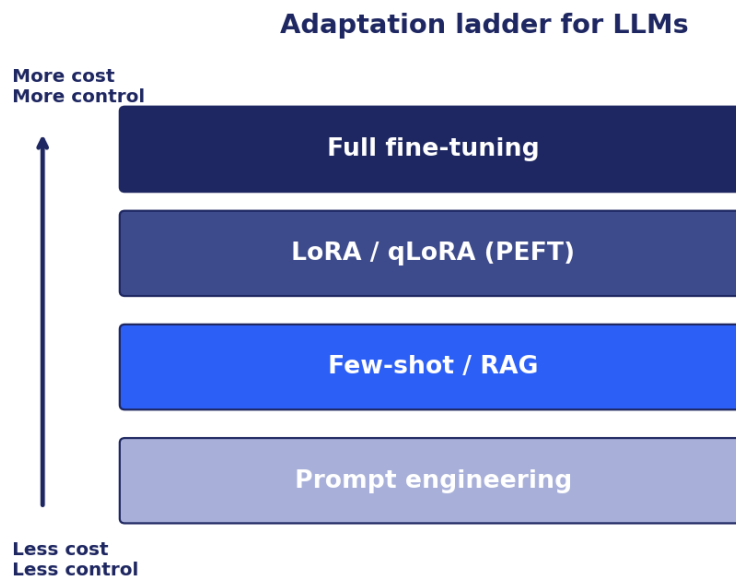
This chapter covers 26 questions on fine-tuning large language models.

Q1. What is fine-tuning?

Fine-tuning is the process of continuing to train a pretrained model on a smaller, task-specific or domain-specific dataset. The model's weights are updated so that it specializes for the new task while retaining knowledge from pretraining.

Why it works: pretraining produces general capabilities; fine-tuning shapes those capabilities for a specific purpose. A code-completion model fine-tuned on your company's codebase becomes much better at your internal patterns. A general LLM fine-tuned on medical notes becomes a credible medical assistant.

Modern variants: full fine-tuning updates every weight. Parameter-efficient fine-tuning (LoRA, qLoRA, adapters) updates a small additional set of parameters while keeping the base frozen. Instruction tuning fine-tunes on (instruction, response) pairs. RLHF aligns the model to human preferences.



Q2. Describe the fine-tuning process step by step.

1. Choose a base model. Match capability and size to your task. Smaller models are cheaper to fine-tune; larger models often produce better results.
2. Prepare data. Collect (input, output) pairs in the format the model expects. Quality matters more than quantity. Deduplicate, filter, balance.
3. Choose method. Full fine-tuning for maximum quality and ample compute. LoRA or qLoRA for cost efficiency and to avoid catastrophic forgetting.
4. Configure training. Learning rate ($1e-5$ to $5e-5$ for full FT; $1e-4$ to $3e-4$ for LoRA). Batch size as large as fits. Epochs 1-3 typically. Warm-up and decay.
5. Train. Monitor train and validation loss. Watch gradient norm and parameter norm. Apply early stopping if validation diverges.
6. Evaluate. Held-out test set for the target task. Regression suite for base capabilities. Compare against base and against alternative methods.
7. Deploy. Merge LoRA into base if applicable. Quantize if needed. Serve behind vLLM or similar. Monitor in production.

Q3. What are different fine-tuning methods?

Full fine-tuning: update every weight. Maximum quality. Highest cost (compute, memory, storage). Risk of catastrophic forgetting.

LoRA (Low-Rank Adaptation): freeze base; learn a low-rank update BA. Trainable params drop to 0.5-2% of full. Quality often within 1-2 points of full FT. The default for most fine-tuning workloads.

qLoRA: LoRA on a 4-bit quantized base. Cuts memory by 4x. Modest accuracy loss vs LoRA. Enables fine-tuning 70B models on a single 80GB GPU.

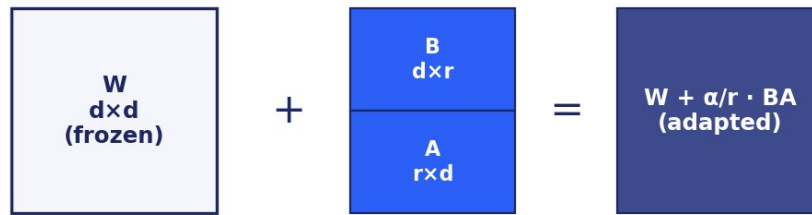
Adapters: insert small trainable modules between transformer layers; freeze the rest. Similar to LoRA in spirit, less popular than LoRA.

Prefix tuning / Prompt tuning: prepend a small set of trainable virtual token embeddings; freeze the rest. Very few trainable params. Lower quality ceiling than LoRA.

BitFit: train only bias terms. Tiny parameter footprint. Limited expressiveness.

Instruction tuning: full or LoRA FT on a curated mix of (instruction, response) pairs to make a base model follow instructions.

RLHF/DPO/GRPO: alignment-stage fine-tuning using human preferences or verifiable rewards.

LoRA: low-rank adapter on a frozen weight

$\text{rank } r \ll d \rightarrow \text{only } B \text{ and } A \text{ are trainable}$

Q4. When should you fine-tune?

Fine-tune when:

- Prompting alone consistently misses your target quality or format.
- You need a specific persona, tone, or output format the base model doesn't reliably produce.
- You have a narrow domain where vocabulary or reasoning patterns differ from general pretraining.
- You have ample examples (typically thousands of high-quality (input, output) pairs) of the desired behavior.
- Latency and cost favor a smaller fine-tuned model over a larger general one.

Do NOT fine-tune when:

- Your task is primarily about knowledge that changes often (use RAG instead).
- You have very little training data (under a few hundred examples).
- The task is well-handled by prompting and few-shot examples.
- You cannot evaluate the fine-tuned model rigorously (you will not catch regressions).

Q5. What is the difference between fine-tuning and transfer learning?

Transfer learning is the broader concept: take knowledge learned in one setting and apply it to another. It includes using features from a pretrained model as input to a new classifier (feature extraction), prompt-based zero/few-shot, RAG, and fine-tuning.

Fine-tuning is a specific transfer learning technique: continue training the pretrained model on target data, updating some or all of its weights.

In modern NLP, 'transfer learning' is sometimes used as a synonym for 'fine-tuning' but the broader meaning includes any reuse of pretrained knowledge.

Q6. Write about instruction fine-tuning and explain how it works.

Instruction fine-tuning teaches a base LLM to follow natural-language instructions. The training data consists of (instruction, response) pairs across diverse task types: 'Summarize this passage', 'Translate to French', 'Write a Python function that...'.

Mechanics: standard supervised fine-tuning with a causal LM objective. The training input is the instruction (and any provided input data); the target is the desired response. Loss is computed only on response tokens (masking out the instruction itself).

Why it works: the base model already has strong language modeling capabilities. Instruction tuning teaches it the pattern of (instruction → response) and exposes it to many task types. This generalizes: the model becomes good at instructions it never saw, because it has learned the meta-pattern.

Datasets: FLAN (Google), Alpaca, Dolly, OpenAssistant, ShareGPT-cleaned. Modern instruction-tuned models (LLaMA-Chat, Mistral-Instruct, Gemma-Instruct) are built this way.

Q7. Explain RLHF in detail.

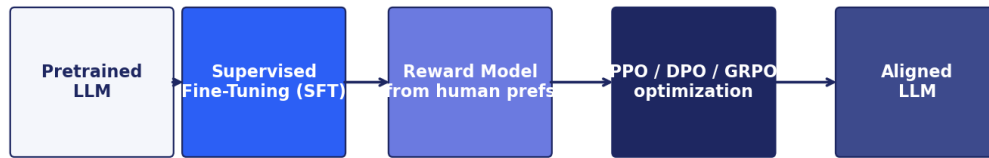
Reinforcement Learning from Human Feedback is the standard alignment pipeline. Three stages.

Stage 1: SFT. Supervised fine-tune the base model on (prompt, ideal response) pairs crafted by humans. The model learns to follow instructions in a desirable style.

Stage 2: Reward Model. For each prompt, generate multiple responses from the SFT model. Have humans rank them (typically pairwise). Train a separate reward model $RM(prompt, response) \rightarrow \text{scalar}$ to predict which response humans prefer. The reward model encodes 'what humans like.'

Stage 3: Policy optimization. Use the reward model as the reward signal to fine-tune the SFT model with PPO. Each step: sample a prompt, generate a response, compute $\text{reward} = RM(prompt, response) - \beta \cdot KL(\text{policy} || \text{SFT})$. The KL term prevents the policy from drifting into reward-hacking regions far from the SFT distribution.

Result: an LLM that is helpful, harmless, and honest within the limits of the preference data. Used to train ChatGPT, Claude, LLaMA-Chat. Modern variants include DPO (skip the reward model), GRPO (drop the value head), and Constitutional AI (AI feedback instead of human).

RLHF: from pretrained base to aligned LLM**Q8. Write about different RLHF techniques.**

Classic RLHF (PPO): three-stage pipeline with SFT, reward model, PPO. Used in ChatGPT-3.5, Claude-1.

Direct Preference Optimization (DPO): skips the explicit reward model. Trains directly on preference pairs (prompt, chosen, rejected) using a closed-form objective derived from the optimal RLHF policy. Simpler pipeline, no sampling at training time, more stable. Lower compute. Often matches PPO quality.

Group Relative Policy Optimization (GRPO): drops the value network. Sample N completions per prompt; compute group-relative advantage as reward minus group mean. Cuts memory by roughly half. Default for math and code RL training (DeepSeekMath, DeepSeek-R1).

RLAIF (RL from AI Feedback): replace human preferences with AI-generated preferences. Scales better than human-only. Used in Constitutional AI.

RLVR (RL with Verifiable Rewards): for tasks with checkable correctness (code compiles, math answer matches), skip the reward model and use the verifier as reward. Faster, less reward hacking. Used for code, math, theorem proving.

PPO-variants: ReMax, ORPO (preference optimization combined with reference-free training), KTO (Kahneman-Tversky Optimization).

Q9. Explain PEFT in detail.

Parameter-Efficient Fine-Tuning (PEFT) refers to methods that adapt a large pretrained model by training a small fraction of its parameters (or a small additional module) while keeping the base frozen.

Motivations: full fine-tuning of a 70B model needs hundreds of GB of GPU memory and is slow. PEFT methods cut trainable parameters to 0.1-2% of the base while preserving most quality.

Main families: (a) LoRA-style low-rank updates; (b) adapter modules inserted between layers; (c) prefix/prompt tuning of trainable virtual tokens; (d) BitFit training only biases; (e) IA3 scaling activations.

Practical benefits: train multiple task-specific adapters cheaply; swap them at inference for multi-tenant or multi-task serving; avoid catastrophic forgetting (base weights stay intact); ship a small adapter file (megabytes) instead of a full model checkpoint (gigabytes).

Hugging Face's `peft` library is the de facto standard implementation. Supports LoRA, qLoRA, adapters, prefix tuning, IA3.

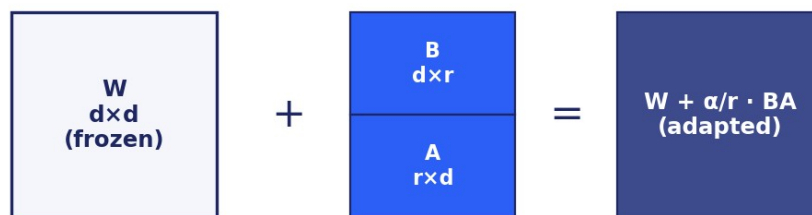
Q10. What is LoRA and qLoRA?

LoRA (Low-Rank Adaptation): freeze the pretrained weight W . Add a low-rank update BA where A is $r \times d$ and B is $d \times r$, rank $r \ll d$ (typically $r = 4$ to 64). The forward pass becomes $W \cdot x + \alpha/r \cdot BA \cdot x$. Only B and A are trained. Trainable parameter count drops to about $r \cdot (d_{\text{in}} + d_{\text{out}})$ per layer, often 0.5% to 2% of full fine-tuning. Quality is usually within 1-2 points of full FT.

qLoRA: LoRA on top of a 4-bit quantized base model. The base is loaded in NF4 (normalized 4-bit floating-point with double quantization). LoRA adapters train in 16-bit. Memory drops 4x; accuracy typically within 1-2 points of LoRA on 16-bit base. Enables fine-tuning a 70B model on a single 80GB GPU.

Tradeoffs: very small ranks ($r=1$, $r=2$) can hurt quality on tasks with strong semantic shift. Higher ranks ($r=32$, $r=64$) are safer but cost more. The α parameter scales the contribution; a common rule is $\alpha = 2r$.

LoRA: low-rank adapter on a frozen weight



$\text{rank } r \ll d \rightarrow \text{only } B \text{ and } A \text{ are trainable}$

Q11. Define pre-training vs fine-tuning in LLMs.

Pre-training: train a model from scratch (or from random init) on a massive general corpus using a self-supervised objective (causal LM, masked LM). The result is a base model with broad capabilities but no specific task alignment. Costs hundreds to thousands of GPU-years for frontier models.

Fine-tuning: continue training a pretrained model on a smaller, task-specific or alignment dataset. The base model's general capabilities are specialized for the task. Costs hours to days for most workloads.

Pre-training happens once per model release (very expensive). Fine-tuning happens many times by many users (much cheaper). The economics of foundation models hinge on this asymmetry: pay once to pretrain, reuse forever for many fine-tuning downstream tasks.

Q12. How do you train LLMs with billions of parameters?

Distributed training. Three parallelism strategies in combination.

Data parallel: same model on every device, sharded data, all-reduce on gradients. Default for everything that fits.

Tensor parallel: split each matrix multiplication across devices. Required when the model is too large for one device.

Pipeline parallel: split layers across devices; stream micro-batches through. For very large models.

ZeRO/FSDP: shard optimizer states, gradients, and parameters across devices to fit larger models. Stage 1, 2, 3 shard progressively more.

Mixed precision: store weights in BF16 or FP8; do most computation in low precision; keep an FP32 master copy of the optimizer state.

Gradient checkpointing: trade compute for memory by recomputing activations during the backward pass instead of storing them.

Training infrastructure: Megatron-LM, DeepSpeed, Hugging Face Accelerate, Ray Train, FairScale. Data pipeline: WebDataset or MosaicML Streaming for efficient streaming. Logging and checkpointing: Weights & Biases, MLflow.

Q13. How does LoRA work mechanically?

Pick a pretrained weight matrix W of size $d_{\text{out}} \times d_{\text{in}}$ (in a linear or attention projection). Freeze W .

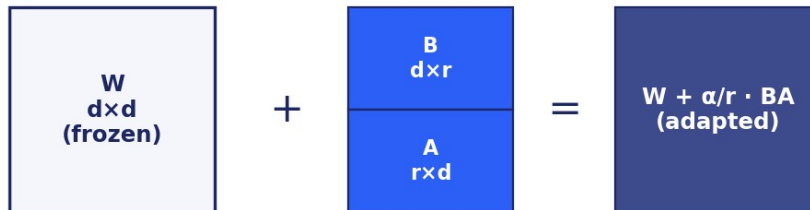
Initialize a pair of matrices: B ($d_{\text{out}} \times r$) initialized to zero and A ($r \times d_{\text{in}}$) initialized with a small random distribution. The product BA has the same shape as W but is constrained to rank $r \ll \min(d_{\text{in}}, d_{\text{out}})$.

The adapted layer computes: $y = W \cdot x + (\alpha/r) \cdot B \cdot A \cdot x$. Only B and A receive gradients during training. The α/r factor scales the LoRA contribution; α is typically set to $2r$ so the effective scaling is 2.

At inference: optionally merge by computing $W' = W + (\alpha/r) \cdot BA$ once, store W' , and serve as if it were a plain weight matrix. Zero inference overhead vs the base model after merging.

Where to apply: query, value (sometimes key), and output projections in attention layers. Sometimes MLP. The right set is empirical; start with Q, V, O.

LoRA: low-rank adapter on a frozen weight



$\text{rank } r \ll d \rightarrow \text{only } B \text{ and } A \text{ are trainable}$

Q14. How do you train an LLM that prevents prompt hallucinations?

Pretraining: filter the training corpus to favor high-quality, factual sources. Deduplicate aggressively; duplicates teach the model to be overconfident.

SFT: include high-quality (instruction, response) pairs where the response is grounded, conservative, and willing to say 'I don't know.' Avoid examples that reward confident-but-wrong responses.

Reward modeling and RLHF: explicitly reward correct refusals and grounded answers. Penalize unsupported claims. Add adversarial examples designed to elicit hallucinations to the preference data.

RAG at inference: ground every response in retrieved evidence. Require citations. Verify citations match claims with an NLI model.

Calibration training: include examples that teach the model to express uncertainty appropriately. Self-consistency: sample multiple times and pick the consistent answer.

Evaluation: build hallucination benchmarks (TruthfulQA, FactScore). Monitor hallucination rate in production via sampling and offline grounding checks.

Q15. How do you prevent bias and harmful prompt generation?

Data curation: filter pretraining and fine-tuning data for harmful content. Use demographic-balanced datasets. Audit for representation across groups.

Constitutional principles: define explicit principles for desired model behavior. Use them to filter SFT examples and to generate AI preferences (Constitutional AI, RLAIIF).

RLHF alignment: collect human preferences across diverse demographics. Reward responses that avoid stereotypes and harmful content.

System prompts: production deployments add a system prompt that establishes behavior rules, including refusal of harmful requests.

Inference-time guardrails: input filters (intent classification, content moderation) and output filters (toxicity, PII, hate speech detection). Multi-layer defense.

Red teaming: actively probe the model for failure modes before deployment. Continuous adversarial evaluation in production.

Transparency: publish model cards documenting known limitations and bias.

Q16. How does proximal policy gradient (PPO) work in prompt generation?

PPO is a reinforcement learning algorithm used to optimize an LLM's generation policy against a reward signal.

Setup: the LLM is the policy $\pi_{\theta}(a|s)$. State s = prompt + tokens generated so far. Action a = next token. Reward = signal from a reward model or programmatic verifier, typically only at the end of generation.

PPO objective: max over θ of $E[\min(r_t \cdot A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) \cdot A_t)]$, where $r_t = \pi_{\theta}(a_t|s_t) / \pi_{\text{old}}(a_t|s_t)$ is the probability ratio and A_t is the advantage. The clip term prevents oversized updates that would destabilize training.

KL penalty: in RLHF, add $-\beta \cdot \text{KL}(\pi_{\theta} || \pi_{\text{SFT}})$ to keep the policy near the SFT reference. Without this, the policy drifts off-manifold and reward-hacks.

Practical notes: small learning rates (1e-6 to 5e-6). KL coefficient typically 0.01 to 0.1, tuned to balance reward chasing vs staying on-manifold. Reward normalization important for stability.

Q17. How does knowledge distillation benefit LLMs?

Knowledge distillation transfers knowledge from a large teacher model to a smaller student model. The student is trained to match the teacher's outputs (soft labels) rather than just ground-truth labels.

Why it helps: the teacher's soft probability distribution carries more information than hard labels. The student learns relative similarities ('this is more like a cat than a dog'), which improves generalization.

For LLMs: distill a 70B teacher into a 7B student that runs much faster and cheaper. The student typically loses 1-5 points on benchmarks but offers 10-50x inference speedup. Examples: DistilBERT (from BERT), TinyLLaMA, Vicuna (from ChatGPT outputs).

Distillation strategies: response-based (match output logits), feature-based (match intermediate representations), relation-based (match relationships between examples). Combinations work best.

Modern pattern: train a strong teacher with RLHF; collect its outputs on diverse prompts; fine-tune a smaller student on those outputs (synthetic SFT). Most open-source instruction-tuned models use some form of distillation from stronger models.

Q18. What is few-shot learning in LLMs?

Few-shot learning is the ability to learn a new task from a small number of examples provided at inference time, without weight updates. Examples are passed in the prompt itself, a technique called in-context learning.

Pattern: a prompt with 1-5 examples of (input, output) pairs followed by a new input. The model continues the pattern by producing the output for the new input.

Zero-shot: no examples, just an instruction. Few-shot: a handful of examples. Many-shot: dozens to hundreds, fitting in long contexts.

Mechanism: the LLM has learned the meta-task of pattern continuation during pretraining. Few-shot examples specify the task implicitly. No backprop, no weight changes.

Limits: prompt length is finite. Quality plateaus after 5-10 examples for most tasks. Sensitive to example ordering and selection. For better quality with abundant examples, use RAG (retrieve relevant examples per query) or fine-tune.

Q19. How do you evaluate LLM performance metrics?

Intrinsic: perplexity on held-out text. Useful sanity check but only weakly correlated with utility.

Generation: BLEU/ROUGE for reference-based tasks; BERTScore for semantic similarity; CIDEr for captioning; LLM-as-a-judge for open-ended.

Reasoning: MMLU (multi-domain knowledge), HellaSwag (commonsense), GSM8K, MATH (math), BIG-Bench (diverse hard tasks), ARC (science), TruthfulQA (factuality).

Code: HumanEval, MBPP, LiveCodeBench (pass@k on programming problems).

Chat: MT-Bench, Chatbot Arena (human preference at scale).

Safety: ToxiGen, BBQ (bias benchmark), AdvBench (jailbreak resistance), WMDP (dangerous capability).

RAG: Ragas (faithfulness, answer relevance, context precision, context recall), RGB (robustness against noise, negatives, integration, counterfactuals).

Production: latency, throughput, cost per query, user satisfaction signals (thumbs, regenerate rate), retention.

Q20. How would you use RLHF to train an LLM?

Step 1: SFT. Curate or buy a high-quality instruction dataset. Fine-tune the base model on (prompt, response) pairs. 1-3 epochs, LR $1e-5$ to $5e-5$.

Step 2: Collect preferences. Generate multiple responses per prompt using the SFT model. Have humans rank them pairwise. Aim for 30K to 100K preference pairs.

Step 3: Train reward model. Initialize from the SFT model or a same-family base. Train to predict $P(\text{response A} > \text{response B} \mid \text{prompt})$. Standard cross-entropy on preference pairs.

Step 4: PPO. Initialize policy from SFT. Initialize reference from SFT. For each step: sample prompts, generate responses from the policy, score with the reward model, compute advantages, take a PPO step with a KL penalty against the reference. Run for 1-5K steps.

Step 5: Evaluate. Held-out preference pairs, MT-Bench, safety benchmarks. Compare against base and SFT. Iterate on the reward model and PPO settings.

Variants: DPO (skip steps 3-4, train directly on preferences) for cost; GRPO (skip the value head) for memory; RLAIIF (AI-generated preferences) for scale.

RLHF: from pretrained base to aligned LLM



Q21. What techniques improve factual accuracy in LLM-generated text?

RAG: ground generation in retrieved evidence. The single biggest factuality improvement available.

Citations: require the model to cite sources for claims; verify citations.

Self-consistency: sample N answers; vote on the majority answer.

Chain-of-thought: explicit reasoning steps surface errors and improve correctness on multi-step tasks.

Tool use: call external tools (calculator, search, code execution) instead of relying on parametric memory for verifiable facts.

NLI verification: verify each claim against retrieved evidence with an NLI model; block or flag unsupported claims.

Fine-tuning on factual data: SFT on high-quality factual sources; preference learning that prefers correct over incorrect responses.

Lower temperature: reduces creative drift for factual tasks.

Constrained decoding: force the output to match a schema or to be drawn from a fixed set of valid options when applicable.

Q22. How do you detect drift in LLM performance over time?

Baseline benchmarks: fix a held-out evaluation set. Re-run the benchmarks on a schedule (daily, weekly). Track metrics over time. Alert on regressions.

Production sampling: sample 1-5% of production traffic. Score offline with automated metrics and periodic human review.

User feedback signals: thumbs up/down, regenerate rate, conversation length, abandonment. Trend over time. Segment by query type.

Distribution drift: monitor the input distribution. Embed queries and detect shifts in embedding space. Alert when the production distribution diverges from the evaluation distribution.

Output drift: track output statistics (length, sentiment, refusal rate). Sudden shifts often indicate a problem.

Canary deployments: shadow-deploy new model versions and compare metrics before promotion.

Causes of drift: data distribution change (users ask different questions over time), upstream service changes (a tool the model relies on changed), model version updates by the provider (silent prompt-response changes).

Q23. Strategies for curating high-quality training data for generative AI?

Quality filtering: deduplicate (exact and near-duplicate); remove low-quality sources (boilerplate, machine-translated, spam); filter for fluency.

Domain coverage: ensure the data spans the topics, tasks, and styles your model should handle.

Underrepresented topics produce weak capabilities.

Toxicity and safety filtering: remove harmful content, but preserve enough negative examples to teach the model to recognize and refuse them.

Source diversity: web alone is biased; supplement with books, scientific papers, code, structured data, multilingual sources.

Annotation quality: for SFT and preference data, hire skilled annotators, provide clear guidelines, run pilot rounds, measure inter-annotator agreement.

Data provenance and licensing: track sources and licenses. Sensitive in 2025+ with growing legal scrutiny.

Validation: hold out a clean test set that was never in the training pipeline. Decontaminate.

Iteration: data curation is iterative. Train, evaluate, identify failure modes, augment data to fix them, retrain.

Q24. Methods to identify and address biases in training data?

Demographic analysis: measure representation across protected attributes (race, gender, age, language). Quantify imbalances.

Topic and source analysis: which topics dominate? Which sources contribute most? Are minority perspectives represented?

Embedding analysis: cluster the data in embedding space; look for over- or under-represented clusters.

Bias benchmarks for evaluation: BBQ, StereoSet, CrowS-Pairs measure stereotype and bias in language models.

Mitigation: oversample underrepresented groups; downsample overrepresented ones; augment with synthetic balanced data; train with bias-mitigation objectives (adversarial debiasing, contrastive debiasing).

Process: bias review board, diverse annotation team, transparent documentation of training data choices (Datasheets for Datasets, Model Cards).

Q25. How would you fine-tune an LLM for a financial or medical domain?

Data: collect domain-specific text: clinical notes, medical literature, financial reports, regulatory filings. Get permission and ensure compliance (HIPAA, GDPR, SOC 2). Apply PII redaction.

Vocabulary: extend the tokenizer with domain-specific terms (drug names, ticker symbols, ICD codes) and briefly continue pretraining so the new tokens learn meaningful embeddings.

Continued pretraining: optional brief pass of LM objective on domain text before instruction tuning. Helps with domain vocabulary and patterns.

Instruction tuning: curate (instruction, response) pairs from domain experts. Include question answering, summarization, structured extraction, common domain tasks.

RLHF or DPO: use domain experts to rate responses for accuracy, helpfulness, safety. Higher cost but produces better-calibrated models.

RAG over domain knowledge bases for current information. Fine-tuning teaches behavior; RAG provides facts.

Evaluation: domain-specific benchmarks (MedQA, FinanceBench), expert review panels. Compliance review for regulated outputs. Continuous monitoring after deployment.

Q26. Explain the architecture of LLaMA and similar modern LLMs.

LLaMA is a decoder-only transformer with several modern refinements over the original architecture.

Pre-normalization: RMSNorm applied before each sub-layer (attention and FFN) instead of after. Stabilizes training at scale.

Rotary Position Embeddings (RoPE): apply position information by rotating the Q and K vectors. Better length extrapolation than absolute positional encoding.

SwiGLU activation in the FFN: a gated variant of Swish/SiLU. Slightly better quality than ReLU or GELU at similar compute.

Grouped-Query Attention (GQA) in LLaMA-2 70B and LLaMA-3: fewer KV heads than Q heads, reducing KV-cache size and speeding up inference.

Tied embeddings (sometimes): share input and output token embeddings to save parameters.

Training: trillions of tokens of mixed web, code, books. SFT, then DPO or RLHF for the chat variants.

Family: Mistral, Mixtral (MoE), Gemma, Qwen, Yi, Falcon all share most of these design choices. Modern decoder-only LLMs have converged on a similar template.

Chapter 8

Vector Databases

This chapter covers 10 questions on vector databases.

Q1. What are vector databases, and how do they differ from relational databases?

A vector database stores high-dimensional vector embeddings and supports similarity search: given a query vector, return the most similar stored vectors. Relational databases store structured rows and support exact lookups, joins, and range queries via SQL.

Use cases: vector DB for semantic search, recommendation, RAG, image search, duplicate detection, anomaly detection. Relational DB for transactions, reporting, structured queries.

Differences: vector DB indexes are ANN structures (HNSW, IVF, PQ), not B-trees. Vector DBs typically also store metadata for filtering. Many production stacks use both: relational for source of truth, vector for similarity search.

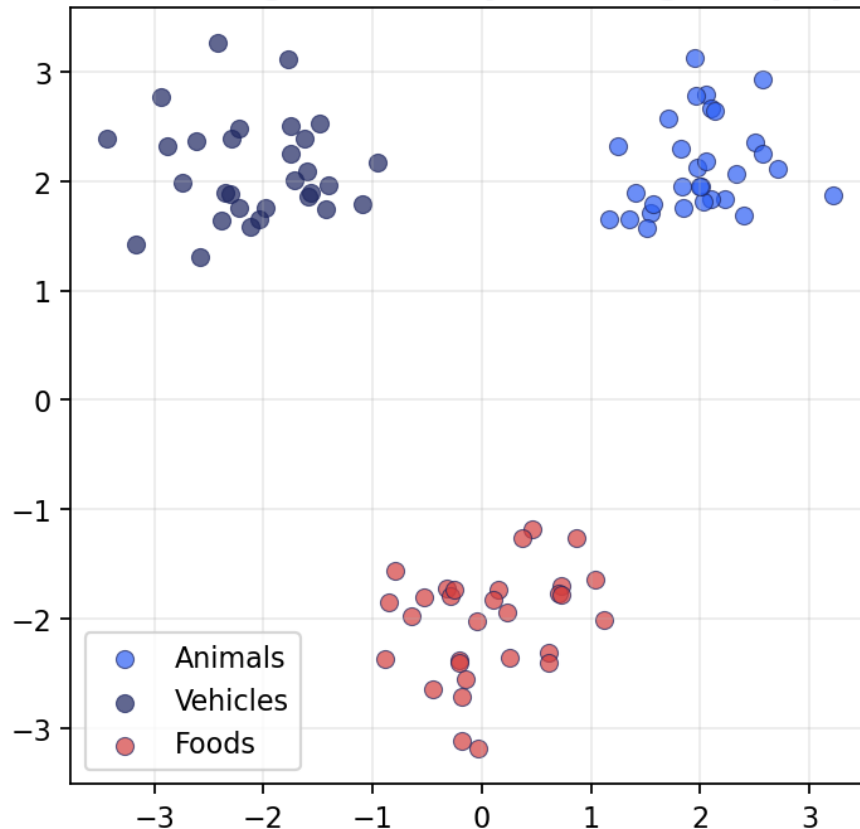
Q2. Explain how vector embeddings are generated and their role.

Generation: pass the raw input (text, image, audio) through a neural encoder. Text: a sentence transformer (BGE, GTE, E5, mpnet) or an LLM embedding endpoint (OpenAI ada, Cohere). Image: a vision encoder (CLIP, ViT). Audio: an audio encoder (Wav2Vec, Whisper encoder).

Role in vector DB: embeddings turn unstructured data into vectors that can be indexed and searched by similarity. The geometry of the embedding space encodes semantic relationships, so 'nearest in vector space' approximates 'most similar in meaning'.

Quality matters: the embedder choice determines retrieval quality. Domain-tuned embedders typically beat general-purpose ones on specialized corpora. Track Recall@k against a held-out test set to compare options.

Word embeddings cluster by meaning (2D projection)



Q3. What are the challenges of indexing and searching high-dimensional vector spaces?

Curse of dimensionality: in high dimensions, distances concentrate (max and min distance to random points become similar). Naive nearest neighbor search loses discriminating power.

Compute cost: brute-force search is $O(n \cdot d)$ per query. For 100M vectors at 768 dimensions, that is 75GB scanned per query. Impractical.

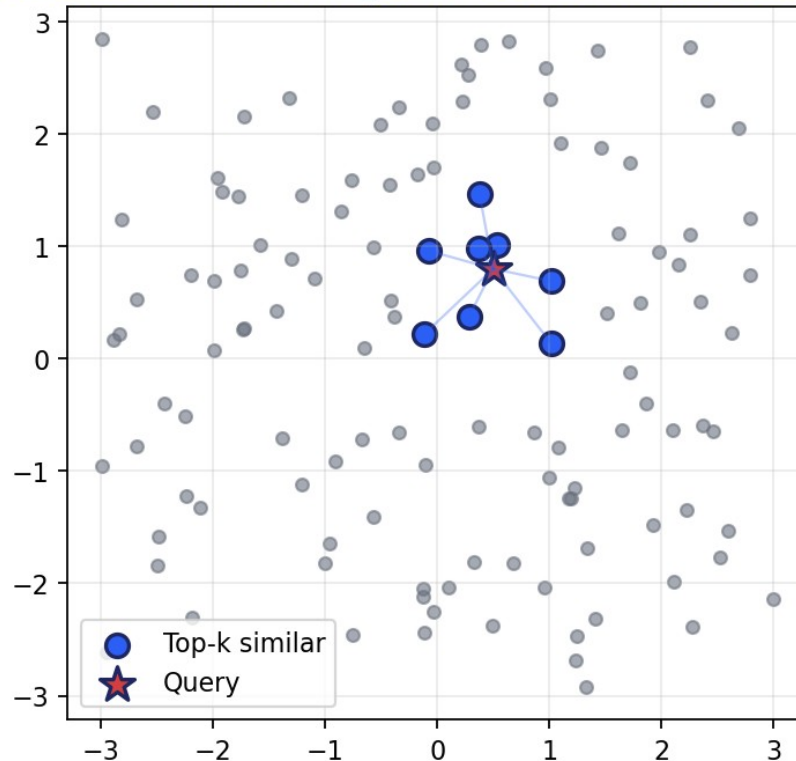
Memory: even a billion 768-dim float32 vectors is 3TB. ANN indexes need clever structures (HNSW graphs, IVF clusters, product quantization) to fit and stay fast.

Accuracy vs speed: ANN trades a small recall loss for orders of magnitude speedup. Tuning the recall-speed tradeoff is application-specific.

Updates: insertion and deletion in indexes like HNSW are not free. Some indexes require periodic rebuilds.

Hybrid queries: combining vector similarity with metadata filters (e.g., search for documents in this category only) requires careful index design.

Approximate nearest neighbor search in embedding space



Q4. How do you evaluate a vector database's performance?

Recall@k: fraction of true top-k neighbors returned. The most important quality metric.

Latency: P50 and P95 query latency. Production targets: under 50 ms P95 for interactive, under 500 ms for batch.

Throughput: queries per second under sustained load. Test at expected load and at 2-3x to find limits.

Index build time: how long to add N new vectors? Some indexes (HNSW) are slow to build; others (IVF) are faster.

Memory footprint: RAM per million vectors. Tighter is cheaper.

Update behavior: insertion latency, deletion semantics, eventual consistency.

Filter performance: hybrid queries with metadata filters; effective filter push-down vs post-filtering.

Methodology: build a held-out evaluation set with known ground-truth nearest neighbors. Test at the index sizes you'll deploy, not at toy scale.

Q5. Describe a scenario where you'd prefer a vector DB over a traditional DB.

Semantic search over enterprise documents: users ask natural-language questions that need to match meaning, not keywords. A relational DB cannot do this. A vector DB indexes document embeddings and returns relevant chunks for the query.

Recommendation: 'find products similar to this one.' Embed products; ANN search for nearest. Relational DBs handle facets and filters; vector DBs handle the similarity.

Image search: 'find images that look like this one.' CLIP embeddings indexed in a vector DB.

Duplicate or near-duplicate detection: embed everything, compute pairwise similarity at scale.

RAG: the most common modern use case. Embed knowledge base; retrieve at query time for grounding LLM responses.

Combined patterns: a hybrid system with relational metadata, full-text search (BM25), and vector similarity is the gold standard for production search.

Q6. What are some popular vector databases?

Pinecone: managed, serverless, easy to start, expensive at scale.

Weaviate: open-source and managed, GraphQL API, hybrid search.

Qdrant: open-source, Rust-based, fast, strong metadata filtering.

Milvus: open-source, mature, supports many index types and scales to billions.

Chroma: lightweight, embeddable, good for prototyping.

FAISS (library, not a DB): Facebook's library for ANN search. Powers many custom solutions and is embedded in tools like LangChain.

pgvector: PostgreSQL extension for vector search. Good when you already use Postgres and don't need extreme scale.

Elasticsearch and OpenSearch: added vector search to their full-text engines. Best when you want hybrid sparse+dense out of the box.

Vespa, MyScale, LanceDB, Marqo: also strong contenders depending on use case.

Selection criteria: scale, latency targets, hybrid search needs, hosting preference, ecosystem maturity, cost.

Q7. How do vector databases support ML workflows?

Feature store role: store entity embeddings (user, item, document) for downstream models to consume. Embeddings become reusable features across tasks.

Real-time inference: a recommendation service embeds the user query/context, calls the vector DB for nearest items, returns recommendations. Sub-100ms round-trip.

Model serving augmentation: in a RAG system, the vector DB is the retrieval layer between user query and LLM.

Training data sampling: nearest-neighbor queries find similar examples for few-shot prompting, active learning, or contrastive training.

Model evaluation: embed eval set; find similar training examples; check for leakage and contamination.

Deduplication: at ingestion time, find near-duplicates and dedupe before training.

Monitoring: embed production traffic; track drift in embedding space; detect anomalies.

Q8. What techniques ensure vector DB scalability?

Sharding: split the index across nodes by vector ID or hash. Each query fans out to all shards, aggregates results. Linear scaling on read.

Replication: multiple replicas per shard for read throughput and fault tolerance.

Hierarchical indexes (HNSW): naturally scale to billions of vectors with good query latency.

IVF + PQ: cluster vectors into IVF buckets; quantize within each bucket with PQ. Trades memory for speed and is the standard for very large indexes.

Tiered storage: hot data in RAM, warm in SSD, cold in object storage. Acceptable if query patterns are skewed.

Incremental indexing: avoid full rebuilds. Pinecone, Qdrant, and Milvus support online updates.

Quantization: int8 or binary embeddings shrink memory 4-32x. Use for first-stage retrieval with re-ranking in higher precision.

Caching: semantic cache for repeated queries; embedding cache for repeated documents.

Q9. How do you handle vectors of different dimensionalities?

Normalize first: most vector DBs assume vectors have a fixed dimension per collection. Embed everything with the same model or with models that produce the same dimension.

Matryoshka embeddings: a single embedder trained to produce useful representations at multiple truncation lengths. Lets you use 64-dim for cheap prune and 768-dim for fine scoring.

Projection: project vectors of different dimensions into a common space with a learned linear projection. Useful when you have multi-modal embedders that produce different dimensions.

Separate indexes per dimension: maintain one index per embedder. Query the right one based on the input type. Combine results post-hoc.

Dimensionality reduction: PCA or random projection to reduce all vectors to a common smaller dimension. Loses some recall.

Q10. What role does vector similarity play in recommendation and NLP?

Recommendation: embed users and items in the same space (collaborative filtering or two-tower models). Similarity = predicted affinity. Nearest items to a user's embedding are the recommendations. Scales to billions of items with ANN.

NLP: embedding similarity is the core operation in semantic search, retrieval-augmented generation, clustering, classification (via prototypes), and similarity-based reasoning.

Similarity functions: cosine similarity is the standard. Dot product is equivalent for L2-normalized vectors. Euclidean distance is sometimes used but behaves worse in high dimensions.

Production patterns: retrieval at a coarse similarity threshold, followed by fine reranking with a stronger model. Hybrid combining vector similarity with lexical (BM25) and structured (metadata filter) constraints.

Limitations: similarity does not imply relevance for all tasks. A semantically similar passage may not contain the answer. Reranking and grounding steps close this gap.

Chapter 9

LLMOps and System Design

This chapter covers 11 questions on llmops and system design.

Q1. Design a system that uses an LLM for near real-time response at massive scale.

Targets: P95 latency under 2 seconds; throughput 10K+ QPS; cost manageable; graceful degradation under load.

Architecture: a thin API gateway in front; request routing by user tenant; LLM serving layer using vLLM or TGI behind a load balancer; semantic cache to short-circuit repeated queries; KV-cache management at the serving layer; horizontal scaling with autoscaling on queue depth.

Optimization: continuous batching (vLLM's PagedAttention) raises throughput 5-24x over naive batching. Speculative decoding for lower latency on certain models. KV-cache reuse for shared prefixes (system prompts, multi-turn conversations). Use the smallest model that meets quality. Quantize aggressively (int8 or int4).

Load balancing: queue-aware routing prefers replicas with shorter queues. Sticky sessions when possible for cache locality.

Observability: distributed tracing across gateway, cache, retrieval, generation. Track P50/P95/P99 latency, tokens per second, cache hit rate, error rate. Alert on regressions.

Q2. How would you incorporate caching to improve LLM performance?

Exact-match cache: hash the input (prompt + parameters); cache the response. Cheap, simple. Hit rate depends on traffic patterns. Useful when the same queries repeat (FAQs, common tasks).

Semantic cache: embed the query; look up similar past queries; return their cached response if similarity exceeds a threshold. Captures more hits at the risk of returning a not-quite-right answer. Tune the threshold against a labeled validation set.

Prompt cache: cache the KV-cache for shared prefixes (system prompts, RAG context). Anthropic, OpenAI, and many serving stacks support this. Saves both latency and cost.

Embedding cache: cache embeddings for repeated documents or queries. Cheap to implement; high hit rate on stable corpora.

Tool-call cache: cache tool responses by argument (deterministic tools only). Cache-and-skip pattern reduces redundant tool calls in agentic loops.

What to cache: stable, repeated, idempotent content. What NOT to cache: personalized responses, time-sensitive data, sensitive content with retention concerns.

Q3. How do you reduce model size for deployment on resource-constrained devices?

Quantization: int8 (4x smaller, modest accuracy loss); int4 (8x smaller, larger but still acceptable loss with good methods like AWQ, GPTQ); int2 and binary for extreme cases.

Pruning: remove weights or neurons that contribute little. Unstructured pruning is easier; structured pruning (entire heads or channels) gives real speedups.

Knowledge distillation: train a smaller student from a larger teacher. Common size ratios: 10x smaller students keep 90-95% of teacher quality.

Architecture search: pick smaller models designed for efficiency (MobileBERT, DistilBERT, Mistral-7B for LLMs, MobileViT for vision).

Model partitioning: split the model between device and edge server. Run preprocessing and small layers on-device; larger layers on the server.

Mixed precision inference: BF16 or FP8 instead of FP32. Free 2-4x speedup on modern hardware.

Compilation: ONNX Runtime, TensorRT, CoreML, TFLite optimize for specific hardware. Use them.

Q4. Discuss tradeoffs of GPUs vs TPUs vs specialized hardware for LLM deployment.

GPUs (NVIDIA H100, H200, B200, RTX series): mature ecosystem, broad framework support, easy to acquire (though expensive). The default for most LLM workloads. Strong performance per dollar for inference and training.

TPUs (Google v4, v5, v6): excellent performance for transformer workloads, tight integration with JAX and TensorFlow, but Google Cloud only. Best for very large training runs where pod size matters.

AMD GPUs (MI300X): catching up; competitive for inference on big models due to large HBM. Software ecosystem (ROCm) maturing but still less polished than CUDA.

AWS Trainium and Inferentia: AWS-specific accelerators with cost advantages for inference on supported models. Smaller ecosystem.

Cerebras, Groq, SambaNova, Etched: specialized inference accelerators. Very high throughput on specific architectures. Lock-in risk and smaller ecosystem.

Apple Silicon: increasingly capable for local inference (M2 Ultra, M3 Max). MLX framework gaining traction.

Selection drivers: workload (training vs inference), scale, framework support, cost, vendor lock-in, ecosystem maturity. For most enterprises, NVIDIA GPUs on a major cloud are the default.

Q5. How would you build a ChatGPT-like system?

Frontend: web app with streaming responses, conversation history, file upload. Mobile apps. APIs for developers.

Backend: stateless API layer. Session management with persistent conversation store. Rate limiting per user.

Model serving: vLLM or TGI behind a load balancer. Continuous batching. KV-cache management. Speculative decoding for latency.

Models: a frontier base LLM trained with SFT + RLHF. Optionally specialized variants (code, reasoning, multimodal).

Tool use: function calling for web search, code execution, image generation, calendar, email. Tool registry; safe execution sandboxes.

RAG: optional, for memory and document upload features.

Safety: input filters (prompt injection, harmful intent), output filters (toxicity, PII), refusal calibration, jailbreak detection. Continuous red teaming.

Operations: observability (traces, metrics, logs), feature flags, A/B testing, incident response. Compliance: SOC 2, GDPR.

Scale: tens of millions of users requires global multi-region deployment, specialized hardware, careful capacity planning.

Q6. Design an LLM-based code generation system. What challenges arise?

Base model: a code-specialized LLM (StarCoder, Code Llama, DeepSeek Coder, Claude). Trained on code corpora plus natural language.

Context: code generation needs the right context, not just the current line. Include: file imports, related files (cross-file context), open editor tabs, documentation. Embed and retrieve relevant snippets from the user's repository.

Inference: low latency is critical for inline completion. Use small models for auto-suggest; larger models for chat-style code generation. Speculative decoding is well-suited to code.

Quality: verify generated code parses and (ideally) compiles before showing. Run unit tests when available. Lint for style issues.

Safety: detect attempts to generate malicious code (malware, exploits). Detect license violations (verbatim copy of GPL code).

Personalization: learn the user's style; respect project conventions. Per-user and per-project context.

Challenges: hallucinated APIs (functions that don't exist), wrong types, outdated patterns, security vulnerabilities. Mitigations: type checking, API verification, security linters, secret detection.

Q7. Describe an approach to generative AI for original music composition.

Pipeline: text prompt → music model → audio output. Models like MusicGen, MusicLM, and Stable Audio take text descriptions and produce audio.

Architecture: transformer or diffusion model trained on (audio, description) pairs. Audio is typically tokenized via a neural audio codec (Encodec, SoundStream) into discrete tokens that an autoregressive transformer can predict.

Conditioning: text describes genre, instruments, mood, tempo. Some models accept melody conditioning (hum a tune; the model orchestrates around it).

Quality considerations: musical coherence over long durations (most models struggle past 30-60 seconds), structural elements (verse, chorus), and instrument fidelity.

Practical patterns: hybrid systems combine an LLM (for high-level structure and lyrics) with a music model (for audio). Human-in-the-loop for refinement.

Ethical: training data licensing, artist attribution, copyright of generated music. Detection of generated content. Pay-for-training models compensate rights holders.

Q8. How would you build an LLM-based QA system for a specific domain or dataset?

Architecture: RAG is the default. Domain corpus → embeddings → vector DB. User query → retrieval → grounded LLM response with citations.

Embedder: domain-tuned if vocabulary diverges from general. Start with BGE or E5 baselines; fine-tune on in-domain (query, relevant-doc) pairs.

Chunking: pick based on document structure. Sections for structured documents, semantic chunks for prose, semantic + hierarchical (mother-child) for high-stakes.

Retrieval: hybrid (BM25 + dense) almost always beats either alone. Reranker on top of dense retrieval for the final stage.

LLM: a strong general LLM is usually enough with good retrieval. Domain fine-tuning helps with format and tone but is rarely necessary for accuracy.

Evaluation: build a held-out QA set with expert-annotated answers. Measure retrieval (Recall@k) and generation (faithfulness, answer relevance via Ragas, exact-match on closed-form answers).

Continuous improvement: log all queries; review failures; expand the corpus or the prompt; retune embedders. The eval set should grow with discovered failure modes.

Q9. What design considerations matter for multi-turn conversational AI with an LLM?

State management: persistent conversation history; conversation summaries for long sessions; identity (which user is this, what do they have access to).

Query understanding: rewrite follow-up queries to standalone form using the history. 'When was it founded?' → 'When was Acme Corp founded?'

Memory: short-term (recent verbatim turns) plus long-term (summarized history or facts about the user). Long-term memory is itself a RAG index.

Tool use: when the user asks for something requiring a tool (search, calculation, file upload), the model decides to call the tool; the result feeds the next response.

Safety: per-turn input and output guardrails. Multi-turn jailbreak detection (some attacks unfold over multiple turns).

Personalization: respect user preferences, prior context, communication style. Privacy considerations for what is stored.

Recovery: handle errors gracefully (tool failure, LLM timeout). Streaming responses with intermediate state.

Evaluation: multi-turn benchmarks (MT-Bench), user satisfaction metrics (thumbs, regenerate rate, session length, return rate).

Q10. How can you control creative output of generative models for specific styles?

Prompting: detailed prompts specifying style, medium, mood, references. Few-shot examples of the desired style in the prompt.

Conditioning: for image models, ControlNet (pose, depth, canny edge) conditions on structural inputs. For LLMs, system prompts define persona and style. IP-adapter passes a style image as condition.

Fine-tuning: LoRA on a small dataset of the target style. Dreambooth for subject-specific image generation. SFT on (instruction, style-matched response) pairs for text.

Sampling controls: temperature, top-k, top-p, repetition penalty. Lower values for tighter style adherence; higher values for more creative variation.

Classifier-free guidance: amplifies the prompt's influence. Higher CFG scale = more on-prompt but potentially less natural.

Negative prompts: specify what to avoid. 'Photo, photographic' as a negative for stylized illustration.

Evaluation: human raters on style match. Reference-based metrics where applicable (CLIP-score against style reference, FID against style corpus).

Q11. How do you monitor LLM systems once productionized?

Per-request: latency (P50, P95, P99), tokens generated, time-to-first-token, cost. Tool calls, retrieval hits.

Aggregate: throughput, error rate, queue depth, cache hit rate, GPU utilization.

Quality: sample 1-5% of traffic for offline scoring (faithfulness, relevance, safety). Build dashboards trending quality over time. Alert on regression.

User signals: thumbs up/down ratio, regenerate rate, conversation length, abandonment, return user rate.

Drift: input distribution drift (embed queries, monitor distribution); output distribution drift (length, refusal rate, sentiment).

Safety: jailbreak attempts (input filter triggers), policy violations, PII exposure events. Audit logs for all retrievals and tool calls.

Cost: dollar cost per user, per query, per session. Trend over time. Alert on spikes that may indicate abuse or runaway agents.

Tools: OpenTelemetry for tracing. Grafana, Datadog, Honeycomb for dashboards. Helicone, LangFuse, Arize for LLM-specific observability.

Chapter 10

Evaluation Methods

This chapter covers 22 questions on evaluation methods.

Q1. What are common NLP evaluation metrics, and how do you choose?

Classification: accuracy, precision, recall, F1, ROC-AUC, PR-AUC. Choose based on class balance and cost of errors. F1 for imbalanced; ROC-AUC for ranking.

Sequence labeling (NER, POS): token-level F1 plus span-level F1 (exact match of predicted spans against gold).

Translation: BLEU (n-gram overlap), chrF (character F-score), COMET (learned metric, often better correlated with human judgment).

Summarization: ROUGE-1/2/L (recall-oriented n-gram overlap). BERTScore for semantic similarity. Human evaluation for coherence and faithfulness.

Generation: BLEU, ROUGE, METEOR, CIDEr (captioning), BERTScore. LLM-as-judge for open-ended generation.

Retrieval: Recall@k, Precision@k, MRR, MAP, NDCG. Choose based on whether you need top-1 (MRR) or coverage (Recall@k).

Selection guidance: pick the metric most aligned with your actual deployment objective. For chat, win-rate against a baseline (MT-Bench, Arena) is more meaningful than BLEU.

Q2. How does evaluation differ between generative tasks and classification tasks?

Classification has a clear correct answer; metrics like accuracy and F1 are well-defined and reliable.

Generative tasks (text, image) have many valid outputs. Reference-based metrics (BLEU, ROUGE) capture only one notion of correctness and correlate weakly with human judgment.

Generative evaluation needs:

- LLM-as-judge with explicit rubric and bias control (position, verbosity, self-preference).
- Human evaluation for high-stakes or ambiguous outputs.
- Task-specific metrics (CodeBLEU, pass@k for code; CLIPScore for image; FID for image distribution match).

- Diversity metrics (self-BLEU, distinct-n) to catch mode collapse.
- Multi-dimensional rubrics: factuality, fluency, relevance, helpfulness, safety. Compute each separately; combine with care.

Classification can be automated cheaply; generative usually cannot. Plan for human-in-the-loop evaluation for generative systems.

Q3. Why is human evaluation important in NLP, especially for generative AI?

Automated metrics miss what humans care about: factuality, helpfulness, tone, creativity, harm. A high BLEU score with low quality is common in summarization. A model that reliably hallucinates may have low perplexity.

When human eval is essential: open-ended generation (chat, summarization, creative writing); high-stakes outputs (medical, legal, financial); subjective quality dimensions (style, tone, helpfulness); model comparison at the frontier where automated metrics saturate.

Methodology: clear rubric; calibration round with examples; multiple annotators per item; inter-annotator agreement check (Cohen's kappa, Krippendorff's alpha); blind comparison (don't tell raters which model produced which output); stratified sampling to cover edge cases.

Tradeoffs: expensive, slow, hard to scale, subjective. Mitigations: LLM-as-judge for screening with human review on borderline cases; preference data collection through paid annotation services or crowdsourcing platforms.

Q4. How do you evaluate models for bias and fairness?

Benchmarks: BBQ (bias benchmark for QA), StereoSet, CrowS-Pairs, WEAT, BBNLI. These probe for stereotype and bias along axes like gender, race, religion, nationality.

Demographic parity in outcomes: when the model classifies or generates, are outcomes equal across demographic groups? Disparate Impact ratio, equal opportunity, equalized odds are formal metrics.

Counterfactual evaluation: change a demographic attribute in the input (he → she); does the output change in a biased way? Build counterfactual pairs and measure consistency.

Generative bias: in open-ended generation, measure how often the model associates roles, traits, or attributes with specific groups. Compare against balanced ground truth.

RAG bias: does retrieval pull representative documents across groups, or only majority sources?

Process: audit before deployment; continuous evaluation in production; publish model cards documenting known biases; collect user feedback on bias incidents.

Q5. What is perplexity, and why is it used to evaluate language models?

Perplexity = $\exp(\text{average negative log-likelihood per token})$ on a held-out corpus. Equivalently, the geometric mean of $1/P(\text{token} | \text{context})$. Lower is better.

Intuition: perplexity measures how 'surprised' the model is by the corpus. Perplexity of 10 means the model is roughly as uncertain as if it were choosing uniformly from 10 options for each token.

Why used: directly tied to the training objective (causal LM). Easy to compute. Comparable across models trained on the same tokenizer and similar evaluation corpus.

Caveats: perplexity depends on the tokenizer (a model with a smaller vocab naturally has higher perplexity). Perplexity on Reddit text says nothing about math reasoning. Below a usable threshold, perplexity decouples from task performance: a model with PPL=5 and PPL=4 are both 'fluent' but differ negligibly in chat quality.

Use perplexity as a sanity check during training and for comparing models within the same family. For downstream quality, use task benchmarks.

Q6. How do you evaluate coherence and relevance of generated text?

Reference-based: BLEU, ROUGE, METEOR, BERTScore against a reference. Capture surface overlap and semantic similarity but not coherence directly.

LLM-as-judge: prompt a separate strong LLM to rate coherence and relevance on a Likert scale or as pairwise preference. Cheap and scalable; control for biases (position, verbosity, self-preference).

Human evaluation: panel ratings on a defined rubric. Especially for chat and summarization where coherence over multiple sentences matters.

Structural metrics: entity coverage in summaries (does the summary include the key entities from the source?); claim alignment (does each claim in the output appear in the source?); topical consistency (do successive sentences stay on topic?).

Engagement signals in production: average response length read by users, regenerate rate, follow-up rate. Indirect but informative.

Q7. Discuss BLEU, METEOR, and human eval for coherence in conversational AI.

BLEU: counts n-gram overlap (1- to 4-grams) between candidate and reference, applies a brevity penalty. Originally designed for machine translation. Strengths: simple, fast, well-understood. Weaknesses: rewards surface overlap; misses paraphrase; one reference is rarely enough.

METEOR: aligns words between candidate and reference using exact, stem, synonym, and paraphrase matches. Higher correlation with human judgment than BLEU. Slower; language-specific resources required.

Human eval: the only reliable metric for open-ended conversational AI. Rate on rubrics covering coherence, helpfulness, factuality, safety. Use pairwise preference (A vs B) when comparing models, since absolute ratings are noisier.

For chat specifically: MT-Bench (a curated set of multi-turn prompts judged by GPT-4) and Chatbot Arena (head-to-head human preference at scale) are the modern standards.

Use BLEU/METEOR as cheap sanity checks; trust human eval for real comparisons.

Q8. How do you assess diversity in generated text?

Self-BLEU: BLEU between pairs of samples from the same model. Higher self-BLEU = more repetitive across samples. Low self-BLEU is good.

Distinct-n: number of distinct n-grams divided by total n-grams across samples. Distinct-1 measures unique unigrams; distinct-2 unique bigrams. Higher = more diverse.

Embedding diversity: embed samples; compute average pairwise distance in embedding space.

Topic coverage: cluster samples by topic; count distinct clusters. Useful for open-ended generation prompts.

Mode collapse detection: a model that always produces similar outputs to different prompts is mode-collapsed. Detect by sampling many outputs and checking for redundancy.

Calibration: sample at multiple temperatures and observe the diversity curve. Healthy models produce more diverse outputs at higher temperature; collapsed models produce similar outputs regardless.

Q9. What role does prompt engineering play in evaluation?

Prompt engineering affects model behavior dramatically. A model that scores poorly under one prompt may score well under another. Fair evaluation must control for this.

For research: report results for both a default prompt and an optimized prompt. Best-of-N prompt sampling sometimes used.

For benchmarks: use the canonical prompt provided by the benchmark. Don't tune the prompt on the test set.

For LLM-as-judge: the judge's rating depends heavily on the rating prompt. Standardize rubrics, randomize candidate order, control for verbosity bias.

For production: prompt engineering is part of the system; evaluate the whole system, not the model in isolation. Track which prompts perform best.

Q10. What are ROUGE scores, and why are they used for summarization?

ROUGE = Recall-Oriented Understudy for Gisting Evaluation. A family of metrics for summarization that count overlap between candidate summary and reference summary.

ROUGE-N: counts n-gram overlap. ROUGE-1 (unigram), ROUGE-2 (bigram) are most common. Recall, precision, and F-measure variants exist.

ROUGE-L: longest common subsequence between candidate and reference. Captures word order without requiring contiguous matches.

ROUGE-W, ROUGE-S, ROUGE-SU: weighted LCS, skip-bigram, and union of skip-bigram and unigram. Less commonly used.

Why used: established baseline, easy to compute, correlates moderately with human judgment for summarization. Most summarization papers report ROUGE.

Limits: rewards surface overlap; misses semantic equivalence (paraphrase); one reference often not enough; can be gamed by extractive systems that copy sentences from the source.

Q11. How do you assess informativeness and conciseness of a summarization model?

Informativeness: does the summary capture the key information? Measure entity coverage (% of source entities included), key fact coverage (% of pre-identified key facts present), ROUGE recall (rewards including reference content).

Conciseness: length penalty (shorter is better if quality is matched); compression ratio (summary length / source length); redundancy (self-BLEU or redundant n-grams within the summary).

Faithfulness: are all claims in the summary supported by the source? FactScore, QAGS (QA-based eval), NLI-based grounding checks.

Human evaluation rubric: informativeness, conciseness, faithfulness, fluency. Rate each on a 5-point scale; compute mean per dimension.

Tradeoff: longer summaries score higher on informativeness but lower on conciseness. Set a length budget appropriate to the use case.

Q12. How do you evaluate retrieval quality in RAG?

Build an evaluation set with (query, relevant-document) pairs. Annotated by experts or curated from production logs with implicit feedback.

Recall@k: fraction of relevant documents in the top-k retrieved. The most important RAG retrieval metric; if relevant docs are not retrieved, the LLM cannot answer.

Precision@k: fraction of top-k that are relevant. Less important than recall for RAG because the LLM filters irrelevant context, but a low precision still wastes context tokens.

MRR: mean reciprocal rank of the first relevant document. Sensitive to ranking quality of the top result.

NDCG: graded relevance with rank discount. Useful when relevance is graded (highly relevant vs somewhat relevant).

End-to-end signals: even with good retrieval metrics, downstream answer quality is what matters. Measure faithfulness and answer relevance via Ragas.

Why it matters: retrieval quality usually dominates RAG outcomes. Invest in evaluation here before tuning the generator.

Q13. What strategies reduce hallucination in RAG?

Better retrieval: stronger embedder, hybrid sparse+dense, reranker, more comprehensive corpus. The biggest lever.

Grounding prompt: instruct the model to answer only from provided context and to refuse if the context does not support an answer.

Citation requirement: model must cite which retrieved passage supports each claim. Verify citations.

NLI verification: post-process by running each claim through an NLI model against the retrieved context. Block contradictions; flag neutral verdicts.

Self-consistency: sample multiple responses; pick the consistent one.

Fine-tuning: train the model on grounded responses with explicit refusals when context is insufficient.

Confidence calibration: model output includes a confidence estimate; route low-confidence outputs to a human or to a refusal.

Monitoring: sample production outputs; verify against sources; trend hallucination rate.

Q14. How do you know if fine-tuning improved a model on a specific task?

Define the task and the metric before starting. Don't move the goalposts after training.

Baseline: measure the base model's performance on a held-out test set. This is the bar to beat.

Fine-tune: train. Track training and validation loss.

Evaluate fine-tuned model on the same held-out test set with the same metric. Compare to baseline.

Compute confidence intervals via bootstrap to avoid celebrating noise.

Regression suite: measure base capabilities (general reasoning, math, code, instruction following) to detect catastrophic forgetting.

Qualitative review: read a sample of outputs from both models on representative queries. Note systematic differences.

A/B test in production: if both pass offline checks, route a fraction of traffic to the new model; compare user signals.

Decision rule: accept the fine-tuned model only if it wins on the target task metric without significant regression on base capabilities.

Q15. What challenges arise when fine-tuning large LLMs, and how do you mitigate?

Overfitting: the fine-tuning set is small relative to pretraining. Mitigations: PEFT (LoRA/qLoRA), weight decay, early stopping, fewer epochs (1-3), data augmentation.

Catastrophic forgetting: aggressive fine-tuning degrades base capabilities. Mitigations: PEFT keeps base frozen; mix general data with task data; lower learning rate; layer-wise LR decay.

Compute and memory: 70B+ models require multi-GPU training. Mitigations: qLoRA, gradient checkpointing, FSDP/ZeRO, mixed precision.

Data quality: noisy or biased training data produces bad models. Mitigations: deduplicate, filter, quality-rank examples, manual review of samples.

Hyperparameter sensitivity: small changes in LR or batch size can swing results. Mitigations: sweep small set; use known-good defaults; reproducible seeds.

Evaluation cost: thorough evaluation across benchmarks is expensive. Mitigations: a focused eval suite tuned to the task plus a sample of broad benchmarks for regression detection.

Deployment: fine-tuned models need to be served alongside or instead of base. Mitigations: LoRA adapters for multi-tenant serving; canary deployments.

Q16. How do you assess the quality of generated samples from a generative model?

Image generation: FID (Frechet Inception Distance) compares the distribution of generated images to real ones. CLIPScore measures text-image alignment. Inception Score measures diversity and recognizability. Human evaluation for aesthetics.

Text generation: BLEU/ROUGE for reference-based; BERTScore for semantic; LLM-as-judge for open-ended; human evaluation for high stakes. Perplexity as a sanity check.

Code: pass@k (fraction of problems solved within k samples). Compile rate, lint score.

Audio: human MOS (Mean Opinion Score) for speech quality; FAD (Frechet Audio Distance) for music.

General principles: combine reference-based, distribution-level, and human metrics. Report variance, not just mean. Stratify by query type. Pre-register the evaluation protocol to avoid p-hacking.

Diversity: distinct-n, self-BLEU, distinct topic count. Avoid mode collapse.

Faithfulness for grounded generation: NLI-based, FactScore, citation accuracy.

Q17. How would you set up an A/B test for two NLP models?

Hypothesis: state precisely what you expect (e.g. 'Model B reduces customer-support response time by 5%'). Pre-register the hypothesis.

Sample size: power analysis to determine N. Effect size estimate $\times$ variance estimate $\rightarrow$ required users per arm. Avoid running too small a test.

Randomization: assign users (not sessions) to arms randomly. Stable assignment so the same user sees the same model.

Control and treatment: control = current model; treatment = new model. Match everything else (prompt, latency budget, etc.) between arms.

Metrics: primary (the hypothesis metric, e.g. resolution time); secondary (satisfaction, regenerate rate, escalation rate); guardrail (safety violations, latency, cost). Don't move the primary mid-test.

Duration: long enough for weekly cycles. Typically 2-4 weeks.

Analysis: t-test or proportions test with Bonferroni correction for multiple metrics. Check for distribution shift between arms (sanity).

Decision: pre-defined launch criteria. If treatment wins primary by X% with $p < 0.05$ and no guardrail violations, launch.

Q18. How do latency and efficiency factor into evaluating NLP models in production?

Latency budgets: define P50, P95, P99 targets. P95 is what users feel. Interactive chat typically targets P95 under 2 seconds; semantic search under 200 ms.

Throughput: requests per second sustained. Determine capacity required and cost.

Time-to-first-token: critical for streaming UX. Users tolerate longer total latency if the first token appears quickly.

Cost per query: dominated by input tokens for RAG. Optimize prompt size, use prompt caching, downsize models when quality allows.

Quality-efficiency tradeoff: a slightly lower-quality smaller model that ships at lower latency and cost often beats a higher-quality slow model in production. Measure end-to-end user outcomes, not just quality benchmarks.

Efficiency improvements: quantization, distillation, speculative decoding, KV-cache reuse, batched serving. Each affects quality differently.

Q19. What is the role of explainability in NLP evaluation for high-stakes applications?

High-stakes domains (medical, legal, financial, hiring) need explanations of model outputs for trust, debugging, and compliance.

Citation-based explanation: for RAG, surface the retrieved passages that supported each claim. Lets users verify and learn.

Attention-based explanation: visualize attention weights to show which input tokens drove the output. Interpretable but not always faithful (attention patterns can be misleading).

Chain-of-thought: have the model surface its reasoning. Catches mistakes earlier and is auditable.

Counterfactual explanations: show how the prediction would change with different inputs. Useful for fairness audits.

Influence functions: identify which training examples most influenced a specific prediction. Expensive but powerful for debugging.

Documentation: model cards, datasheets for training data, system cards describing the full pipeline.

Evaluation: explanation faithfulness (does the explanation match what drove the prediction?) and explanation utility (do users actually understand and trust the output better?).

Q20. How do you measure user satisfaction with a deployed NLP model?

Direct: thumbs up/down on each response. Survey at session end (CSAT, NPS).

Indirect: regenerate rate (users dissatisfied with first response), conversation length (longer $\neq$ better, depends on use case), abandonment rate, time to resolution.

Behavioral: return user rate, frequency of use, breadth of features used.

Aggregate: weekly/monthly active users, retention curves. Segment by user cohort and query type.

Qualitative: structured user interviews. Open-ended feedback collection. Support ticket analysis for pain points.

Triangulate: no single metric is sufficient. A high thumbs-up rate with high regenerate rate suggests users are satisfied with what they finally got but had to work for it.

Distinguish satisfaction from sycophancy: a model that flatters users may get high ratings without being helpful. Cross-check with task completion metrics.

Q21. What is domain adaptation, and how do you evaluate it?

Domain adaptation is the process of taking a model trained on a general distribution and adapting it to a specific domain (medical, legal, financial). Mechanisms: continued pretraining on domain text, instruction tuning on domain (prompt, response) pairs, PEFT for behavior changes, domain-specific RAG.

Evaluation: build a held-out domain test set with expert-annotated ground truth. Measure task metrics on this set. Compare to: (a) the base model (does adaptation help?), (b) a base model with RAG (is the gain worth the fine-tuning cost?), (c) a model adapted with more or different data (is your data sufficient?).

Regression suite: measure general capabilities (MMLU, HumanEval, GSM8K) to catch catastrophic forgetting.

Domain-specific benchmarks: MedQA, FinanceBench, LegalBench. Use them for comparable numbers.

Expert review: in regulated domains, human expert review of model outputs is the gold standard. Sample stratified across query types.

Q22. How do you evaluate robustness to adversarial attacks?

Adversarial benchmarks: AdvBench (jailbreak prompts), Anthropic's red-teaming datasets, HarmBench. Measure refusal rate and harm rate on adversarial inputs.

Prompt injection: test with retrieved content containing injection attempts. Measure how often the model follows injected instructions vs the system prompt.

Jailbreaks: structured probes (role-play, hypothetical scenarios, language tricks, prompt encoding). Track jailbreak success rate over a fixed test suite.

Input perturbation: typos, paraphrasing, capitalization. Measure consistency of outputs.

Continuous red teaming: dedicated team probing the model for novel failure modes. Add new attacks to the test suite as they are discovered.

Production monitoring: log and alert on attempted attacks. Trend successful vs blocked over time.

Defense layers: input filters, output filters, RLHF safety training, constitutional principles. Evaluate the full system, not just the model.

Chapter 11

Miscellaneous and Advanced Topics

This chapter covers 21 questions on miscellaneous and advanced topics.

Q1. What ethical considerations are crucial when deploying generative models?

Truthfulness: generative models hallucinate. Deploy with grounding (RAG), citations, and refusal-when-unsure behavior. Disclose AI-generated content.

Harm prevention: refuse to help with illegal, harmful, or dangerous requests (weapons, malware, self-harm). Continuous red teaming; safety RLHF.

Bias and fairness: audit for representational harm across demographics. Apply balanced training data, fairness benchmarks (BBQ, BBNLI), continuous evaluation.

Privacy: do not memorize and reproduce training data; protect PII; respect data subjects' rights (GDPR, CCPA).

Copyright and attribution: training data licensing; provenance; respect for rights holders.

Consent: humans should know when they are interacting with AI. Disclose model limitations.

Job displacement: think about downstream effects on workers. Engage with affected communities.

Misuse: deepfakes, disinformation, fraud. Build detection tools; cooperate with platforms.

Environmental: training and inference have energy costs. Optimize for efficiency.

Governance: model cards, system cards, ethics review boards, post-deployment audits.

Q2. Describe a challenging project involving generative models.

An interview answer should be a real story, structured as: situation, task, action, result.

Example template: 'I led the build of [SYSTEM] for [DOMAIN]. The hardest part was [SPECIFIC CHALLENGE: data quality, hallucination, latency, regulatory review]. I tried [APPROACH 1] which failed because [REASON]. I then [APPROACH 2] which worked because [REASON]. We measured [METRIC] and improved from X to Y. What I would do differently: [HONEST REFLECTION].'

Strong answers show: ownership (you made decisions, not just executed), technical depth (specific tools, metrics, tradeoffs), pragmatism (you compromised when needed), self-awareness (you can critique your own choices).

Weak answers: vague ('I worked on AI'), credit-claiming without specifics, no metrics, no failure mode acknowledged, blames others for problems.

Q3. Explain the concept of latent space in generative models.

Latent space is a learned compressed representation of the data. Each point in latent space corresponds to a possible output; generation samples a point and decodes it.

VAEs explicitly learn a latent space with a prior (typically standard normal) and a posterior conditioned on the input. The latent variable z follows $q(z|x)$, and the decoder maps z to x .

GANs implicitly define a latent space: noise vectors z fed into the generator produce data samples. No explicit prior structure, but interpolation between z vectors produces smooth interpolation in output.

Diffusion models in their latent variant (Stable Diffusion) operate in the latent space of a separately-trained VAE. The diffusion process runs in this compressed space, then a decoder produces pixels.

Use cases: interpolation between latents produces smooth transitions; arithmetic in latent space encodes attribute manipulation ('add smile to face'); semantic editing via inversion (encode image, edit latent, decode); anomaly detection (distance from typical latents).

Q4. Have you implemented conditional generative models? What conditioning techniques?

Conditional generative models accept a condition c and produce samples from $p(x | c)$. The condition can be a class label, text description, image, layout, or other structured input.

Conditioning techniques:

- Concatenation: append the condition embedding to the input or hidden states. Simple, works for class-conditional GANs.
- Cross-attention: the generator cross-attends to encoded conditions. Used in text-to-image diffusion (Stable Diffusion attends to text encoder output).
- Adaptive normalization (AdaIN, AdaLN): the condition modulates the normalization parameters of generator layers. Used in StyleGAN.
- ControlNet: an auxiliary network that takes structural conditions (pose, edge map, depth) and modulates the main diffusion model.

- Classifier-free guidance: train with and without the condition; at inference, extrapolate in the direction of the conditional prediction. Boosts conditioning fidelity.
- LoRA-style: train small adapters per condition style. Used in custom-style image generators.

Q5. Discuss the tradeoffs between GANs and VAEs.

GANs: adversarial training of generator vs discriminator. Strengths: high-quality, sharp samples.

Weaknesses: training instability (mode collapse, non-convergence), no explicit likelihood, hard to evaluate.

VAEs: probabilistic encoder-decoder with variational inference. Strengths: stable training, principled latent space, explicit likelihood (lower bound). Weaknesses: blurry samples (compared to GANs and diffusion) because of pixel-wise MSE in the reconstruction loss.

Diffusion models have largely won for image generation: training stability of VAEs, sample quality matching or exceeding GANs, controllable via conditioning. VAEs remain useful as the encoder/decoder in latent diffusion. GANs persist for narrow high-throughput use cases (StyleGAN for fast face generation).

Choose: diffusion for state-of-the-art quality and controllability; VAE for explicit latent structure and density estimation; GAN for legacy and high-throughput inference where quality is acceptable.

Q6. What are the primary differences between Hugging Face Transformers, Datasets, and Tokenizers?

transformers: model code and pretrained checkpoints for thousands of models. AutoModel, AutoTokenizer, pipelines, training APIs (Trainer, TRL). The flagship library.

datasets: dataset loading, processing, streaming. Memory-mapped Arrow format. Built-in handling for many public datasets. Lazy operations for huge datasets.

tokenizers: fast Rust implementations of BPE, WordPiece, Unigram. Used by transformers internally; also usable standalone for training custom tokenizers.

Integration: datasets prepares data, tokenizers turns text into IDs, transformers runs the model. Workflow: load model and tokenizer; load dataset; tokenize with dataset.map; train with Trainer. This three-library stack covers most NLP workflows.

Supporting libraries: peft (PEFT), accelerate (multi-GPU and mixed precision), evaluate (metrics), trl (RLHF and DPO), diffusers (diffusion models).

Q7. Describe Hugging Face Pipelines for end-to-end inference.

A pipeline wraps preprocessing, model inference, and postprocessing into one callable. Pick a task; load a default model; call the pipeline on input data.

Supported tasks: text-classification, token-classification (NER), question-answering, summarization, translation, text-generation, fill-mask, feature-extraction, image-classification, object-detection, image-segmentation, speech-recognition, text-to-speech, image-to-text, visual-question-answering, zero-shot-classification, zero-shot-object-detection, and more.

Advantages: quick prototyping (one line to a working model); consistent API across tasks; sensible defaults; handles batching; works with any model on the Hub that supports the task.

When to use: prototyping, experimentation, demos, simple production deployments. For production at scale, prefer direct AutoModel + AutoTokenizer for finer control over batching, device placement, and preprocessing.

Q8. How does Hugging Face Accelerate improve training, and what challenges does it solve?

accelerate provides a thin wrapper that runs the same training code across CPU, single GPU, multi-GPU, multi-node, and TPU without rewriting the code. The same script scales from a laptop to a cluster.

Challenges it addresses: device placement boilerplate, distributed data parallel setup, mixed precision configuration, gradient accumulation, checkpoint handling across processes. accelerate handles these uniformly.

How to use: wrap the model, optimizer, and dataloader in `accelerator.prepare()`; replace `loss.backward()` with `accelerator.backward()`; set a config via `accelerate launch`.

Beyond DDP: integration with DeepSpeed and FSDP for very large models. Megatron-LM integration. Quantization (bitsandbytes) support.

Net: lower the friction of moving a small experiment to a large training run. Reduces bugs from manually rewriting code for different hardware setups.

Q9. How does Hugging Face's transformers library facilitate transfer learning?

Pretrained models: thousands of checkpoints on the Hub. `AutoModel.from_pretrained` downloads and loads any model by name. Includes weights, tokenizer, configuration.

Task heads: pretrained models can be loaded with task-specific heads (`AutoModelForSequenceClassification`, `AutoModelForTokenClassification`, `AutoModelForQuestionAnswering`, `AutoModelForCausalLM`). The head is randomly initialized; the backbone is pretrained.

Trainer: a high-level training API that handles fine-tuning loops, evaluation, logging, checkpointing, mixed precision, distributed training. Integrated with datasets and evaluate.

PEFT integration: peft library plugs in. Wrap a model with `get_peft_model` and a `LoraConfig`; train normally with Trainer. Save and load adapters separately from base.

Typical fine-tuning steps: pick base model and task head; load and preprocess data; configure `TrainingArguments`; instantiate Trainer with model, data, tokenizer, metrics; call `trainer.train()`; save the result; push to the Hub for sharing.

Q10. What role does multi-modality play in the latest LLMs, and how does it enhance functionality?

Modern frontier LLMs (GPT-4o, Claude 3.5, Gemini 1.5/2.0) natively handle text, image, audio, and increasingly video. The single model accepts mixed inputs and may produce mixed outputs.

Enhanced functionality: visual question answering, document understanding, chart and table interpretation, code from screenshots, image generation from text, speech recognition and synthesis, video understanding, generative manipulation.

User experience: natural interaction. Show a photo of a broken appliance; the model identifies it and provides repair instructions. Paste a screenshot of code; the model debugs. Speak; the model responds in your voice.

Architectural patterns: shared embedding space (CLIP-style) for retrieval; cross-attention bridges (Q-Former, Perceiver Resampler); direct token projection (LLaVA, GPT-4V). Modern models combine techniques.

Impact: multimodal is now the default for foundation models. Single-modality models persist for narrow latency- or cost-sensitive uses.

Q11. Implications of rapid LLM advancement for healthcare, education, and content creation?

Healthcare: clinical documentation automation (ambient AI scribes), diagnostic decision support, literature search, drug discovery (protein design with AlphaFold-style models), patient communication. Concerns: regulatory clearance, liability, bias against underrepresented populations, privacy.

Education: personalized tutoring at scale, lesson plan generation, language learning, grading assistance. Concerns: cheating, equity of access, role of teachers, accuracy of explanations.

Content creation: writing assistance, image and video generation, music composition, code generation.
Concerns: copyright and attribution, deepfakes and misinformation, job displacement, homogenization of aesthetics.

Cross-cutting: trust calibration, transparency about AI involvement, governance frameworks. Each domain has unique stakes; thoughtful deployment matters more than raw capability.

Timeline: capabilities are advancing faster than regulatory and societal frameworks. Practitioners should engage actively in shaping deployment patterns.

Q12. What is in-context learning, and why does it work?

In-context learning is the LLM's ability to learn a task from examples provided in the prompt, with no weight updates. The model sees a pattern (input → output, input → output, new input) and produces the corresponding output.

Why it works: during pretraining, the model is exposed to billions of sequences that include similar patterns (lists, structured QA, code with similar functions). Predicting the next token in such sequences requires generalizing the pattern. This generalization is the meta-learning that enables in-context learning at inference.

Empirical findings: in-context learning quality scales with model size; larger models exhibit emergent in-context abilities. Sensitive to example ordering, format, and selection. More examples help up to a point.

Implications: a strong base model can solve many tasks without fine-tuning, just by clever prompting. This is why prompt engineering and RAG are such effective levers without touching model weights.

Q13. What is constitutional AI?

Constitutional AI is an alignment approach where the model is trained to follow a written set of principles ('constitution') rather than relying solely on human preference data.

Process: write the constitution (rules like 'be helpful, be harmless, avoid deception'). Use it in two ways: (1) supervised stage: model critiques and revises its own outputs based on constitutional principles. (2) RL stage: AI-generated preferences using the constitution as a rubric. Train with RLHF on these AI preferences (RLAIF).

Why it matters: scales beyond what human preference data can cover; makes behavior more transparent (you can read the rules); easier to update behavior by editing the constitution.

Limitations: the constitution itself encodes biases; AI judging AI can amplify errors; rigid rule-following can produce unhelpful refusals. Most production systems combine constitutional principles with human preference data.

Q14. What is chain-of-thought prompting and why is it effective?

Chain-of-thought (CoT) prompting asks the model to produce reasoning steps before the final answer. Instead of 'Q: ... A: 42', it produces 'Q: ... Let me think step by step. First... then... so the answer is 42.'

Why it helps: complex reasoning requires intermediate steps. Asking for the final answer directly forces the model to do all reasoning in one forward pass without externalizing intermediate state. CoT lets the model spread reasoning across multiple tokens.

Variants: zero-shot CoT ('let's think step by step'); few-shot CoT (provide examples with reasoning); self-consistency (sample multiple CoT chains and vote); Tree of Thoughts (explore branches and backtrack).

Modern frontier models like o1 and Claude 3.5 with thinking are trained to do extensive chain-of-thought reasoning automatically. They significantly outperform non-reasoning models on math, code, and complex reasoning tasks.

Q15. What is a reasoning model like o1 or Claude with thinking?

A reasoning model is an LLM that, during inference, generates an extensive internal chain-of-thought before producing the final response. The thinking can be hundreds or thousands of tokens long.

How they differ from standard LLMs: trained with RL on reasoning tasks (typically with RLVR using verifiable rewards). The model learns to spend inference compute on thinking rather than just emitting the final answer.

Performance: dramatically better on math (AIME), code (Codeforces), and logic-heavy benchmarks. Trade higher inference cost for higher quality.

User experience: longer latency to first answer but higher accuracy. Some products hide the thinking; others show it. Thinking is sometimes truncated or summarized.

Implications: inference-time compute scaling becomes a lever. A standard model with 10x thinking compute can match a much larger model without thinking. This shifts cost from training to inference.

Q16. What is the difference between continued pretraining and fine-tuning?

Continued pretraining (also called continued training or domain pretraining): extend pretraining with more data, usually from a specific domain. Same self-supervised objective (causal LM or masked LM). Output: a domain-specialized base model that has not been instruction-tuned.

Fine-tuning: usually refers to instruction tuning or task-specific tuning. Supervised loss on (input, target) pairs.

When to use continued pretraining: domain vocabulary and patterns differ significantly from general (medical, legal, code). Want to inject domain knowledge before instruction tuning.

When to use fine-tuning: change model behavior, format, persona. Teach specific tasks. Most enterprise adaptations.

Combination: continued pretraining → instruction fine-tuning → RLHF is the full pipeline. Most teams skip continued pretraining if their domain is well-represented in the base model.

Q17. What is the difference between prompt engineering and prompt optimization?

Prompt engineering: human writes and iterates on prompts. Uses patterns (CO-STAR, few-shot, chain-of-thought) and intuition. Fast for simple tasks; labor-intensive for complex ones.

Prompt optimization: programmatic search over prompt space. Frameworks like DSPy (COPRO, MIPRO), GEPA (evolutionary), TextGrad (text-as-tensor). The framework treats the prompt as a program to be compiled against an evaluator.

When prompt engineering suffices: simple tasks, one-off scripts, exploration. When optimization wins: complex pipelines with multiple LLM calls, where small improvements compound; tasks with a clear automated evaluator; settings where you can afford the optimization runtime cost.

Modern practice: start with prompt engineering for a working baseline; switch to programmatic optimization when the human effort plateaus and an evaluator is available.

Q18. Why does a Mixture of Experts (MoE) model offer compute-efficient scaling?

MoE replaces dense feed-forward layers with many expert subnetworks plus a router. Per token, only a few experts (typically 1-2 out of 8-128) are activated.

Total parameters can be very large (Mixtral 8x7B has 47B total) but active parameters per token are much smaller (about 13B for Mixtral). FLOPs per token scale with active parameters; memory scales with total.

Result: a MoE with N total experts and k active is roughly the inference cost of a dense model with k experts' worth of parameters but the quality of a model with N experts' worth. Net: similar quality to a large dense model at a fraction of the inference compute.

Caveats: total memory remains high (all experts must be in memory). Training is more complex (load balancing across experts, routing stability). Latency may not improve as much as throughput due to MoE-specific overheads.

Examples: Mixtral 8x7B (Mistral), Switch Transformer (Google), GLaM (Google), DeepSeek-V3.

Q19. Explain Retrieval-Augmented Fine-Tuning (RAFT).

RAFT is a fine-tuning paradigm specifically for RAG. Train the model to use retrieved context correctly: cite relevant passages, ignore distractor passages, refuse when context is insufficient.

Training data: (question, context, answer) triples where the context includes both relevant and distractor passages. The target answer references the relevant passages explicitly.

Why it helps: standard SFT data does not teach the model to navigate retrieved context well. Vanilla fine-tuning can make the model worse at RAG by teaching it to rely on parametric memory. RAFT-style data teaches the model to rely on context appropriately.

Impact: improved faithfulness, better refusal calibration when retrieval misses, stronger citation behavior.

Q20. What is the difference between encoder-only, decoder-only, and encoder-decoder models?

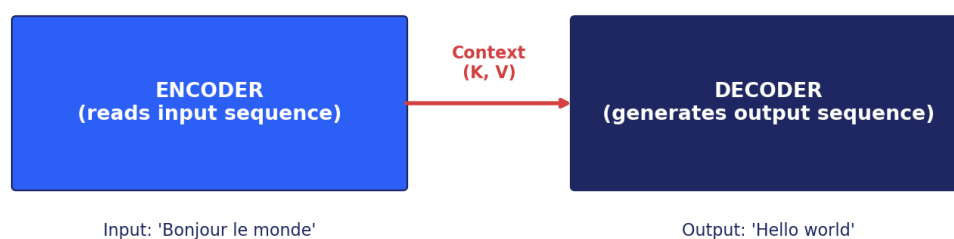
Encoder-only (BERT, RoBERTa, DeBERTa): bidirectional self-attention. Sees full input at once. Produces rich contextual representations. Best for classification, NER, embeddings, similarity. Not naturally generative.

Decoder-only (GPT, LLaMA, Mistral, Claude): causal self-attention (left to right). Generates token by token. Best for generation. Modern instruction-tuned versions handle understanding tasks via prompting.

Encoder-decoder (T5, BART, mT5): encoder reads input bidirectionally; decoder generates output left-to-right while cross-attending to encoder. Strong on translation, summarization, text-to-text tasks. Trickier to train than decoder-only at scale.

Modern trend: decoder-only models dominate due to scaling efficiency and prompt-based versatility. Encoder-only models persist for embeddings and classification at scale where their efficiency wins.

Encoder-decoder transformer (translation example)



Q21. What is sparse vs dense retrieval, and when is each preferred?

Sparse retrieval: based on lexical matching (BM25, TF-IDF, SPLADE). Each document is represented as a sparse vector of term weights. Inverted indexes make queries fast even at scale.

Dense retrieval: based on neural embeddings (Sentence-BERT, BGE, E5, OpenAI ada). Documents and queries are dense vectors; similarity is cosine.

Sparse strengths: exact keyword matching, rare terms (drug names, identifiers), no training required, fast, mature infrastructure.

Dense strengths: semantic similarity, paraphrase handling, cross-lingual support (with multilingual embedders).

Hybrid: combine both via reciprocal rank fusion or weighted scoring. Almost always beats either alone on diverse query distributions. The production default in 2025.

When to prefer one: pure technical search (sparse); conversational search over concepts (dense); production search at any scale (hybrid).

Chapter 12

Supplementary Topics, Formulas, and Deep Dives

This chapter covers 13 questions on supplementary topics, formulas, and deep dives.

Q1. Mathematical formulas reference: perplexity, attention, TF-IDF, triplet loss

This question gathers the most-asked formulas in one place so that you can recall them under interview pressure.

Perplexity. For a sequence $w_1, \dots, w_N$, perplexity is $P = 2^{(-1/N \cdot \sum_i \log_2 P(w_i | w_1, \dots, w_{i-1}))}$. Lower is better; equivalent to the exponential of average per-token cross-entropy.

Scaled dot-product attention. $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$. The $\sqrt{d_k}$ scaling keeps softmax in a usable range as the head dimension grows; without it, large dot products push softmax into saturation and gradients vanish.

Multi-head attention. Run h attention computations in parallel with different W_Q, W_K, W_V projections each of dimension d_{model} / h ; concatenate; project back with W_O .

Term Frequency: $\text{TF}(t, d) = \text{count}(t \text{ in } d) / \text{total terms in } d$.

Inverse Document Frequency: $\text{IDF}(t) = \log(N / \text{df}(t))$, where N is total documents and $\text{df}(t)$ is documents containing t .

TF-IDF(t, d) = $\text{TF}(t, d) \cdot \text{IDF}(t)$.

Triplet loss: $L(A, P, N) = \max(0, d(A, P) - d(A, N) + \text{margin})$, where d is a distance (Euclidean or cosine). The margin enforces minimum separation between matched and mismatched pairs.

InfoNCE loss: $L = -\log(\exp(\text{sim}(a, p)/\tau) / \sum_j \exp(\text{sim}(a, j)/\tau))$, where the sum is over the positive plus all in-batch negatives; τ is the temperature.

BM25 score: $\sum \text{IDF}(q_i) \cdot (f(q_i, d)(k+1)) / (f(q_i, d) + k(1-b+b \cdot |d|/\text{avgdl}))$. Saturates with term frequency (controlled by k) and normalizes for document length (controlled by b).

Cross-entropy loss: $-\sum y_i \log(p_i)$, where y is the one-hot label and p is the predicted probability. Reduces to $-\log(p_{\text{true_class}})$ for one-hot targets.

KL divergence: $D_{\text{KL}}(p || q) = \sum p(x) \log(p(x) / q(x))$. Asymmetric. Used as a regularizer in PPO (keep policy close to reference) and in VAEs (latent prior matching).

Q2. Beyond MLM and CLM, what other pretraining objectives have been used for LLMs?

Permutation Language Modeling (PLM). Introduced in XLNet. Train the model to predict tokens in a randomly permuted order so that, for any given target token, the model sees a mixture of left and right context (without using masking that breaks BERT's pretrain/fine-tune consistency). Blends the bidirectional benefit of MLM with the autoregressive property of CLM. More complex to train than either.

Denoising Auto-encoding (span corruption). Used in T5 and BART. Take a sentence, corrupt it by masking spans of tokens (T5) or by general noise functions like deletion, infilling, sentence permutation, and rotation (BART). Train the decoder to reconstruct the original. Generalizes MLM from single tokens to contiguous spans; aligns naturally with text-to-text framing.

Replaced Token Detection (ELECTRA). A small generator proposes replacements for masked tokens; a larger discriminator classifies each token as 'original' or 'replaced'. More sample-efficient than MLM because every token contributes to the loss.

Prefix Language Modeling. Bidirectional attention over a prefix, causal attention over a suffix. Used in some encoder-decoder hybrids to combine BERT-style understanding with GPT-style generation.

Why this matters in interviews: knowing the menu of objectives signals that you understand the design space, not just the BERT/GPT binary.

Q3. What efficient or memory-augmented transformer variants exist for long context?

The vanilla transformer's $O(n^2)$ self-attention is the binding constraint for long sequences. Several families relax it.

Sparse / windowed attention. Longformer combines sliding-window local attention with a few designated global tokens. BigBird adds random attention on top. Each token attends to $O(n)$ tokens instead of $O(n^2)$. Used in long-document tasks (legal, biomedical, document QA).

Linearized attention. Linformer projects K and V to a lower-rank k_{proj} before attention, giving $O(n \cdot k)$ complexity. Performer approximates softmax with random feature maps. Reformer replaces softmax with LSH-based bucketing. Each trades exact attention for asymptotic improvement; quality varies by task.

Memory-enhanced models. Transformer-XL caches hidden states from the previous segment and lets the current segment attend to them. Equivalent to an extended context window without re-computing. Compressive Transformer compresses old states rather than dropping them.

State-space models. Mamba and S4 use selective state-space updates with sub-quadratic time. Competitive with transformers on long-context benchmarks at significantly lower cost.

Retrieval augmentation as an alternative. Instead of extending the context window, retrieve relevant chunks per query (RAG). Often the right answer when you can structure your problem this way.

Position-encoding tricks for length extrapolation. RoPE with NTK-aware or YaRN scaling, ALiBi position bias. Let a model trained at short context generalize to longer contexts without retraining.

Q4. Beyond Recall@k, what quantitative metrics evaluate embedding quality?

Recall@k, Precision@k, MRR, NDCG cover retrieval quality. Beyond those, several intrinsic and clustering-based metrics matter for evaluating embeddings as embeddings.

Cosine similarity correlation with human judgments. Compute the cosine similarity of an embedding pair and correlate it with human-labeled similarity (Spearman or Pearson). Standard for sentence similarity (STS-B). Spearman is the de facto reporting metric for embedding model leaderboards (MTEB).

Average Pairwise Distance (APD). Mean pairwise distance within a cluster (anisotropy proxy at corpus scale). Lower APD across unrelated documents = healthier (less anisotropic) embedding space.

Silhouette Score. For each point, $(b - a) / \max(a, b)$, where a is mean distance to same-cluster points and b is mean distance to nearest other cluster. Range $[-1, 1]$. Higher = better separation. Useful when you have ground-truth clusters.

Davies-Bouldin Index. Ratio of within-cluster scatter to between-cluster separation, averaged over clusters. Lower = better. Used in topic-modeling evaluation of embeddings.

Calinski-Harabasz Index. Variance-ratio criterion. Higher = better.

Analogy benchmarks. Classic 'king - man + woman $\approx$ queen' style. Useful as a sanity check; less informative than retrieval benchmarks on modern embedders.

MTEB. Massive Text Embedding Benchmark. 50+ tasks across classification, clustering, retrieval, reranking, similarity. The comprehensive standard.

Q5. Explain the main types of embedding quantization and their tradeoffs.

Quantization reduces the precision of stored embeddings to save memory at some cost to recall. Three main families.

Uniform (scalar) quantization. Each dimension is quantized independently to a fixed range, typically int8 or int4. Cheapest to implement. Embedding $\times$ dimension $\times$ bits/8 bytes per vector. Loss usually under 1 percentage point on retrieval. Default first step.

Non-Uniform quantization. The quantization grid adapts to the value distribution. K-means quantization (each dimension's values split into k clusters) and learned quantizers (mu-law, NF4 from QLoRA) fit this category. Better quality at the same bit rate; more complex to compute.

Product Quantization (PQ). Split each vector into m subvectors. Train k codewords per subvector via k -means. Replace each subvector with its nearest codeword index. Storage: $m \cdot \log_2(k)$ bits per vector, often 32x smaller than fp32. Used by FAISS at billion-scale. Approximate distance is computed by table lookup. Variants: OPQ (rotate before quantizing), Residual PQ (multi-stage), and Inverted File + PQ for the standard FAISS index.

Binary embeddings. One bit per dimension. Hamming distance instead of cosine. 32x compression. Usable for first-stage retrieval; production systems rerank with full-precision embeddings on the candidate set.

Choosing: start with int8 if memory is not the binding constraint; move to PQ when memory is. Always benchmark the recall hit on your own data.

Q6. What advanced strategies exist for handling rare words and OOV terms?

Subword tokenization (BPE, WordPiece, SentencePiece) is the modern default and eliminates most OOV issues. The remaining failure cases are rare and domain-specific; the techniques below cover them.

FastText subword embeddings. Represent each word as a sum of its character n -gram embeddings. Even an unseen word receives a meaningful vector from its character pieces.

Retrofitting with external lexical resources. Modify pretrained embeddings to bring semantically related words (from WordNet, PPDB, or a domain ontology) closer together. Cheap post-hoc fix that improves embedding quality on domains where lexical relations matter more than corpus statistics.

Contextual averaging. For a very rare word, average the contextual embeddings of its surrounding context words from a large corpus. Gives a rough estimate when the word itself has too few occurrences for a stable embedding.

Phonetic or morphological fallback (Soundex, Metaphone, Levenshtein-based matching). When a rare word does not decompose well into known subwords, match it phonetically to a known word and use that embedding. Useful for noisy text (OCR, ASR transcripts).

Embedding imputation. For a missing token, use the mean of similar tokens' embeddings, the embedding of the lemma, or the embedding of the closest known word.

Domain tokenizer extension. Add domain-specific tokens (drug names, chemical formulas, ticker symbols, code identifiers) to the tokenizer; continue pretraining briefly so the new tokens learn good embeddings. Standard pattern for production domain adaptation.

Few-shot / zero-shot embedding inference. Modern LLMs can produce a contextual embedding for any string at inference; no separate OOV handling needed at the model level.

Q7. Deep dive: image synthesis losses including Total Variation and Style/Content losses

Pixel-wise losses (L1, L2). Simple per-pixel difference. Easy to compute, prone to blurry outputs because the optimal solution under L2 is the mean of plausible images. Effective as a baseline or as one term in a composite loss.

Perceptual loss (LPIPS, VGG-feature). Compare generated and target in a frozen pretrained network's feature space. Much better correlated with human perception than pixel-wise loss. Standard in super-resolution, style transfer, and image-to-image translation.

Adversarial loss. GAN discriminator trained to distinguish real from generated; generator trained to fool it. Pushes outputs onto the manifold of real images; produces sharper results than pixel-wise losses. Hard to stabilize.

Style and Content Loss (Gatys et al.). For style transfer, decompose: content loss is feature-map L2 against the content image at deep layers; style loss is the L2 between Gram matrices of feature maps at shallow layers (Gram matrix captures texture statistics independent of spatial position). Total loss is a weighted sum; the weight controls content-vs-style emphasis.

Total Variation (TV) loss. Penalizes the sum of absolute differences between neighboring pixels: $TV(x) = \sum |x_{\{i+1,j\}} - x_{\{i,j\}}| + |x_{\{i,j+1\}} - x_{\{i,j\}}|$. Encourages smoothness. Reduces checkerboard artifacts and noise in generated images. Used as a regularizer in super-resolution and inpainting.

Diffusion model losses. Predict the noise added to a clean image at random timesteps, minimizing MSE between predicted and actual noise. Simple and stable. v-prediction and EDM parameterizations are variants. Latent diffusion (Stable Diffusion) applies this in the VAE latent space, not pixel space.

Composite recipes. Production image generation uses combinations: for super-resolution, pixel + perceptual + adversarial + TV. For style transfer, content + style + optional TV. Tuning the weights is empirical.

Q8. What are common diffusion samplers and how do they differ?

Diffusion models generate by iteratively denoising a random latent. The sampler is the integration scheme used to step through the reverse-diffusion ODE/SDE.

DDPM (denoising diffusion probabilistic models). The original ancestral sampler. Stochastic. Needs many steps (~1000) for high quality.

DDIM (denoising diffusion implicit models). Deterministic, allows much fewer steps (20-50). Same trained model, different sampling. Standard default for Stable Diffusion.

PNM (pseudo numerical methods for diffusion models). Uses a linear multistep method (Adams-Bashforth-Moulton style) for higher accuracy at fewer steps. Faster than DDIM at similar quality.

DPM-Solver and DPM-Solver++. Use exponential integrators to take very large steps. 10-20 steps often suffice. Common in production image generation services.

Euler, Euler ancestral, Heun. ODE solvers used by k-diffusion / Karras-style schedulers. Different tradeoffs for stochasticity and step count.

UniPC. Universal Predictor-Corrector method. Very few steps (~10) for similar quality to many-step samplers.

Practical guidance: DPM-Solver++ at 20-30 steps is a strong default for SDXL and similar. Lower steps = faster but less detailed; higher steps = diminishing returns.

Q9. What metrics specifically evaluate multimodal RAG quality?

Text generation metrics. BLEU, ROUGE, METEOR for surface overlap; BERTScore for semantic similarity. Inherited from text-only evaluation; necessary but not sufficient for multimodal.

Image captioning metrics. CIDEr (Consensus-based Image Description Evaluation) weights n-grams by their TF-IDF across reference captions. Higher = better. SPICE evaluates the semantic propositional content (objects, attributes, relations) extracted from captions; captures meaning beyond surface form.

Vision-language alignment. CLIP Score: cosine similarity between the CLIP image embedding and the CLIP text embedding of the caption. Cheap and language-agnostic. VLP Score: similar with a vision-language pretraining model.

Multimodal Relevance Score. Combine per-modality retrieval scores into a single measure. Approaches: rank fusion, weighted sum, or joint reranking with a cross-modal scorer.

Faithfulness across modalities. Verify that generated text and generated images do not contradict each other. NLI-style checks (text claims vs image content) and vision-language entailment.

LMM-as-a-Judge. Use a strong multimodal model (GPT-4o, Gemini) to rate outputs on a rubric. Extends LLM-as-a-judge to multimodal. Same biases apply (position, verbosity, self-preference); control with randomization and different judge models.

Human evaluation. Still gold standard for nuanced qualities: creativity, aesthetic quality, factual grounding across modalities.

Modality-specific precision and recall. Compute per-modality so a weakness in one modality is not masked by strength in another.

Q10. What specific metrics and frameworks evaluate hallucination in LLMs and RAG?

FactCC. Trained classifier that scores whether a generated sentence is consistent with a source document. Originally for summarization. Use as a faithfulness metric on RAG outputs.

FEVER (Fact Extraction and Verification). Benchmark and pipeline for verifying claims against a Wikipedia-derived knowledge base. Three-way classification: supported, refuted, not enough info. Adapted for LLM output verification.

QAGS (Question-Answer Generation and Summarization). Generate questions from the candidate text; answer them using the source. Mismatch indicates hallucination. More robust than n-gram-overlap metrics.

FactScore. Decompose generated text into atomic facts. Verify each fact against a knowledge source. Report the fraction supported. Strong correlation with human judgment.

TruthfulQA. Benchmark of 817 questions designed to elicit common false beliefs (urban legends, misconceptions). Tests whether the model parrots common falsehoods or knows the truth.

SelfCheckGPT. Sample multiple responses; compare consistency. Inconsistent answers signal hallucination. No external knowledge base needed.

Confidence scoring and thresholding. Use the model's own predicted probability or a calibrated estimate; refuse or flag low-confidence outputs. Calibration techniques: temperature scaling, verbalized confidence, ensemble agreement.

Retrieval-grounded verification. For RAG, check that each claim in the output is entailed by the retrieved passages (NLI-based). Block contradictions; flag neutral verdicts.

Citation accuracy. The model cites a source; verify that the cited source actually contains the claim. Common metric in agentic RAG and grounded generation.

Q11. What is ImageBind, and how does it unify multimodal embeddings?

ImageBind (Meta, 2023) is a model that learns a single embedding space across six modalities: images, text, audio, depth, thermal, and IMU (inertial measurement unit) data.

Key idea: pair every non-image modality with images during training. Images become the binding modality. (image, audio) pairs, (image, text) pairs, (image, depth) pairs and so on. Train with contrastive loss. The result: even modalities never paired with each other (e.g., audio and text) end up aligned through their shared alignment with images.

Why it matters: cross-modal retrieval and generation without explicit pairings between every modality combination. Search audio with text, find images matching a thermal signature, drive image generation from sound.

Use cases: multimodal search engines that accept any modality as query; multimodal RAG systems; generative models conditioned on non-traditional inputs.

Successors: AudioCLIP, LanguageBind, OneLLM extend the pattern. The space is active and competitive.

Q12. When and how do you use phonetic similarity (Soundex) in NLP?

Soundex is an algorithm that maps words to a short code based on how they sound. Words that sound alike map to the same code. Robert and Rupert map to R163; Smith and Smythe to S530.

Algorithm: keep the first letter. Encode the rest by category: BFPV → 1, CGJKQSZ → 2, DT → 3, L → 4, MN → 5, R → 6. Drop vowels and HW. Drop consecutive duplicates. Pad with zeros to 4 characters.

Variants. Metaphone and Double Metaphone are more linguistically principled (handle silent letters, common digraphs). Caverphone for names with New Zealand accents. NYSIIS for surname matching.

Use cases. Fuzzy name matching in databases (especially genealogical, medical, customer records). OCR error correction when subword tokenizers fail. Voice-to-text fallback when ASR produces near-homophone errors. Domain-specific entity resolution where spelling varies (Mohammed, Muhammad, Mohamed).

When to use: as a fallback when exact-match and subword-based matching fail. Combine with edit distance and embedding similarity in a layered matching pipeline.

Limits: English-centric. Loses meaningful information for many languages. Useless for tonal languages and logographic scripts.

Q13. What is modality dropout, and why is it useful in multimodal training?

Modality dropout randomly hides one or more modalities during training, forcing the model to learn useful representations from any subset of available modalities.

Why it helps. Without modality dropout, the model can overfit to the most informative modality, ignoring others. With modality dropout, the model must produce useful outputs even when its favorite modality is unavailable. The trained model is robust to missing modalities at inference (a video without audio, an image without caption).

Implementation. For each training example, sample a binary mask over modalities (with some probability of including each). Zero out or mask the embeddings of the dropped modalities. Adjust the attention layers so they ignore zeroed positions.

Hyperparameter. Typical dropout probability per modality: 0.2 to 0.4. Tune on a validation set with simulated missing modalities.

Related techniques. Modality-specific learning rates to prevent one modality from dominating. Gradient balancing (uncertainty weighting, GradNorm) to balance contributions during training.

Appendix A: Recommended Reading

- Attention Is All You Need (Vaswani et al., 2017).
- BERT: Pre-training of Deep Bidirectional Transformers (Devlin et al., 2018).
- Language Models are Few-Shot Learners (Brown et al., 2020). GPT-3.
- LoRA: Low-Rank Adaptation of Large Language Models (Hu et al., 2021).
- QLoRA: Efficient Finetuning of Quantized LLMs (Dettmers et al., 2023).
- Direct Preference Optimization (Rafailov et al., 2023).
- Training Compute-Optimal Large Language Models (Hoffmann et al., 2022). Chinchilla.
- Retrieval-Augmented Generation for Knowledge-Intensive NLP (Lewis et al., 2020).
- DSPy: Compiling Declarative LM Calls into Self-Improving Pipelines (Khattab et al., 2023).
- RAGAS: Automated Evaluation of Retrieval Augmented Generation (Es et al., 2023).
- ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction (Khattab and Zaharia, 2020).
- From Local to Global: A GraphRAG Approach (Edge et al., 2024).
- LLaMA: Open and Efficient Foundation Language Models (Touvron et al., 2023).
- DeepSeekMath: Pushing the Limits of Mathematical Reasoning (Shao et al., 2024). GRPO.
- Mixture of Experts Meets Instruction Tuning (Shen et al., 2024).

Appendix B: Suggested Interview Preparation Path

For ML/AI Engineer roles (3-5 years experience)

- Week 1: Chapters 1, 2, 3 (foundations).
- Week 2: Chapters 4, 5 (LLMs and embeddings).
- Week 3: Chapters 6, 7, 8 (RAG, fine-tuning, vector DB).
- Week 4: Chapters 9, 10, 11 (LLMOps, evaluation, ethics).

Build a small project end-to-end (RAG over a personal document collection, or a fine-tuned classifier) to ground the concepts.

For Senior/Staff/Principal AI roles

- Focus on tradeoff discussions: when does each technique fail?
- System design: production architectures for chat, search, RAG, agents.
- Leadership: how do you choose models, evaluate vendors, manage costs?
- Ethics and risk: bias, hallucination, safety, governance.

Appendix C: Glossary of Key Terms

ANN: Artificial Neural Network or Approximate Nearest Neighbor depending on context.

Attention: Mechanism that weights inputs by relevance to a query.

Backpropagation: Algorithm for computing gradients in neural networks.

BPE: Byte Pair Encoding, a subword tokenization algorithm.

Chinchilla scaling: Compute-optimal: 20 tokens per parameter.

CoT: Chain-of-Thought prompting / reasoning.

CLIP: Contrastive Language-Image Pretraining.

DPO: Direct Preference Optimization.

Embedding: Dense vector representation of an object.

FAISS: Facebook AI Similarity Search library.

GRPO: Group Relative Policy Optimization.

HNSW: Hierarchical Navigable Small World graph for ANN search.

KL: Kullback-Leibler divergence.

LLM: Large Language Model.

LoRA: Low-Rank Adaptation, a PEFT method.

MoE: Mixture of Experts.

PEFT: Parameter-Efficient Fine-Tuning.

PPO: Proximal Policy Optimization.

qLoRA: Quantized LoRA: 4-bit base with 16-bit adapters.

RAG: Retrieval-Augmented Generation.

RLHF: Reinforcement Learning from Human Feedback.

RLVR: Reinforcement Learning with Verifiable Rewards.

RoPE: Rotary Position Embeddings.

SFT: Supervised Fine-Tuning.

ViT: Vision Transformer.

About the Author

Pavan R

Pavan R is the lead instructor at Proxiant Academy. He designs and delivers the academy's curriculum across AI fundamentals, agents, fine-tuning, and retrieval-augmented generation. His work focuses on translating frontier AI techniques into reliable engineering practice for teams shipping production systems.

Colophon

AI, ML, and Generative AI Interview Questions

First Edition, Revision 1.1, 2026

Set in Calibri 11pt body, navy chapter titles.

Diagrams produced with matplotlib. Layout assembled with python-docx.

PDF generated via LibreOffice headless conversion.

Published by Proxiant Academy. Reference identifier: PA-AIMLIQ-2026-01.

© 2026 Proxiant Academy. All rights reserved.